

INN Hotels Project

Context

A significant number of hotel bookings are called-off due to cancellations or no-shows. The typical reasons for cancellations include change of plans, scheduling conflicts, etc. This is often made easier by the option to do so free of charge or preferably at a low cost which is beneficial to hotel guests but it is a less desirable and possibly revenue-diminishing factor for hotels to deal with. Such losses are particularly high on last-minute cancellations.

The new technologies involving online booking channels have dramatically changed customers' booking possibilities and behavior. This adds a further dimension to the challenge of how hotels handle cancellations, which are no longer limited to traditional booking and guest characteristics.

The cancellation of bookings impact a hotel on various fronts:

- * Loss of resources (revenue) when the hotel cannot resell the room.
- * Additional costs of distribution channels by increasing commissions or paying for publicity to help sell these rooms.
- * Lowering prices last minute, so the hotel can resell a room, resulting in reducing the profit margin.
- * Human resources to make arrangements for the guests.

Objective

The increasing number of cancellations calls for a Machine Learning based solution that can help in predicting which booking is likely to be canceled. INN Hotels Group has a chain of hotels in Portugal, they are facing problems with the high number of booking cancellations and have reached out to your firm for data-driven solutions. You as a data scientist have to analyze the data provided to find which factors have a high influence on booking cancellations, build a predictive model that can predict which booking is going to be canceled in advance, and help in formulating profitable policies for cancellations and refunds.

Data Description

The data contains the different attributes of customers' booking details. The detailed data dictionary is

given below.

Data Dictionary

- * Booking_ID: unique identifier of each booking
- * no_of_adults: Number of adults
- * no_of_children: Number of Children
- * no_of_weekend_nights: Number of weekend nights (Saturday or Sunday) the guest stayed or booked to stay at the hotel
- * no_of_week_nights: Number of week nights (Monday to Friday) the guest stayed or booked to stay at the hotel
- * type_of_meal_plan: Type of meal plan booked by the customer:
 - * Not Selected – No meal plan selected
 - * Meal Plan 1 – Breakfast
 - * Meal Plan 2 – Half board (breakfast and one other meal)
 - * Meal Plan 3 – Full board (breakfast, lunch, and dinner)
- * required_car_parking_space: Does the customer require a car parking space? (0 - No, 1- Yes)
- * room_type_reserved: Type of room reserved by the customer. The values are ciphered (encoded) by INN Hotels.
- * lead_time: Number of days between the date of booking and the arrival date
- * arrival_year: Year of arrival date
- * arrival_month: Month of arrival date
- * arrival_date: Date of the month
- * market_segment_type: Market segment designation.
- * repeated_guest: Is the customer a repeated guest? (0 - No, 1- Yes)
- * no_of_previous_cancellations: Number of previous bookings that were canceled by the customer prior to the current booking
- * no_of_previous_bookings_not_canceled: Number of previous bookings not canceled by the customer prior to the current booking
- * avg_price_per_room: Average price per day of the reservation; prices of the rooms are dynamic. (in euros)
- * no_of_special_requests: Total number of special requests made by the customer (e.g. high floor, view from the room, etc)
- * booking_status: Flag indicating if the booking was canceled or not.

Importing necessary libraries and data

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
pd.set_option("display.max_columns", None)
pd.set_option("display.max_rows", 200)
pd.set_option("display.float_format", lambda x: "%.5f" % x)
```

```

from sklearn.model_selection import train_test_split
import statsmodels.stats.api as sms
from statsmodels.stats.outliers_influence import variance_inflation_factor
import statsmodels.api as sm
from statsmodels.tools.tools import add_constant
from sklearn.tree import DecisionTreeClassifier
from sklearn import tree
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import (
    f1_score,
    accuracy_score,
    recall_score,
    precision_score,
    confusion_matrix,
    roc_auc_score,
    precision_recall_curve,
    roc_curve,
    make_scorer,
)

import warnings
warnings.filterwarnings("ignore")

from statsmodels.tools.sm_exceptions import ConvergenceWarning
warnings.simplefilter("ignore", ConvergenceWarning)

```

Note: After running the above cell, kindly restart the notebook kernel and run all cells sequentially from the start again.

```

In [2]: from google.colab import drive
drive.mount('/content/drive')

df = pd.read_csv('/content/drive/MyDrive/Colab Notebooks/INNHotelsGroup.csv')

df.head()

```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

Out[2]:

	Booking_ID	no_of_adults	no_of_children	no_of_weekend_nights	no_of_we
0	INN00001	2	0	1	2
1	INN00002	2	0	2	3
2	INN00003	1	0	2	1
3	INN00004	2	0	0	2
4	INN00005	2	0	1	1

Data Overview

- Observations
- Sanity checks

In [3]: `df.tail()`

Out[3]:

	Booking_ID	no_of_adults	no_of_children	no_of_weekend_nights	no_o
36270	INN36271	3	0	2	6
36271	INN36272	2	0	1	3
36272	INN36273	2	0	2	6
36273	INN36274	2	0	0	3
36274	INN36275	2	0	1	2

In [4]: `df.shape`

Out[4]: (36275, 19)

In [5]: `df.describe()`

Out[5]:

	no_of_adults	no_of_children	no_of_weekend_nights	no_of_week_nights
count	36275.00000	36275.00000	36275.00000	36275.00000
mean	1.84496	0.10528	0.81072	2.20430
std	0.51871	0.40265	0.87064	1.41090
min	0.00000	0.00000	0.00000	0.00000
25%	2.00000	0.00000	0.00000	1.00000
50%	2.00000	0.00000	1.00000	2.00000
75%	2.00000	0.00000	2.00000	3.00000
max	4.00000	10.00000	7.00000	17.00000

In [6]: `df.isnull().sum()`

Out[6]:

	0
Booking_ID	0
no_of_adults	0
no_of_children	0
no_of_weekend_nights	0
no_of_week_nights	0
type_of_meal_plan	0
required_car_parking_space	0
room_type_reserved	0
lead_time	0
arrival_year	0
arrival_month	0
arrival_date	0
market_segment_type	0
repeated_guest	0
no_of_previous_cancellations	0
no_of_previous_bookings_not_canceled	0
avg_price_per_room	0
no_of_special_requests	0
booking_status	0

dtype: int64

In [7]: df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 36275 entries, 0 to 36274
Data columns (total 19 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Booking_ID                            36275 non-null  object
1   no_of_adults                          36275 non-null  int64
2   no_of_children                        36275 non-null  int64
3   no_of_weekend_nights                  36275 non-null  int64
4   no_of_week_nights                     36275 non-null  int64
5   type_of_meal_plan                     36275 non-null  object
6   required_car_parking_space            36275 non-null  int64
7   room_type_reserved                    36275 non-null  object
```

```

8    lead_time      36275 non-null  int64
9    arrival_year   36275 non-null  int64
10   arrival_month  36275 non-null  int64
11   arrival_date   36275 non-null  int64
12   market_segment_type  36275 non-null  object
13   repeated_guest  36275 non-null  int64
14   no_of_previous_cancellations  36275 non-null  int64
15   no_of_previous_bookings_not_canceled  36275 non-null  int64
16   avg_price_per_room  36275 non-null  float64
17   no_of_special_requests  36275 non-null  int64
18   booking_status  36275 non-null  object
dtypes: float64(1), int64(13), object(5)
memory usage: 5.3+ MB

```

```
In [8]: df.duplicated().sum()
```

```
Out[8]: np.int64(0)
```

```
In [9]: df= df.drop(['Booking_ID'], axis=1)
```

```
In [10]: df.head()
```

```
Out[10]:
```

	no_of_adults	no_of_children	no_of_weekend_nights	no_of_week_nights	ty
0	2	0	1	2	Me
1	2	0	2	3	Nc
2	1	0	2	1	Me
3	2	0	0	2	Me
4	2	0	1	1	Nc

Exploratory Data Analysis (EDA)

- EDA is an important part of any project involving data.
- It is important to investigate and understand the data better before building a model with it.
- A few questions have been mentioned below which will help you approach the analysis in the right manner and generate insights from the data.
- A thorough analysis of the data, in addition to the questions mentioned below, should be done.

Leading Questions:

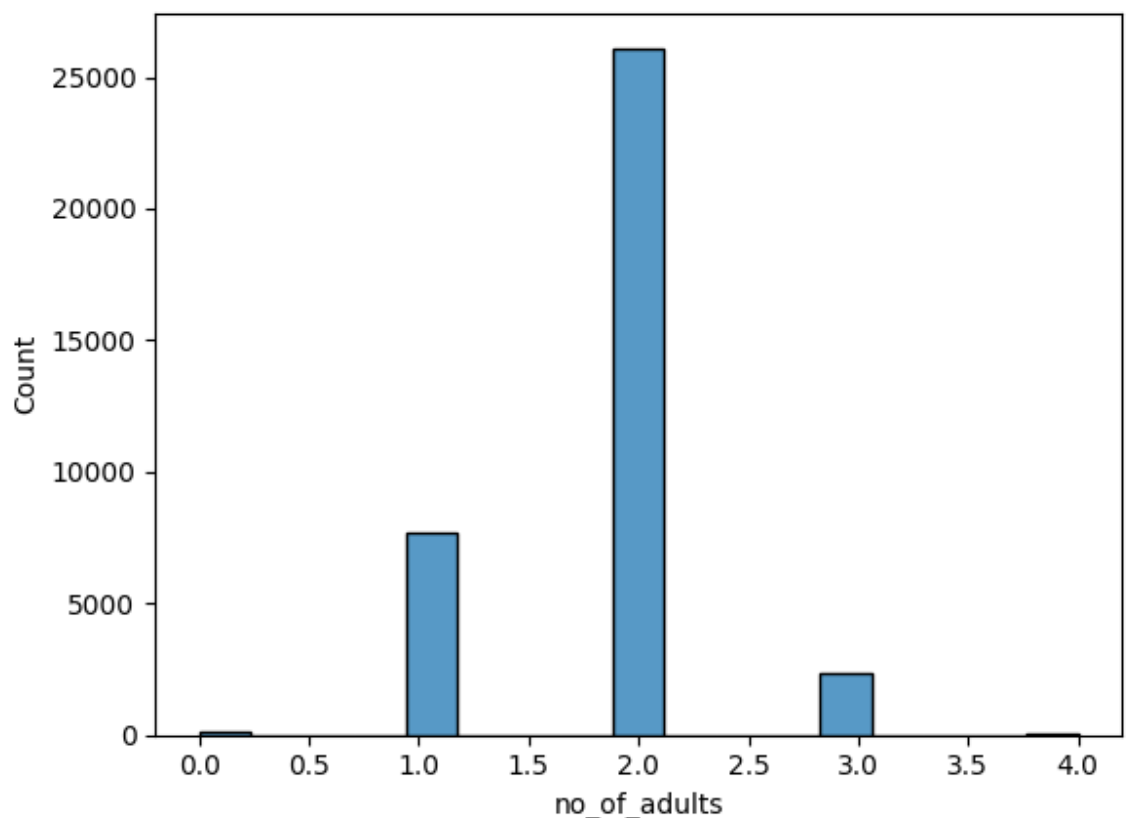
1. What are the busiest months in the hotel?
2. Which market segment do most of the guests come from?

3. Hotel rates are dynamic and change according to demand and customer demographics. What are the differences in room prices in different market segments?
4. What percentage of bookings are canceled?
5. Repeating guests are the guests who stay in the hotel often and are important to brand equity. What percentage of repeating guests cancel?
6. Many guests have special requirements when booking a hotel room. Do these requirements affect booking cancellation?

Univariate Analysis

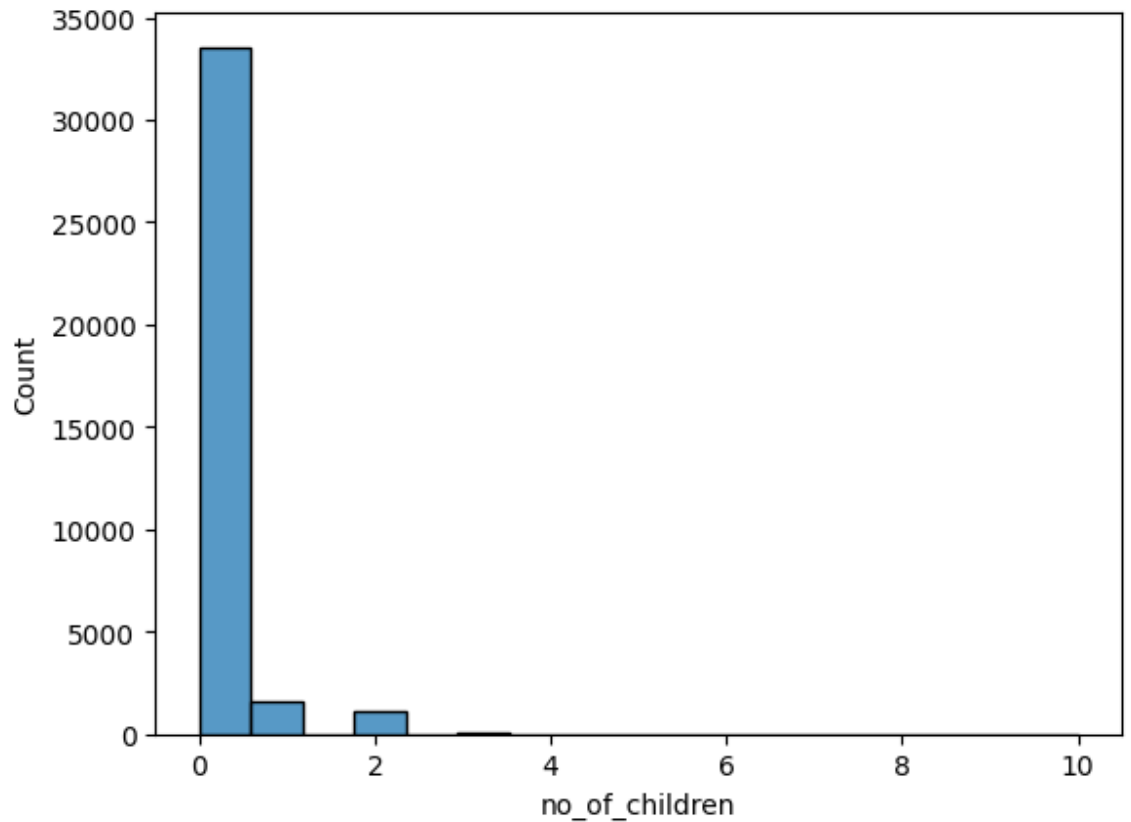
```
In [11]: sns.histplot(df['no_of_adults'])
```

```
Out[11]: <Axes: xlabel='no_of_adults', ylabel='Count'>
```



```
In [12]: sns.histplot(df['no_of_children'])
```

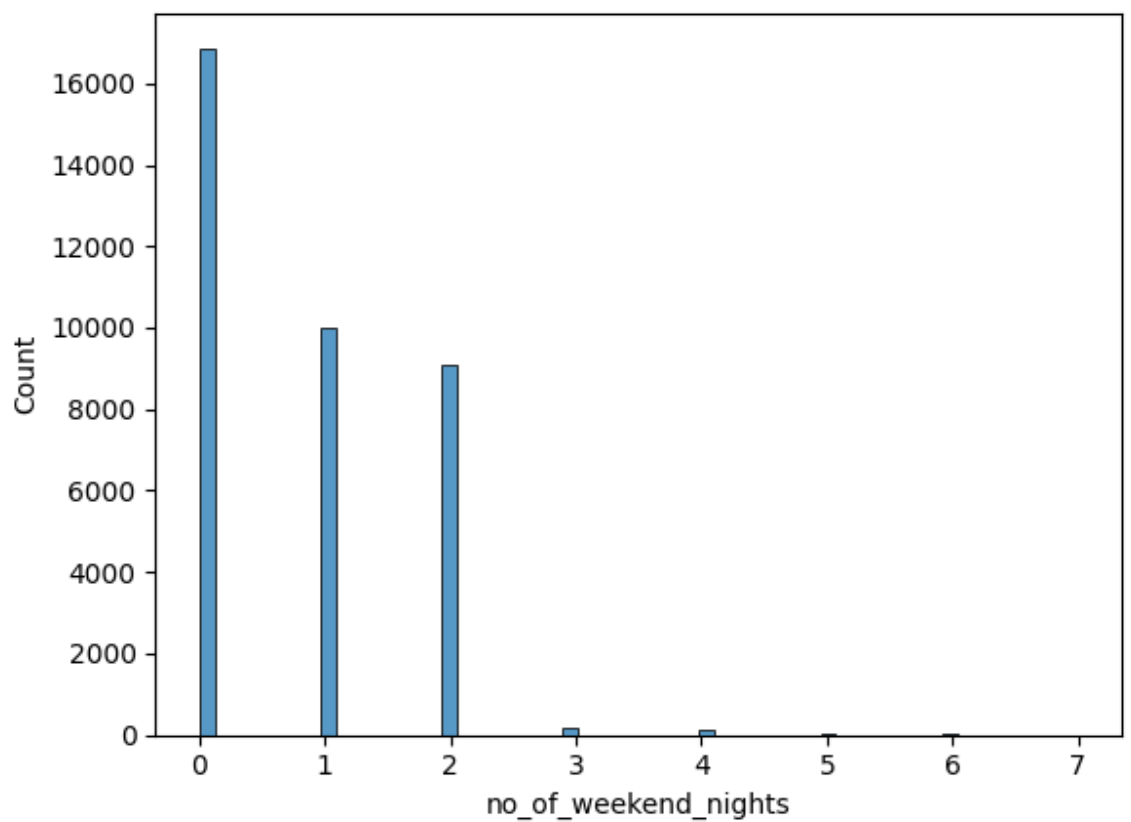
```
Out[12]: <Axes: xlabel='no_of_children', ylabel='Count'>
```



```
In [13]: sns.histplot(df['no_of_weekend_nights'])
```

[13]:

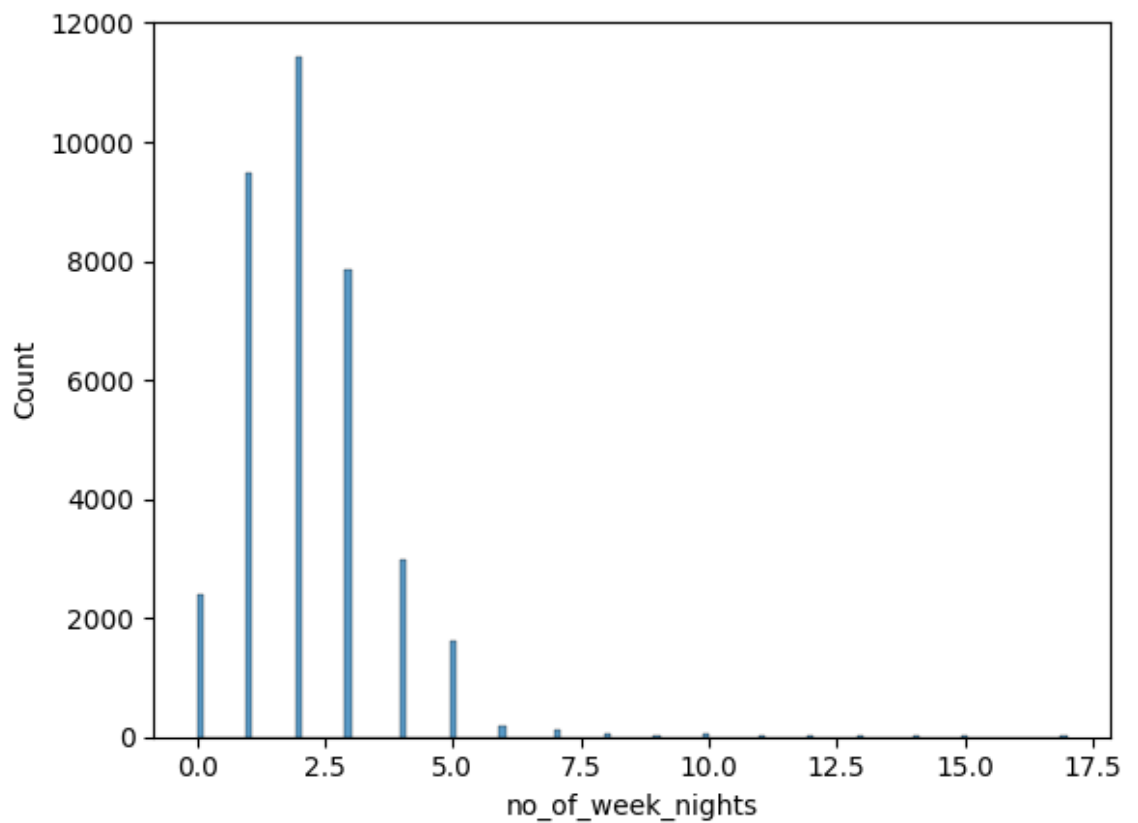
```
Out[13]: <Axes: xlabel='no_of_weekend_nights', ylabel='Count'>
```



```
In sns.histplot(df['no_of_week_nights'])
```

[14]:

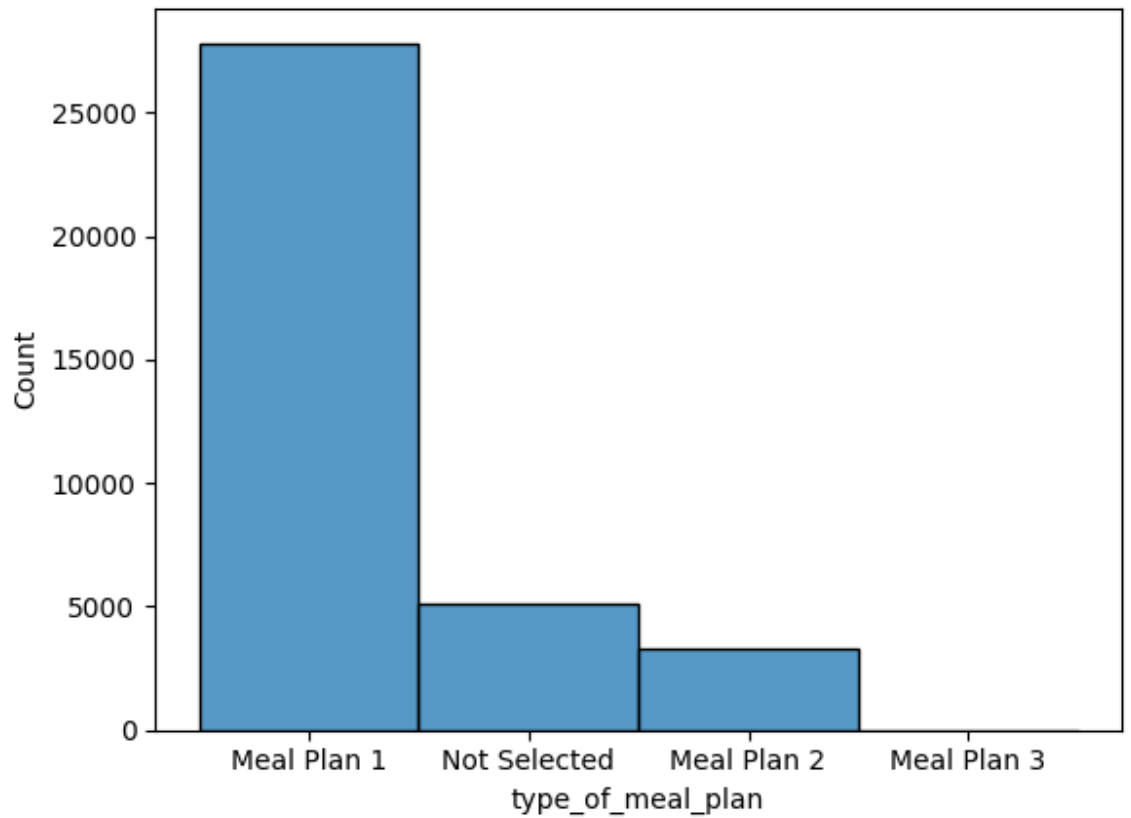
```
Out[14]: <Axes: xlabel='no_of_week_nights', ylabel='Count'>
```



```
In sns.histplot(df['type_of_meal_plan'])
```

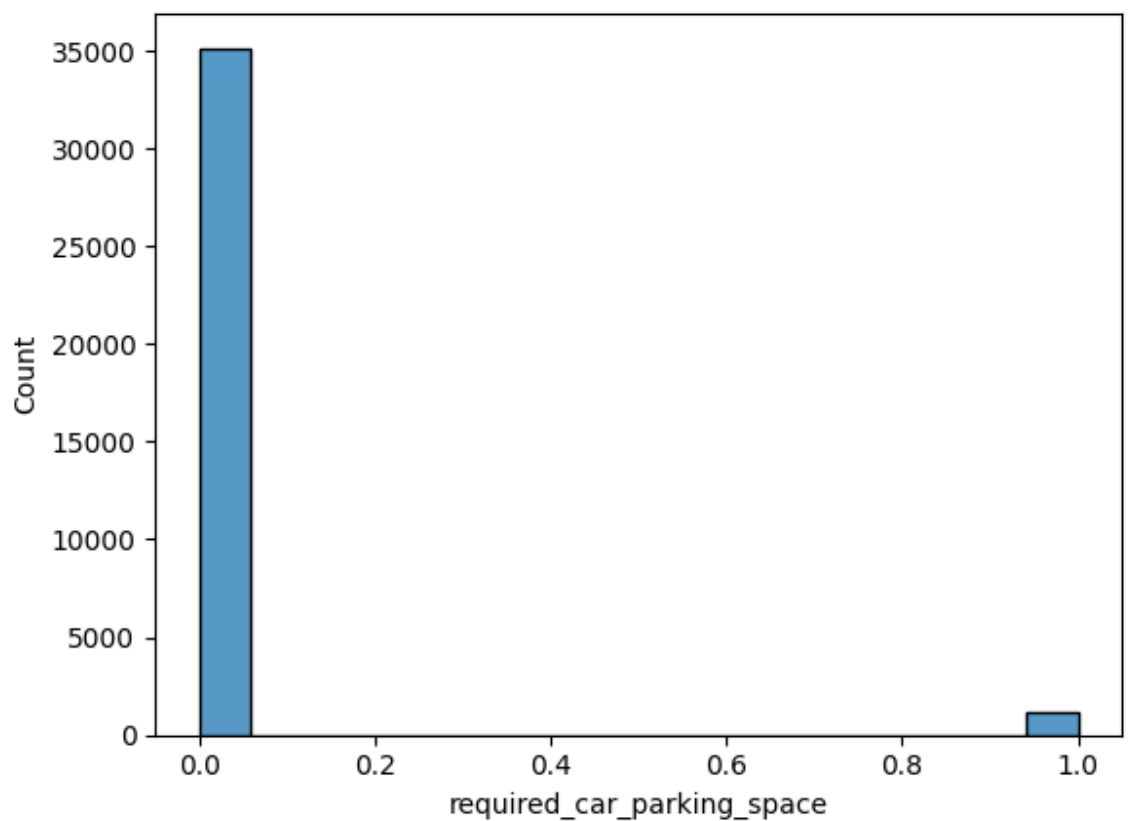
[15]:

```
Out[15]: <Axes: xlabel='type_of_meal_plan', ylabel='Count'>
```



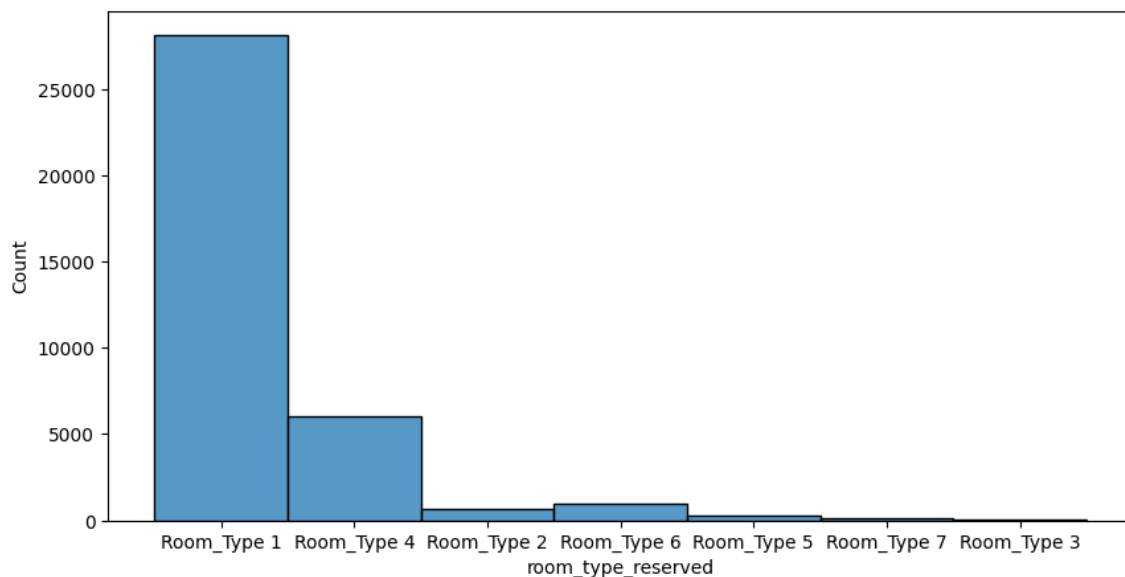
```
In [16]: sns.histplot(df['required_car_parking_space'])
```

```
Out[16]: <Axes: xlabel='required_car_parking_space', ylabel='Count'>
```



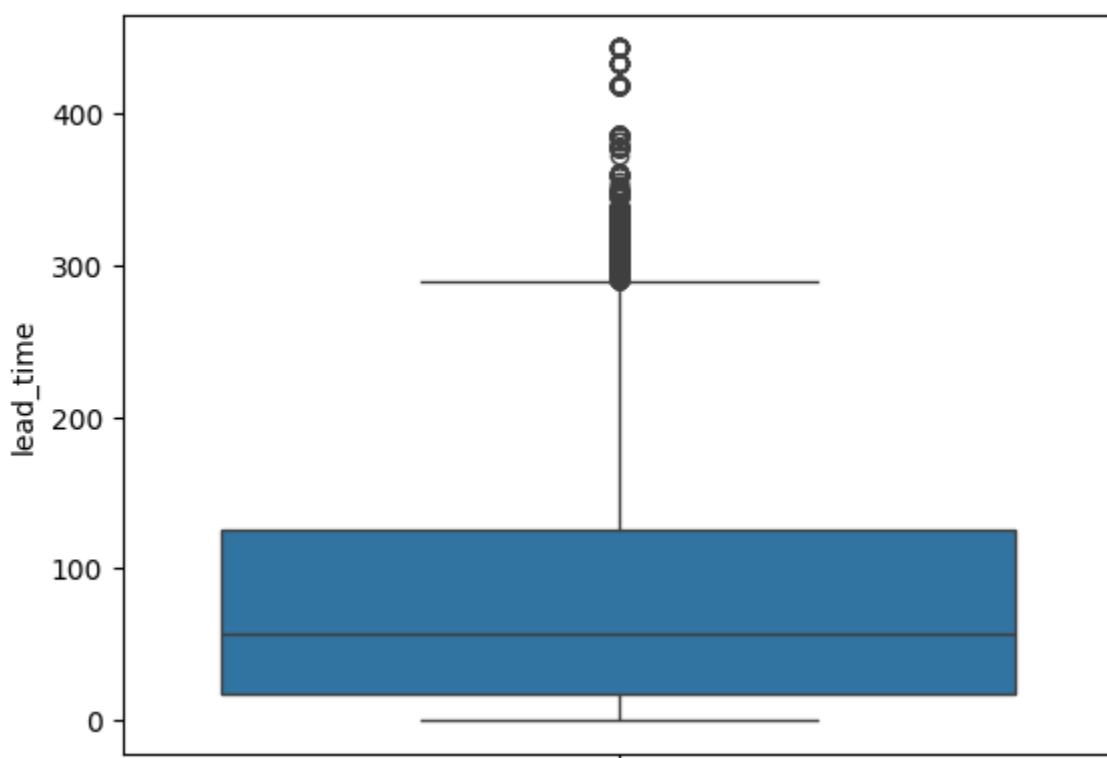
```
In [17]: plt.figure(figsize=(10,5))
sns.histplot(df['room_type_reserved'])
```

Out[17]: <Axes: xlabel='room_type_reserved', ylabel='Count'>



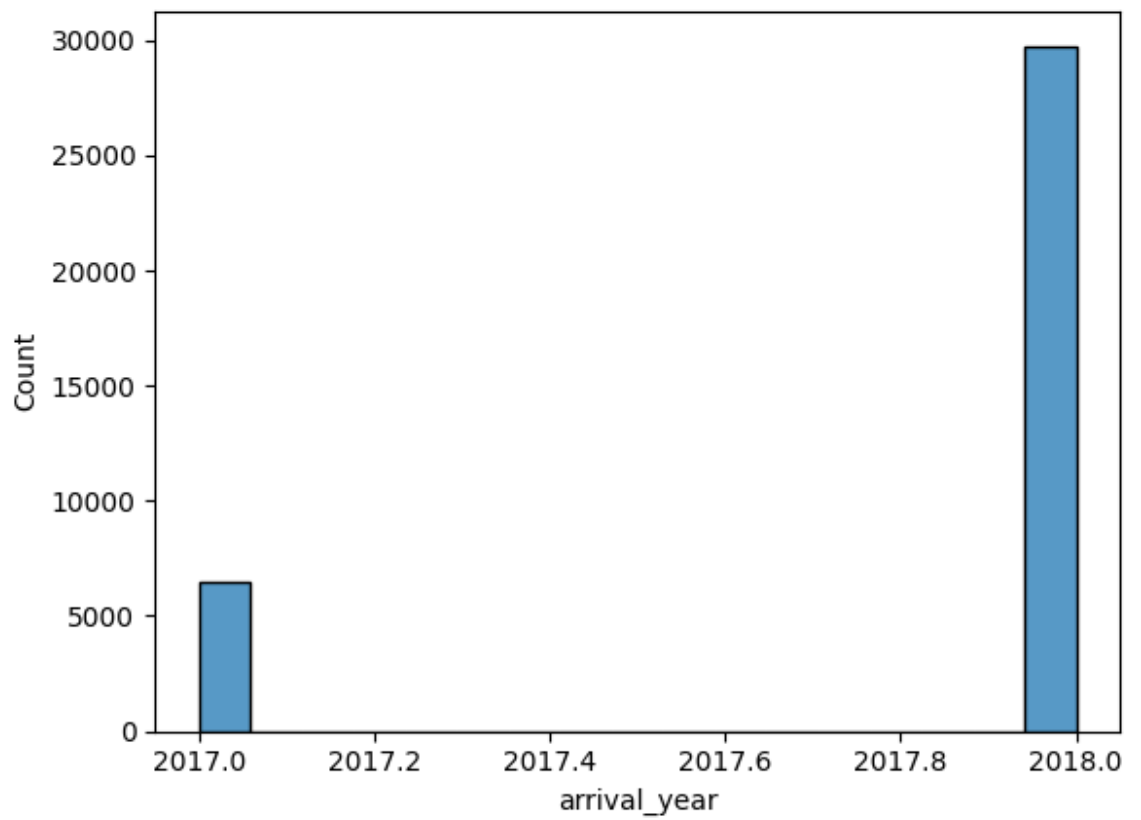
```
In [18]: sns.boxplot(df['lead_time'])
```

Out[18]: <Axes: ylabel='lead_time'>



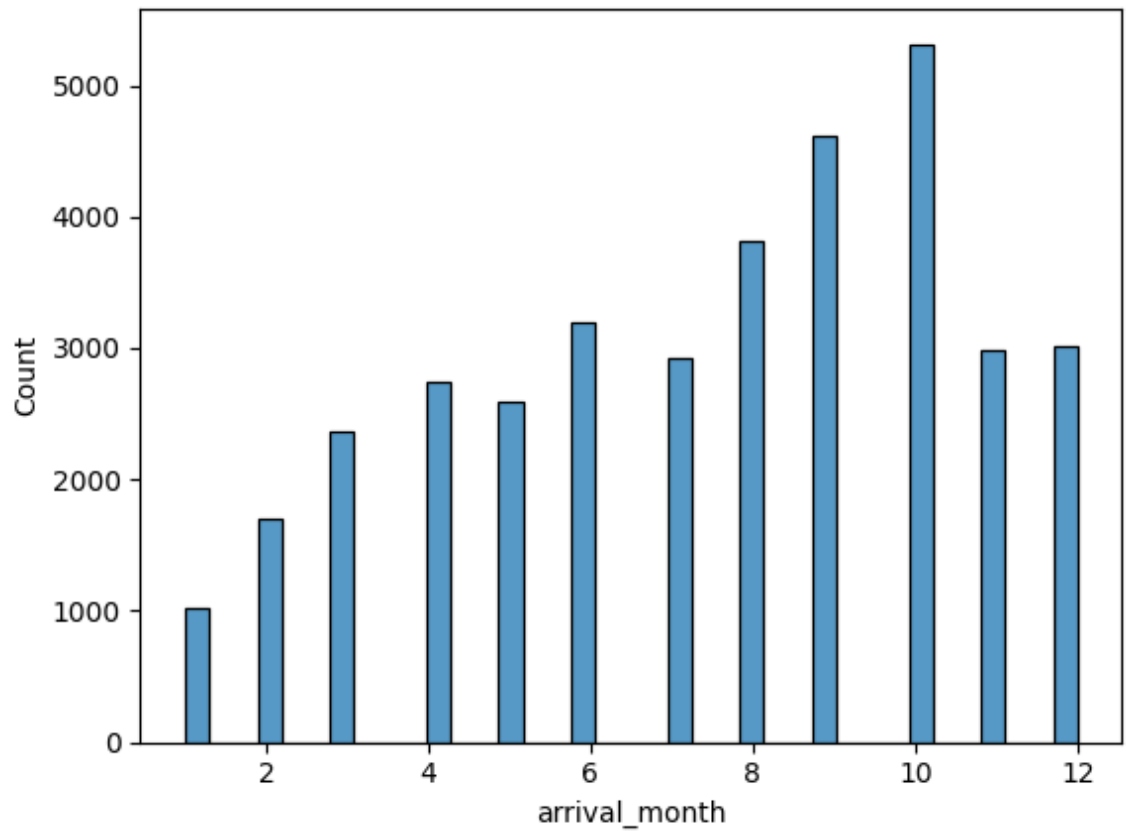
```
In [19]: sns.histplot(df['arrival_year'])
```

Out[19]: <Axes: xlabel='arrival_year', ylabel='Count'>



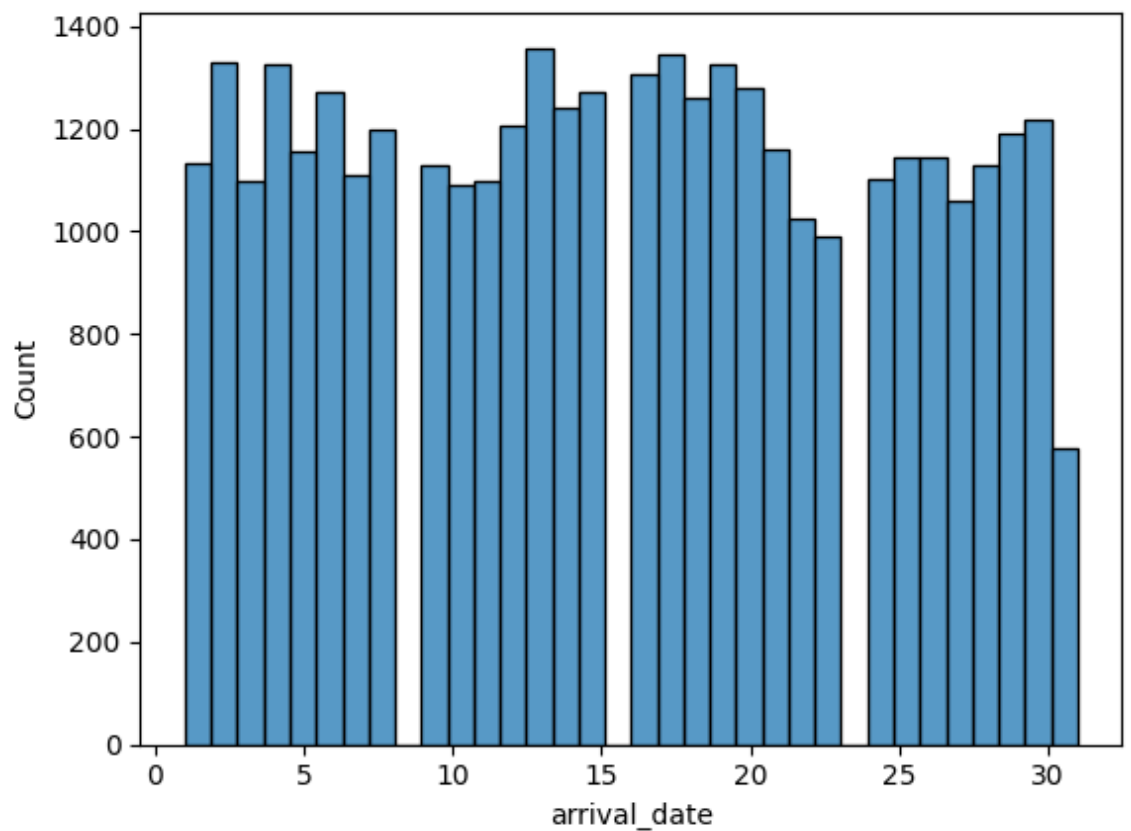
In [20]: `sns.histplot(df['arrival_month'])`

Out[20]: <Axes: xlabel='arrival_month', ylabel='Count'>



```
In [21]: sns.histplot(df['arrival_date'])
```

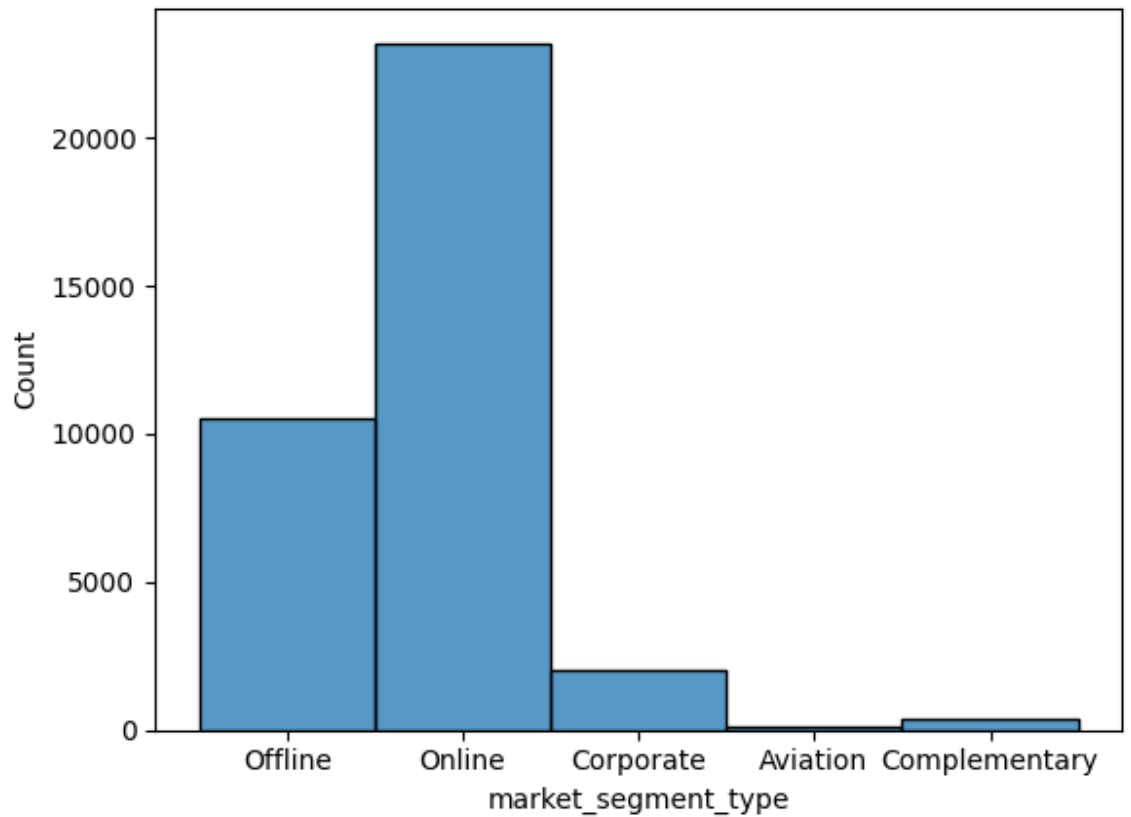
```
Out[21]: <Axes: xlabel='arrival_date', ylabel='Count'>
```



```
In      sns.histplot(df['market_segment_type'])
```

```
[22]:
```

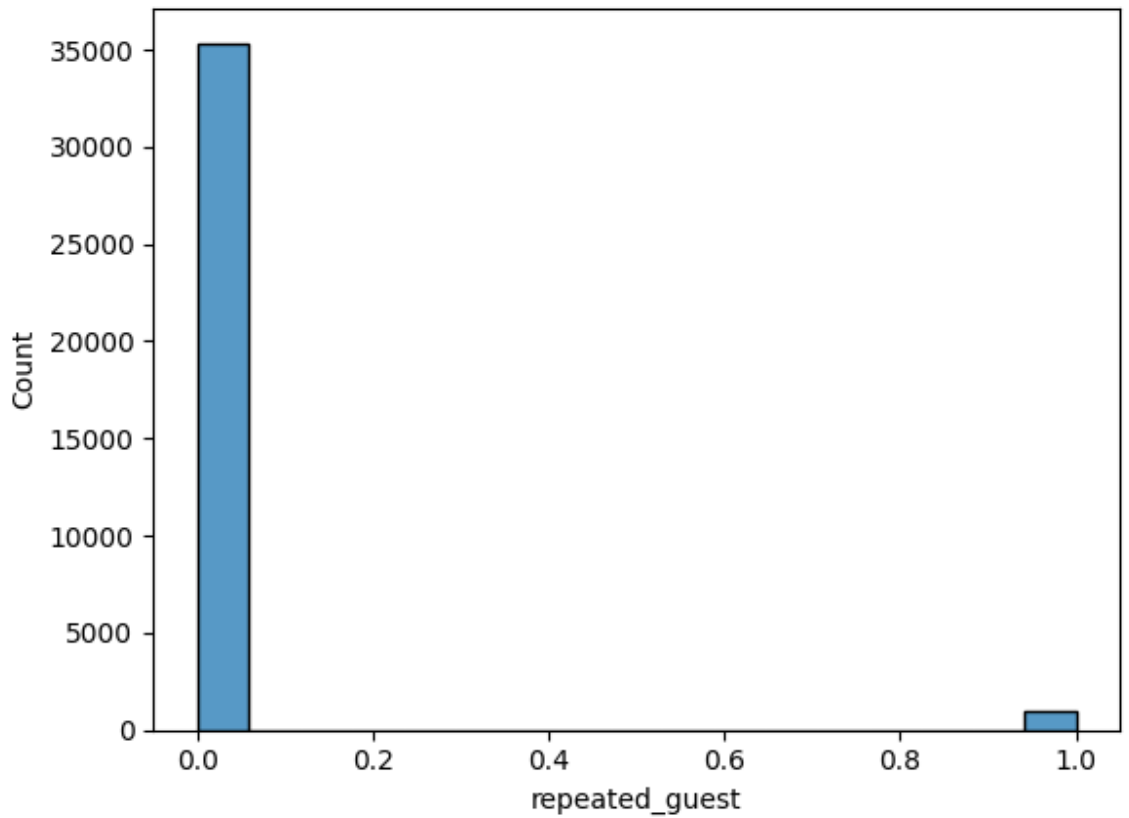
```
Out[22]: <Axes: xlabel='market_segment_type', ylabel='Count'>
```



```
In      sns.histplot(df['repeated_guest'])
```

```
[23]:
```

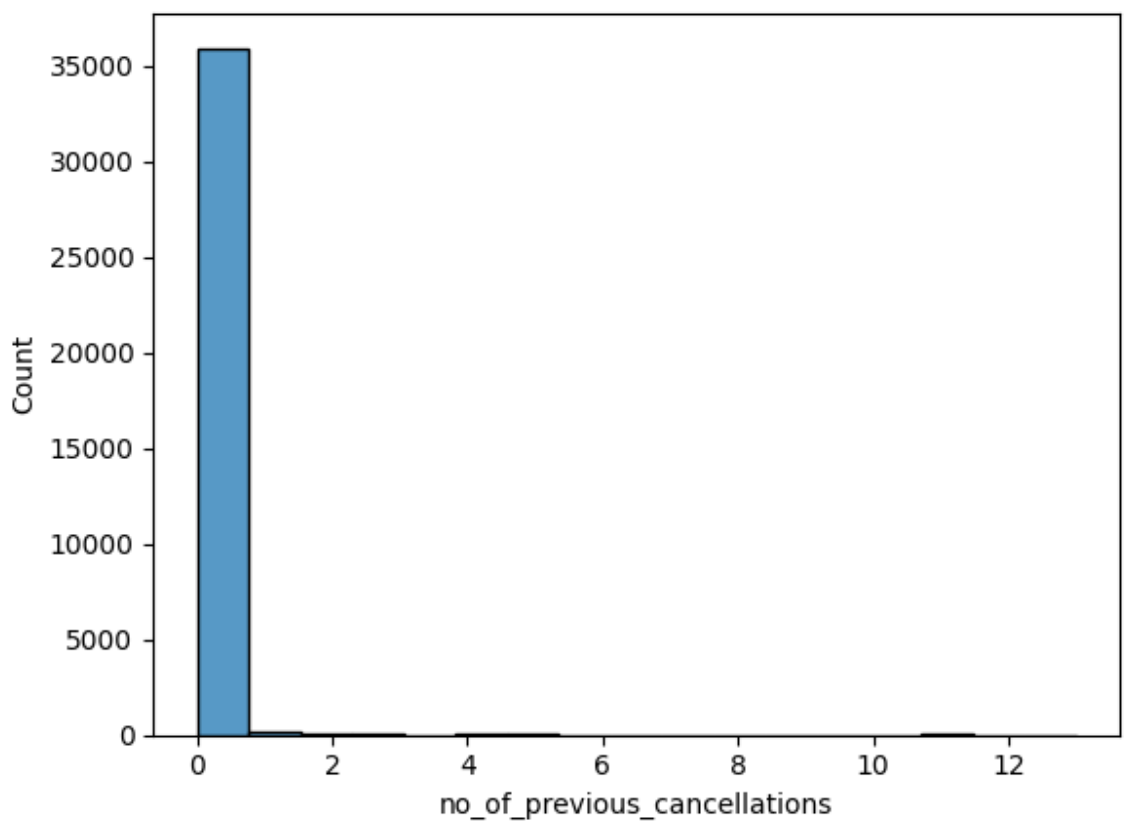
```
Out[23]: <Axes: xlabel='repeated_guest', ylabel='Count'>
```



```
In [24]: sns.histplot(df['no_of_previous_cancellations'])
```

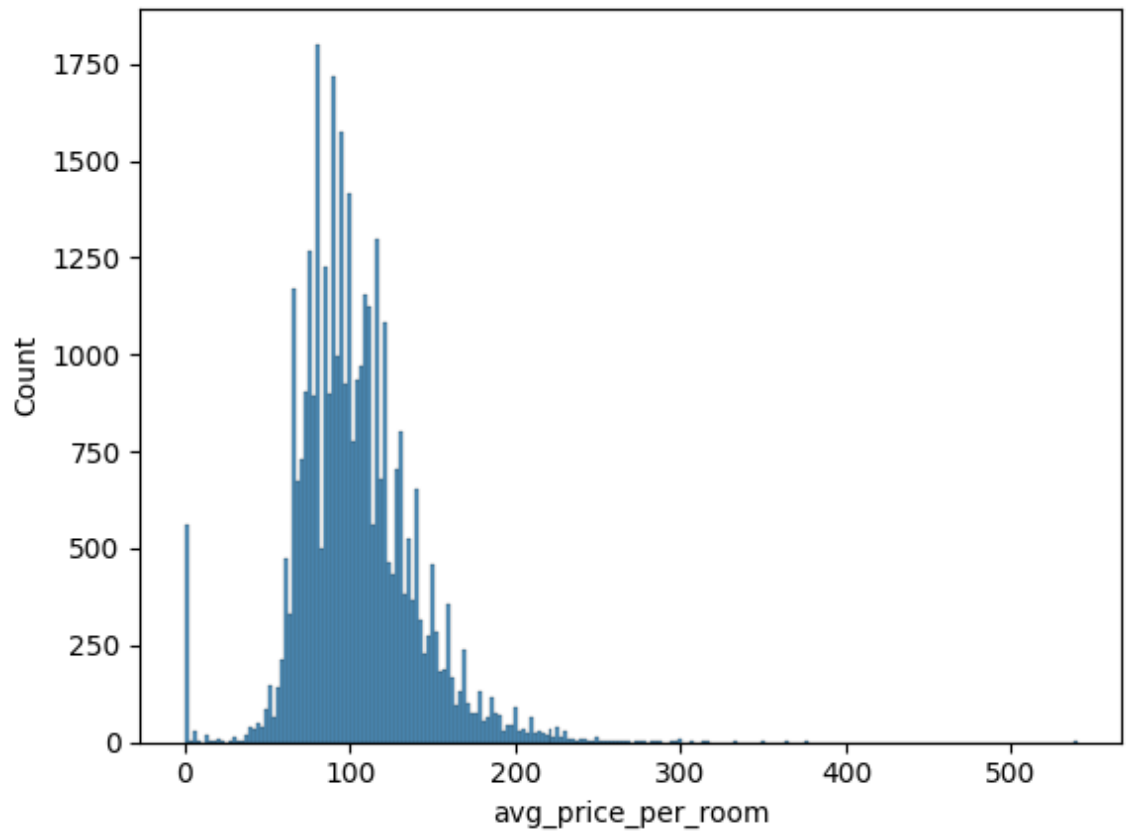
[24]:

Out[24]: <Axes: xlabel='no_of_previous_cancellations', ylabel='Count'>



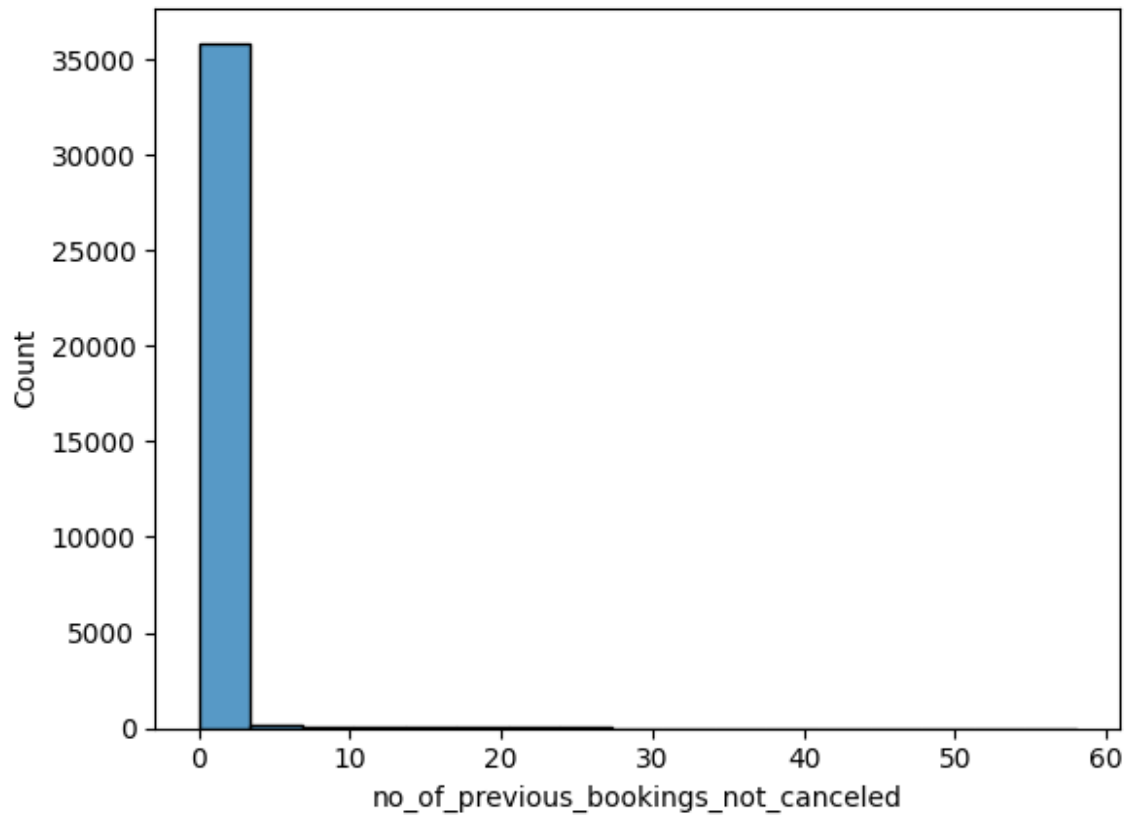
```
In [25]: sns.histplot(df['avg_price_per_room'])
```

```
Out[25]: <Axes: xlabel='avg_price_per_room', ylabel='Count'>
```



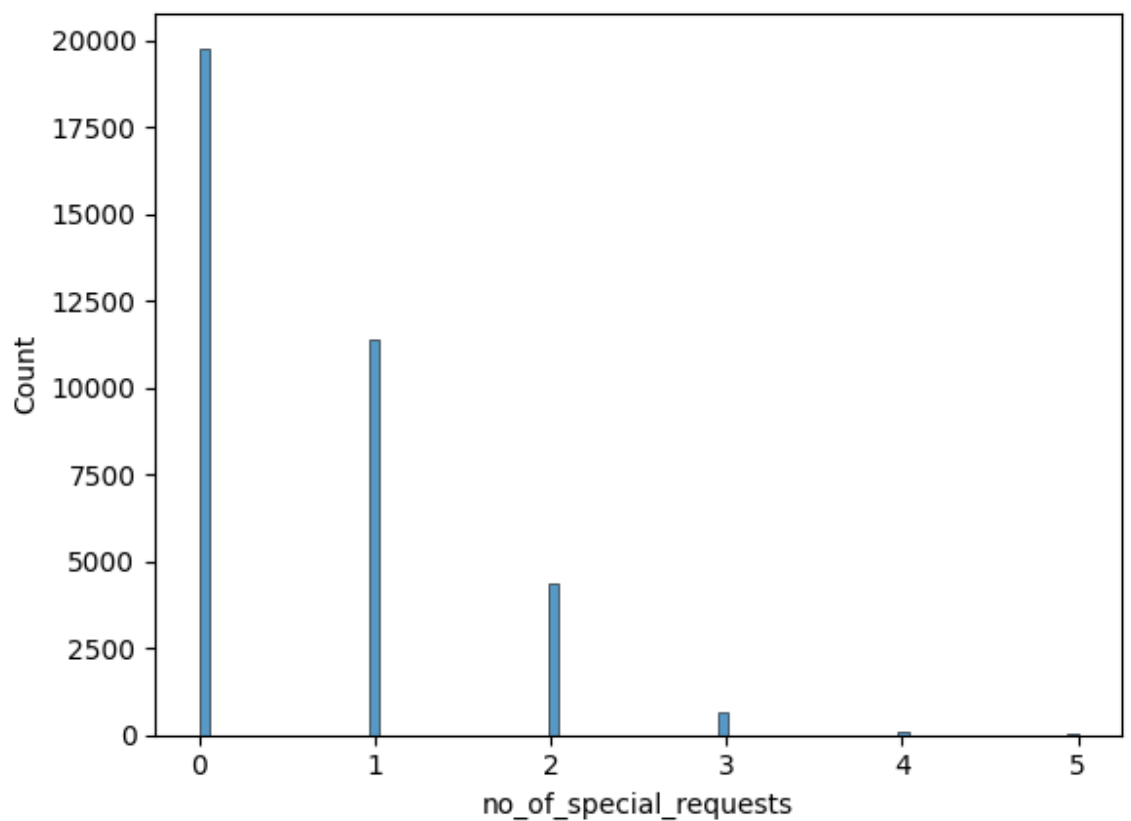
```
In [26]: sns.histplot(df['no_of_previous_bookings_not_canceled'])
```

```
Out[26]: <Axes: xlabel='no_of_previous_bookings_not_canceled', ylabel='Count'>
```



```
In [27]: sns.histplot(df['no_of_special_requests'])
```

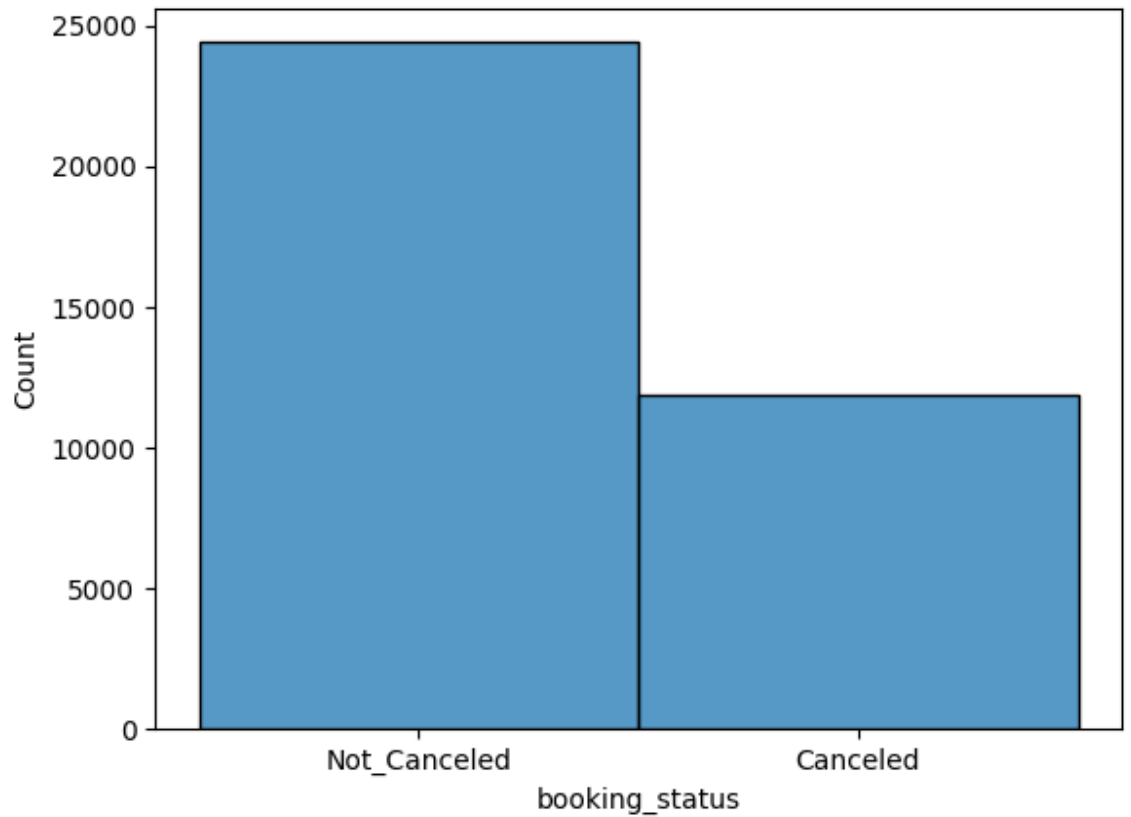
```
Out[27]: <Axes: xlabel='no_of_special_requests', ylabel='Count'>
```



In
[28]:

```
sns.histplot(df['booking_status'])
```

Out[28]: <Axes: xlabel='booking_status', ylabel='Count'>



Bivariate Analysis

In
[29]:

```
# October is the busiest month with september and august coming after and  
january being the least busy month  
df.value_counts('arrival_month')
```

Out[29]:

	count
arrival_month	
10	5317
9	4611
8	3813
6	3203
12	3021
11	2980
7	2920
4	2736

	count
arrival_month	
5	2598
3	2358
2	1704
1	1014

dtype: int64

```
In [30]: # October is still the buisiest month with 3437 not canceled bookings
not_canceled_bookings = df[df['booking_status'] == 'Not_Canceled']
monthly_not_canceled =
not_canceled_bookings['arrival_month'].value_counts().sort_values(ascending=False)
display(monthly_not_canceled)
```

	count
arrival_month	
10	3437
9	3073
12	2619
8	2325
11	2105
6	1912
4	1741
3	1658
5	1650
7	1606
2	1274
1	990

dtype: int64

```
In [31]: # Most customers come from an online market segement
df.value_counts('market_segment_type')
```

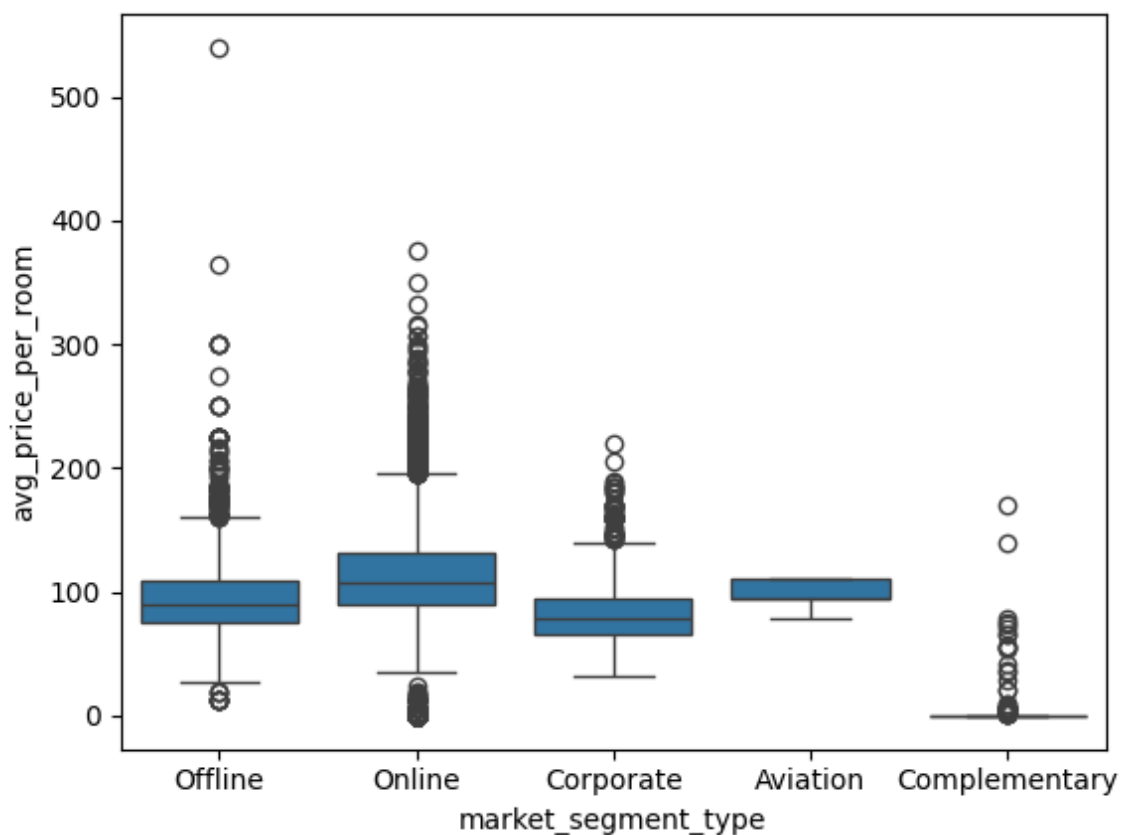
Out[31]:

	count
market_segment_type	
Online	23214
Offline	10528
Corporate	2017
Complementary	391
Aviation	125

dtype: int64

```
In [32]: # Online demo has the highest avg room cost, complementary has the lowest avg
room price
sns.boxplot(x='market_segment_type', y='avg_price_per_room', data=df)
```

Out[32]: <Axes: xlabel='market_segment_type', ylabel='avg_price_per_room'>



```
In [33]: # 32.7% of bookings are canceled
canceled_percentage = df['booking_status'].value_counts(normalize=True) * 100
canceled_percentage
```

Out[33]:

	proportion
booking_status	
Not_Canceled	67.23639
Canceled	32.76361

dtype: float64

In [34]:

```
#98% of repeated guests dont cancel
repeated_guests_df = df[df['repeated_guest'] == 1]
canceled_repeated_percentage =
repeated_guests_df['booking_status'].value_counts(normalize=True) * 100
canceled_repeated_percentage
```

Out[34]:

	proportion
booking_status	
Not_Canceled	98.27957
Canceled	1.72043

dtype: float64

In [35]:

```
cancellation_by_special_requests = df.groupby('no_of_special_requests')
['booking_status'].value_counts(normalize=True).unstack() * 100
cancellation_by_special_requests
```

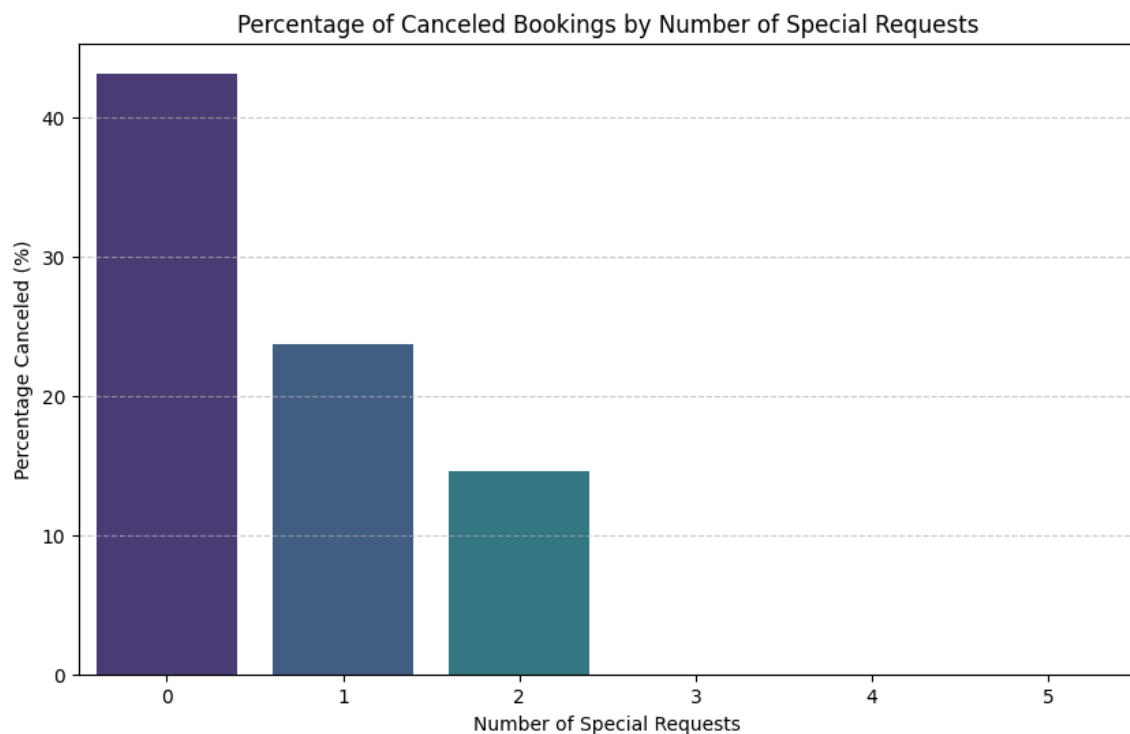
Out[35]:

	booking_status	Canceled	Not_Canceled
no_of_special_requests			
0		43.20676	56.79324
1		23.76682	76.23318
2		14.59670	85.40330
3		NaN	100.00000
4		NaN	100.00000
5		NaN	100.00000

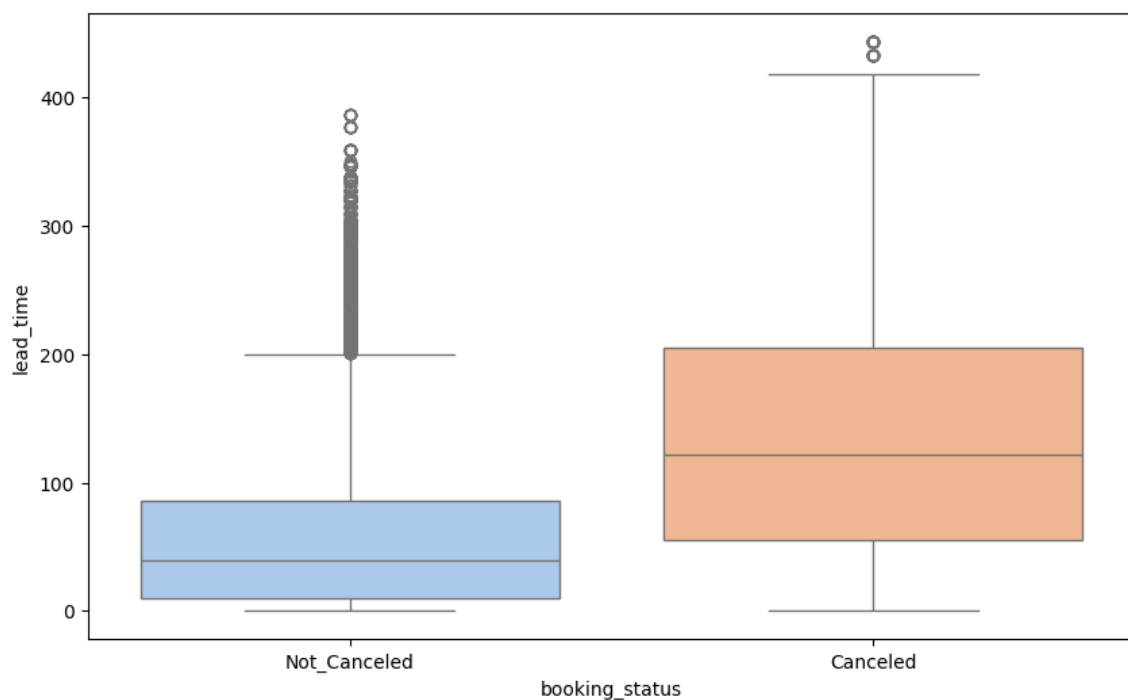
In [36]:

```
#When one special request is made around 23% of those customers cancel and
when 2 requests are made even less cancel at around 15%
plt.figure(figsize=(10, 6))
sns.barplot(x=cancellation_by_special_requests.index,
y=cancellation_by_special_requests['Canceled'], palette='viridis')
plt.title('Percentage of Canceled Bookings by Number of Special Requests')
```

```
plt.xlabel('Number of Special Requests')
plt.ylabel('Percentage Canceled (%)')
plt.xticks(rotation=0)
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```

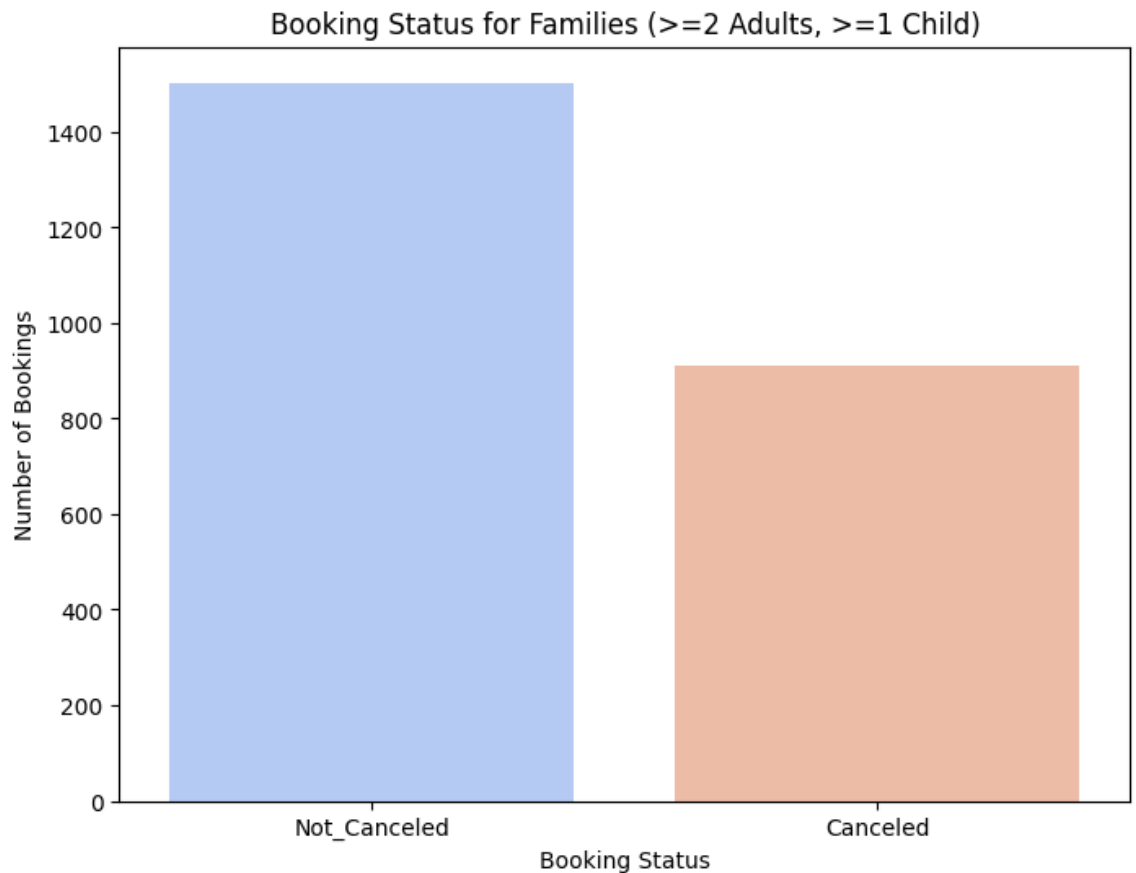


```
In [37]: # Customers will most likely not cancel if there booking date is at around 50
days away with the avg cancelation happening around 100 days before
plt.figure(figsize=(10, 6))
sns.boxplot(x='booking_status', y='lead_time', data=df, palette='pastel')
plt.show()
```



```
In [38]: # Cancellation rates for families with at least 2 adults and at one child are
         # significant
         family_bookings = df[(df['no_of_adults'] >= 2) & (df['no_of_children'] >= 1)]

         plt.figure(figsize=(8, 6))
         sns.countplot(x='booking_status', data=family_bookings, palette='coolwarm')
         plt.title('Booking Status for Families (>=2 Adults, >=1 Child)')
         plt.xlabel('Booking Status')
         plt.ylabel('Number of Bookings')
         plt.show()
```



```
In [39]: # No correlation between number of nights booked and booking status
         df['total_nights'] = df['no_of_weekend_nights'] + df['no_of_week_nights']

         sns.boxplot(x='booking_status', y='total_nights', data=df, palette='pastel')
         plt.title('Total Nights Booked vs. Booking Status')
         plt.xlabel('Booking Status')
         plt.ylabel('Total Nights Booked')
         plt.show()
```



Data Preprocessing

- Missing value treatment (if needed)
- Feature engineering (if needed)
- Outlier detection and treatment (if needed)
- Preparing data for modeling
- Any other preprocessing steps (if needed)

```
In [40]: # Preparing data for modeling
X = df.drop('booking_status', axis=1)
y = y = df['booking_status'].map({'Not_Canceled': 0, 'Canceled': 1})

X = sm.add_constant(X)
X = pd.get_dummies(X, drop_first=True)

# Convert all boolean columns to int type
for col in X.select_dtypes(include=['bool']).columns:
    X[col] = X[col].astype(int)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
                                                    random_state=1)
```

EDA

- It is a good idea to explore the data once again after manipulating it.

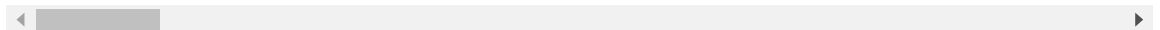
In X

[41]:

Out[41]:

	const	no_of_adults	no_of_children	no_of_weekend_nights	no_of_we
0	1.00000	2	0	1	2
1	1.00000	2	0	2	3
2	1.00000	1	0	2	1
3	1.00000	2	0	0	2
4	1.00000	2	0	1	1
...	...	...	...	...	...
36270	1.00000	3	0	2	6
36271	1.00000	2	0	1	3
36272	1.00000	2	0	2	6
36273	1.00000	2	0	0	3
36274	1.00000	2	0	1	2

36275 rows × 29 columns



In y

[42]:

Out[42]:

	booking_status
0	0
1	0
2	1
3	1
4	1
...	...

	booking_status
36270	0
36271	1
36272	0
36273	1
36274	0

36275 rows × 1 columns

dtype: int64

In
[43]:

X.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 36275 entries, 0 to 36274
Data columns (total 29 columns):
 #   Column                                          Non-Null Count  Dtype
---  -
 0   const                                          36275 non-null  float64
 1   no_of_adults                                  36275 non-null  int64
 2   no_of_children                                36275 non-null  int64
 3   no_of_weekend_nights                          36275 non-null  int64
 4   no_of_week_nights                            36275 non-null  int64
 5   required_car_parking_space                    36275 non-null  int64
 6   lead_time                                     36275 non-null  int64
 7   arrival_year                                  36275 non-null  int64
 8   arrival_month                                 36275 non-null  int64
 9   arrival_date                                  36275 non-null  int64
10  repeated_guest                                36275 non-null  int64
11  no_of_previous_cancellations                  36275 non-null  int64
12  no_of_previous_bookings_not_canceled          36275 non-null  int64
13  avg_price_per_room                            36275 non-null  float64
14  no_of_special_requests                        36275 non-null  int64
15  total_nights                                  36275 non-null  int64
16  type_of_meal_plan_Meal Plan 2                 36275 non-null  int64
17  type_of_meal_plan_Meal Plan 3                 36275 non-null  int64
18  type_of_meal_plan_Not Selected                 36275 non-null  int64
19  room_type_reserved_Room_Type 2                 36275 non-null  int64
20  room_type_reserved_Room_Type 3                 36275 non-null  int64
21  room_type_reserved_Room_Type 4                 36275 non-null  int64
22  room_type_reserved_Room_Type 5                 36275 non-null  int64
23  room_type_reserved_Room_Type 6                 36275 non-null  int64
24  room_type_reserved_Room_Type 7                 36275 non-null  int64
25  market_segment_type_Complementary              36275 non-null  int64
26  market_segment_type_Corporate                  36275 non-null  int64
27  market_segment_type_Offline                    36275 non-null  int64
28  market_segment_type_Online                     36275 non-null  int64
dtypes: float64(2), int64(27)
memory usage: 8.0 MB
```

Checking Multicollinearity

- In order to make statistical inferences from a logistic regression model, it is important to ensure that there is no multicollinearity present in the data.

```
In [44]: def get_metrics(model, X, y):
# Predict probabilities
y_pred_proba = model.predict(X)
y_pred = (y_pred_proba >= 0.5).astype(int)

accuracy = accuracy_score(y, y_pred)
precision = precision_score(y, y_pred)
recall = recall_score(y, y_pred)
f1 = f1_score(y, y_pred)

print(f"Accuracy: {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall: {recall:.4f}")
print(f"F1 Score: {f1:.4f}")
```

```
In [45]: def plot_confusion_matrix(y_true, y_pred, model_name="Model"):
cm = confusion_matrix(y_true, y_pred)
plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title(f'Confusion Matrix for {model_name}')
plt.xlabel('Predicted')
plt.ylabel('True')
plt.show()
```

```
In [46]: model1 = sm.Logit(y_train, X_train)
lg = model1.fit()
print(lg.summary())
```

```
Warning: Maximum number of iterations has been exceeded.
Current function value: 0.425036
Iterations: 35
```

```
Logit Regression Results
=====
=====
Dep. Variable:          booking_status    No. Observations:
25392
Model:                  Logit            Df Residuals:
25363
Method:                 MLE              Df Model:
28
Date:                  Tue, 28 Jul 2026    Pseudo R-squ.:
0.3293
Time:                  21:56:06           Log-Likelihood:
```

```

-10793.
converged:                               False   LL-Null:
-16091.
Covariance Type:                         nonrobust   LLR p-value:
0.000

```

			=====		
			=====		
			coef	std err	z
P> z	[0.025	0.975]	-----		

const			-924.5923	120.817	-7.653
0.000	-1161.390	-687.795			
no_of_adults			0.1135	0.038	3.017
0.003	0.040	0.187			
no_of_children			0.1563	0.057	2.732
0.006	0.044	0.268			
no_of_weekend_nights			0.0583	1.72e+05	3.39e-07
1.000	-3.37e+05	3.37e+05			
no_of_week_nights			-0.0086	1.72e+05	-5.02e-08
1.000	-3.37e+05	3.37e+05			
required_car_parking_space			-1.5939	0.138	-11.561
0.000	-1.864	-1.324			
lead_time			0.0157	0.000	58.868
0.000	0.015	0.016			
arrival_year			0.4570	0.060	7.633
0.000	0.340	0.574			
arrival_month			-0.0415	0.006	-6.418
0.000	-0.054	-0.029			
arrival_date			0.0005	0.002	0.252
0.801	-0.003	0.004			
repeated_guest			-2.3469	0.617	-3.805
0.000	-3.556	-1.138			
no_of_previous_cancellations			0.2664	0.086	3.108
0.002	0.098	0.434			
no_of_previous_bookings_not_canceled			-0.1727	0.153	-1.131
0.258	-0.472	0.127			
avg_price_per_room			0.0188	0.001	25.404
0.000	0.017	0.020			
no_of_special_requests			-1.4690	0.030	-48.790
0.000	-1.528	-1.410			
total_nights			0.0484	1.72e+05	2.82e-07
1.000	-3.37e+05	3.37e+05			
type_of_meal_plan_Meal Plan 2			0.1768	0.067	2.654
0.008	0.046	0.307			
type_of_meal_plan_Meal Plan 3			17.8379	5057.771	0.004
0.997	-9895.212	9930.887			
type_of_meal_plan_Not Selected			0.2782	0.053	5.245
0.000	0.174	0.382			
room_type_reserved_Room_Type 2			-0.3610	0.131	-2.761
0.006	-0.617	-0.105			
room_type_reserved_Room_Type 3			-0.0009	1.310	-0.001
0.999	-2.569	2.567			
room_type_reserved_Room_Type 4			-0.2821	0.053	-5.305
0.000	-0.386	-0.178			

room_type_reserved_Room_Type 5	-0.7176	0.209	-3.432
0.001 -1.127 -0.308			
room_type_reserved_Room_Type 6	-0.9456	0.147	-6.434
0.000 -1.234 -0.658			
room_type_reserved_Room_Type 7	-1.3964	0.293	-4.767
0.000 -1.971 -0.822			
market_segment_type_Complementary	-41.8798	8.42e+05	-4.98e-05
1.000 -1.65e+06 1.65e+06			
market_segment_type_Corporate	-1.1935	0.266	-4.487
0.000 -1.715 -0.672			
market_segment_type_Offline	-2.1955	0.255	-8.625
0.000 -2.694 -1.697			
market_segment_type_Online	-0.3990	0.251	-1.588
0.112 -0.891 0.093			

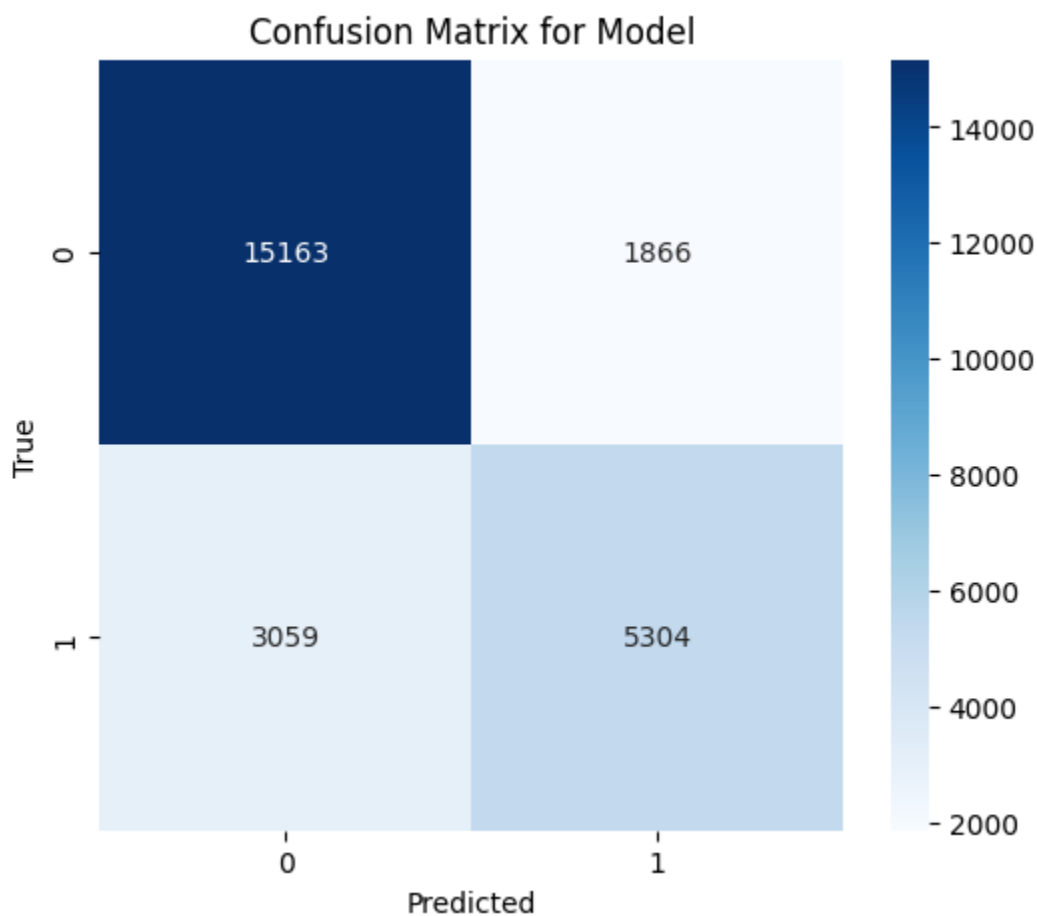
=====

=====

```
In #Train metrics
[47]: get_metrics(lg, X_train, y_train)
```

```
Accuracy: 0.8060
Precision: 0.7397
Recall: 0.6342
F1 Score: 0.6829
```

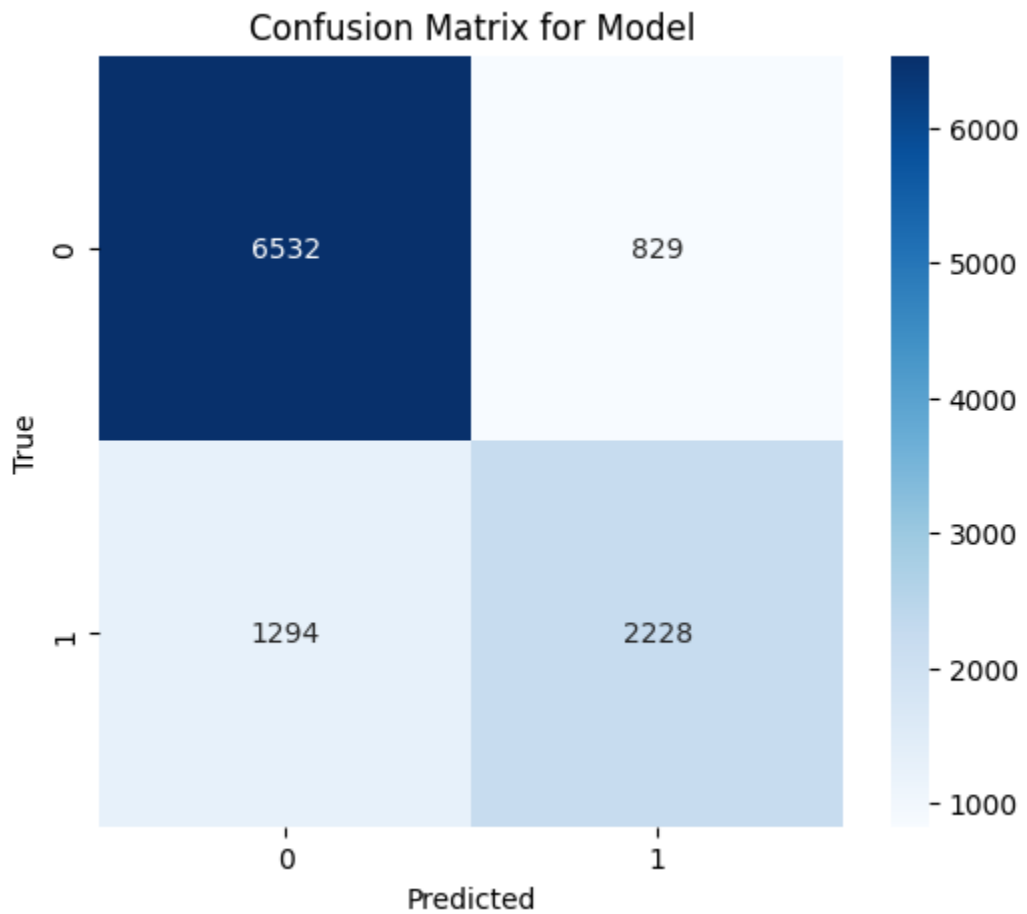
```
In #Train Matrix
[48]: plot_confusion_matrix(y_train, lg.predict(X_train) >= 0.5, model_name="Model")
```



```
In [49]: #Test Metrics  
get_metrics(lg, X_test, y_test)
```

```
Accuracy: 0.8049  
Precision: 0.7288  
Recall: 0.6326  
F1 Score: 0.6773
```

```
In [50]: #Test Matrix  
plot_confusion_matrix(y_test, lg.predict(X_test) >= 0.5, model_name="Model")
```



```
In [51]: # Calculate VIF scores for the model's features
vif_data = pd.DataFrame()
vif_data["feature"] = X_train.columns
vif_data["VIF"] = [variance_inflation_factor(X_train.values, i) for i in
range(X_train.shape[1])]
display(vif_data.sort_values(by='VIF', ascending=False))
```

	feature	VIF
4	no_of_week_nights	inf
3	no_of_weekend_nights	inf
15	total_nights	inf
0	const	39468156.70600
28	market_segment_type_Online	71.17643
27	market_segment_type_Offline	64.11392
26	market_segment_type_Corporate	16.92844
25	market_segment_type_Complementary	4.50011
13	avg_price_per_room	2.05042
2	no_of_children	1.97823

	feature	VIF
23	room_type_reserved_Room_Type 6	1.97307
10	repeated_guest	1.78352
12	no_of_previous_bookings_not_canceled	1.65199
7	arrival_year	1.43083
11	no_of_previous_cancellations	1.39569
6	lead_time	1.39491
21	room_type_reserved_Room_Type 4	1.36152
1	no_of_adults	1.34815
8	arrival_month	1.27567
18	type_of_meal_plan_Not Selected	1.27218
16	type_of_meal_plan_Meal Plan 2	1.27185
14	no_of_special_requests	1.24728
24	room_type_reserved_Room_Type 7	1.11512
19	room_type_reserved_Room_Type 2	1.10144
5	required_car_parking_space	1.03993
22	room_type_reserved_Room_Type 5	1.02781
17	type_of_meal_plan_Meal Plan 3	1.02522
9	arrival_date	1.00674
20	room_type_reserved_Room_Type 3	1.00330

Building a Logistic Regression model

```
In [52]: # Create copies to perform iterative removal without modifying original
dataframes
X_train_filtered = X_train.copy()
X_test_filtered = X_test.copy()

p_value_threshold = 0.05
iteration = 0
removed_features = []

print("Starting iterative feature removal based on p-values (threshold =
0.05):")
```

```

while True:
    iteration += 1
    print(f"\n--- Iteration {iteration} ---")

    # Train the logistic regression model with the current feature set
    model = sm.Logit(y_train, X_train_filtered)
    lg_filtered = model.fit(dis=False)

    # Get p-values
    p_values = lg_filtered.pvalues

    # Find the feature with the highest p-value, excluding 'const'
    if 'const' in p_values.index:
        p_values_for_removal = p_values.drop('const')
    else:
        p_values_for_removal = p_values

    # Check if there are any features left to consider for removal
    if p_values_for_removal.empty:
        print("No more features to evaluate for removal based on p-value.")
        break

    max_pvalue_feature = p_values_for_removal.idxmax()
    max_pvalue = p_values_for_removal.max()

    if max_pvalue > p_value_threshold:
        # Remove the feature
        removed_features.append(max_pvalue_feature)
        X_train_filtered = X_train_filtered.drop(columns=[max_pvalue_feature])
        X_test_filtered = X_test_filtered.drop(columns=[max_pvalue_feature])
        print(f"Removed feature: '{max_pvalue_feature}' (P-value:
{max_pvalue:.4f})")

        # Re-train the model with the reduced feature set and display metrics
        print("Model performance after removal:")
        # Re-train model for metrics, as lg_filtered was trained with the
*previous* set
        model_after_removal = sm.Logit(y_train, X_train_filtered)
        lg_after_removal = model_after_removal.fit(dis=False)
        get_metrics(lg_after_removal, X_test_filtered, y_test)

    else:
        print(f"All remaining features have p-values below the threshold of
{p_value_threshold}. Stopping.")
        break

# No significant change in performance
print("\n--- Final Model Features and P-values ---")
model2 = sm.Logit(y_train, X_train_filtered)
lg2 = model2.fit(dis=False)
print(lg2.summary())

```

Starting iterative feature removal based on p-values (threshold = 0.05):

--- Iteration 1 ---

Removed feature: 'no_of_week_nights' (P-value: 1.0000)
Model performance after removal:
Accuracy: 0.8049
Precision: 0.7288
Recall: 0.6326
F1 Score: 0.6773

--- Iteration 2 ---

Removed feature: 'market_segment_type_Complementary' (P-value: 1.0000)
Model performance after removal:
Accuracy: 0.8048
Precision: 0.7295
Recall: 0.6309
F1 Score: 0.6766

--- Iteration 3 ---

Removed feature: 'room_type_reserved_Room_Type 3' (P-value: 0.9408)
Model performance after removal:
Accuracy: 0.8048
Precision: 0.7295
Recall: 0.6309
F1 Score: 0.6766

--- Iteration 4 ---

Removed feature: 'arrival_date' (P-value: 0.8391)
Model performance after removal:
Accuracy: 0.8047
Precision: 0.7292
Recall: 0.6309
F1 Score: 0.6765

--- Iteration 5 ---

Removed feature: 'market_segment_type_Online' (P-value: 0.6361)
Model performance after removal:
Accuracy: 0.8047
Precision: 0.7292
Recall: 0.6309
F1 Score: 0.6765

--- Iteration 6 ---

Removed feature: 'no_of_previous_bookings_not_canceled' (P-value: 0.2566)
Model performance after removal:
Accuracy: 0.8047
Precision: 0.7292
Recall: 0.6309
F1 Score: 0.6765

--- Iteration 7 ---

Removed feature: 'type_of_meal_plan_Meal Plan 3' (P-value: 0.2130)
Model performance after removal:
Accuracy: 0.8046
Precision: 0.7290
Recall: 0.6309
F1 Score: 0.6764

--- Iteration 8 ---

All remaining features have p-values below the threshold of 0.05.
Stopping.

--- Final Model Features and P-values ---

Logit Regression Results

=====

Dep. Variable: booking_status No. Observations:
25392

Model: Logit Df Residuals:
25370

Method: MLE Df Model:
21

Date: Tue, 28 Jul 2026 Pseudo R-squ.:
0.3283

Time: 21:56:23 Log-Likelihood:
-10809.

converged: True LL-Null:
-16091.

Covariance Type: nonrobust LLR p-value:
0.000

=====

			coef	std err	z
P> z	[0.025	0.975]			

const			-917.2860	120.456	-7.615
0.000	-1153.376	-681.196			
no_of_adults			0.1086	0.037	2.914
0.004	0.036	0.182			
no_of_children			0.1522	0.057	2.660
0.008	0.040	0.264			
no_of_weekend_nights			0.0669	0.025	2.667
0.008	0.018	0.116			
required_car_parking_space			-1.5943	0.138	-11.561
0.000	-1.865	-1.324			
lead_time			0.0157	0.000	59.218
0.000	0.015	0.016			
arrival_year			0.4531	0.060	7.591
0.000	0.336	0.570			
arrival_month			-0.0424	0.006	-6.568
0.000	-0.055	-0.030			
repeated_guest			-2.7365	0.557	-4.915
0.000	-3.828	-1.645			
no_of_previous_cancellations			0.2289	0.077	2.983
0.003	0.078	0.379			
avg_price_per_room			0.0192	0.001	26.343
0.000	0.018	0.021			
no_of_special_requests			-1.4699	0.030	-48.892
0.000	-1.529	-1.411			
total_nights			0.0418	0.012	3.403
0.001	0.018	0.066			
type_of_meal_plan_Meal Plan 2			0.1654	0.067	2.487

```

0.013      0.035      0.296
type_of_meal_plan_Not Selected      0.2858      0.053      5.405
0.000      0.182      0.389
room_type_reserved_Room_Type 2      -0.3560      0.131      -2.725
0.006      -0.612      -0.100
room_type_reserved_Room_Type 4      -0.2826      0.053      -5.330
0.000      -0.387      -0.179
room_type_reserved_Room_Type 5      -0.7352      0.208      -3.529
0.000      -1.143      -0.327
room_type_reserved_Room_Type 6      -0.9650      0.147      -6.572
0.000      -1.253      -0.677
room_type_reserved_Room_Type 7      -1.4312      0.293      -4.892
0.000      -2.005      -0.858
market_segment_type_Corporate      -0.7928      0.103      -7.711
0.000      -0.994      -0.591
market_segment_type_Offline      -1.7867      0.052      -34.391
0.000      -1.889      -1.685
=====
=====

```

```

In      #converting coefficients to odds
[53]:  odds = np.exp(lg2.params)
      pd.DataFrame(odds, columns=['Odds']).T

```

Out[53]:

	const	no_of_adults	no_of_children	no_of_weekend_nights	required_
Odds	0.00000	1.11475	1.16436	1.06917	0.20305

Model performance evaluation

```

In      #Checking train performance of model after p values removed
[54]:  get_metrics(lg2, X_train_filtered, y_train)

```

```

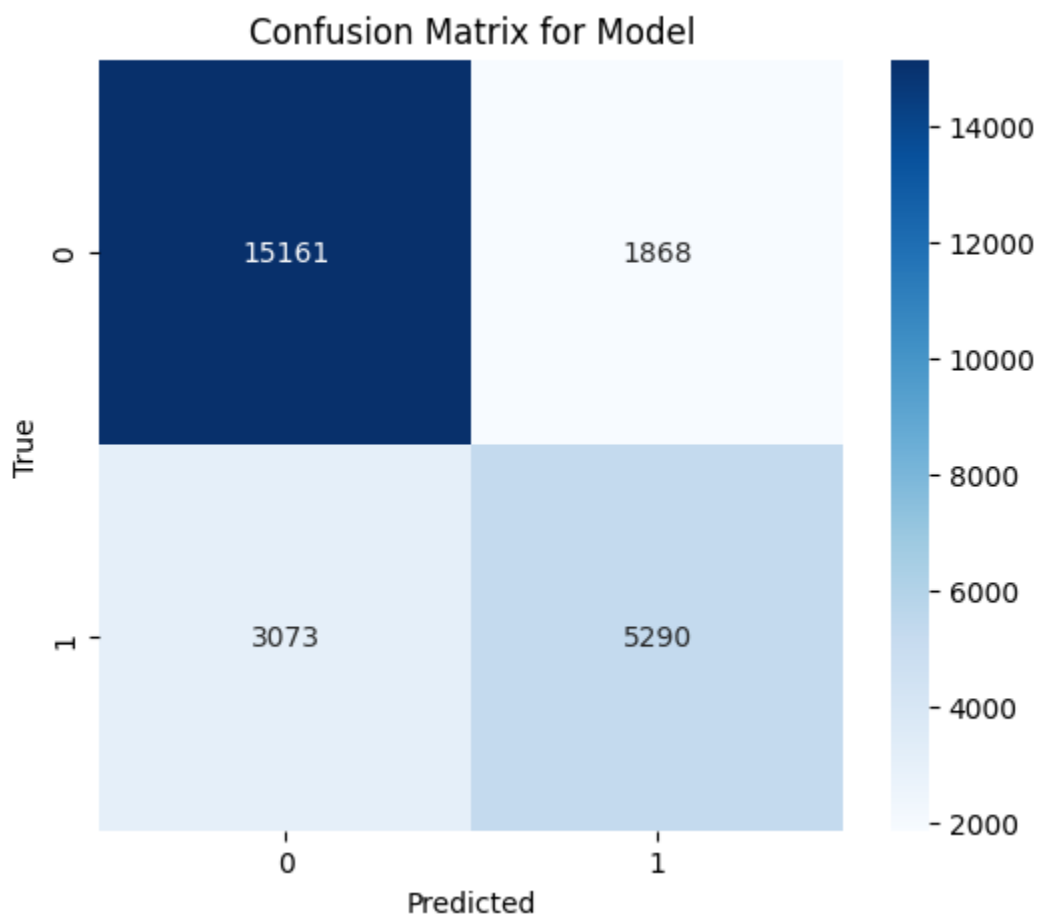
Accuracy: 0.8054
Precision: 0.7390
Recall: 0.6325
F1 Score: 0.6817

```

```

In      #Checking train confusion matrix of new model
[55]:  y_train_pred = lg2.predict(X_train_filtered)
      y_train_pred = (y_train_pred >= 0.5).astype(int)
      plot_confusion_matrix(y_train, y_train_pred)

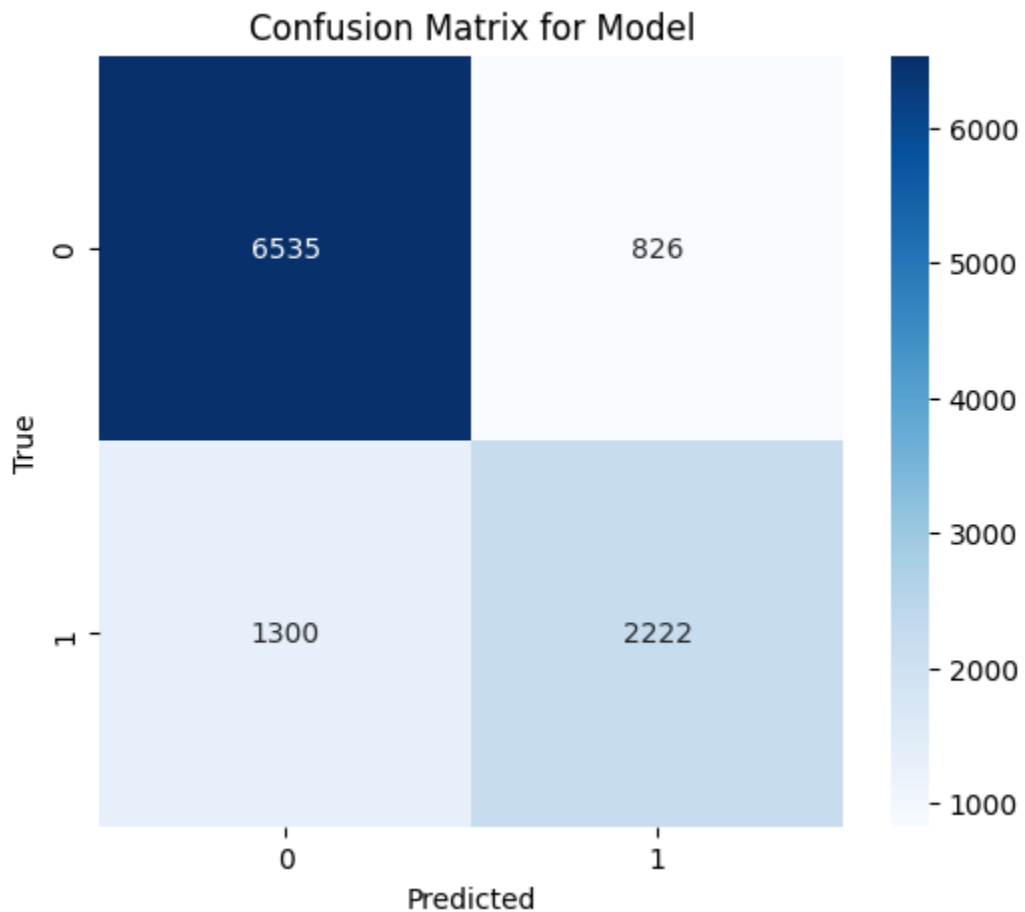
```



```
In [56]: # New test metrics  
get_metrics(lg2, X_test_filtered, y_test)
```

```
Accuracy: 0.8046  
Precision: 0.7290  
Recall: 0.6309  
F1 Score: 0.6764
```

```
In [57]: # New test confusion matrix  
y_test_pred = lg2.predict(X_test_filtered)  
y_test_pred = (y_test_pred >= 0.5).astype(int)  
plot_confusion_matrix(y_test, y_test_pred)
```



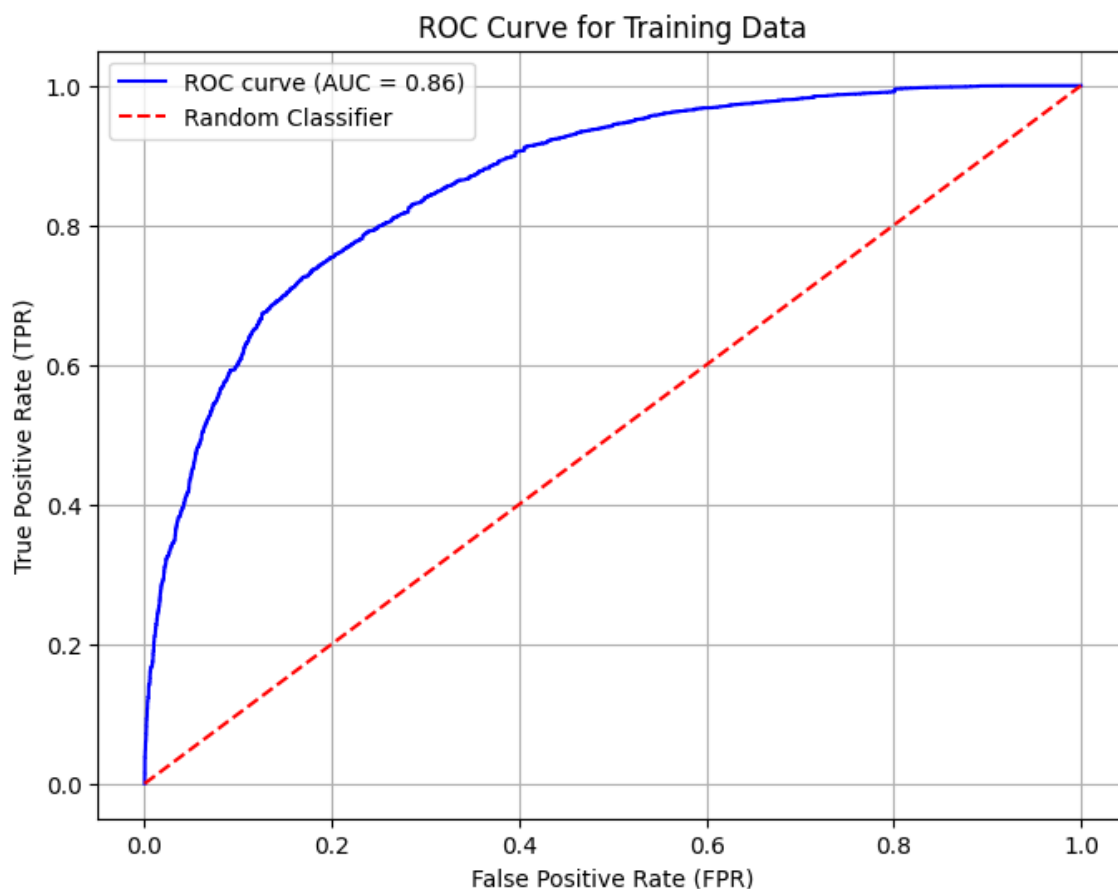
```
In [58]: # Comparing thresholds training performance
def get_metrics_for_threshold(model, X, y, threshold_value):
    """Calculates and returns a dictionary of metrics for a given
    threshold."""
    y_pred_proba = model.predict(X)
    y_pred = (y_pred_proba >= threshold_value).astype(int)

    return {
        'Accuracy': accuracy_score(y, y_pred),
        'Precision': precision_score(y, y_pred),
        'Recall': recall_score(y, y_pred),
        'F1 Score': f1_score(y, y_pred)
    }
```

```
In [59]: #checking the roc and auc on training set
y_train_pred_proba = lg2.predict(X_train_filtered)
roc_auc_train = roc_auc_score(y_train, y_train_pred_proba)
fpr, tpr, thresholds = roc_curve(y_train, y_train_pred_proba)

plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, color='blue', label=f'ROC curve (AUC =
{roc_auc_train:.2f})')
plt.plot([0, 1], [0, 1], color='red', linestyle='--', label='Random
Classifier')
plt.xlabel('False Positive Rate (FPR)')
plt.ylabel('True Positive Rate (TPR)')
```

```
plt.title('ROC Curve for Training Data')
plt.legend()
plt.grid(True)
plt.show()
```



```
In [60]: # Calculate ROC curve metrics for the training data
y_train_pred_proba = lg2.predict(X_train_filtered)
fpr, tpr, thresholds = roc_curve(y_train, y_train_pred_proba)

# Find the optimal threshold
optimal_idx = np.argmax(np.sqrt(tpr * (1 - fpr)))
optimal_threshold = thresholds[optimal_idx]

print(f"Optimal Threshold: {optimal_threshold:.4f}")
```

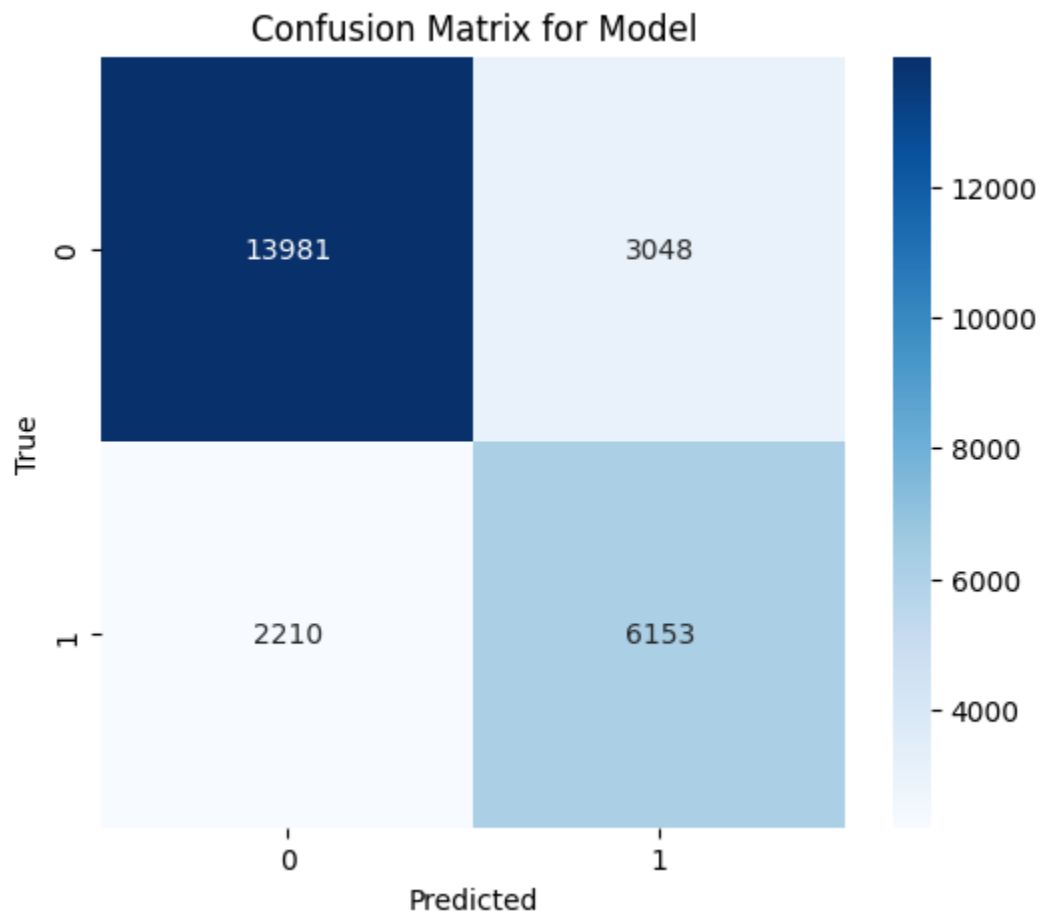
Optimal Threshold: 0.3710

```
In [61]: print("Training metrics with Optimal Threshold:")
metrics_optimal_train = get_metrics_for_threshold(lg2, X_train_filtered,
y_train, optimal_threshold)
print(f"Accuracy: {metrics_optimal_train['Accuracy']:.4f}")
print(f"Precision: {metrics_optimal_train['Precision']:.4f}")
print(f"Recall: {metrics_optimal_train['Recall']:.4f}")
print(f"F1 Score: {metrics_optimal_train['F1 Score']:.4f}")
```

Training metrics with Optimal Threshold:
Accuracy: 0.7929

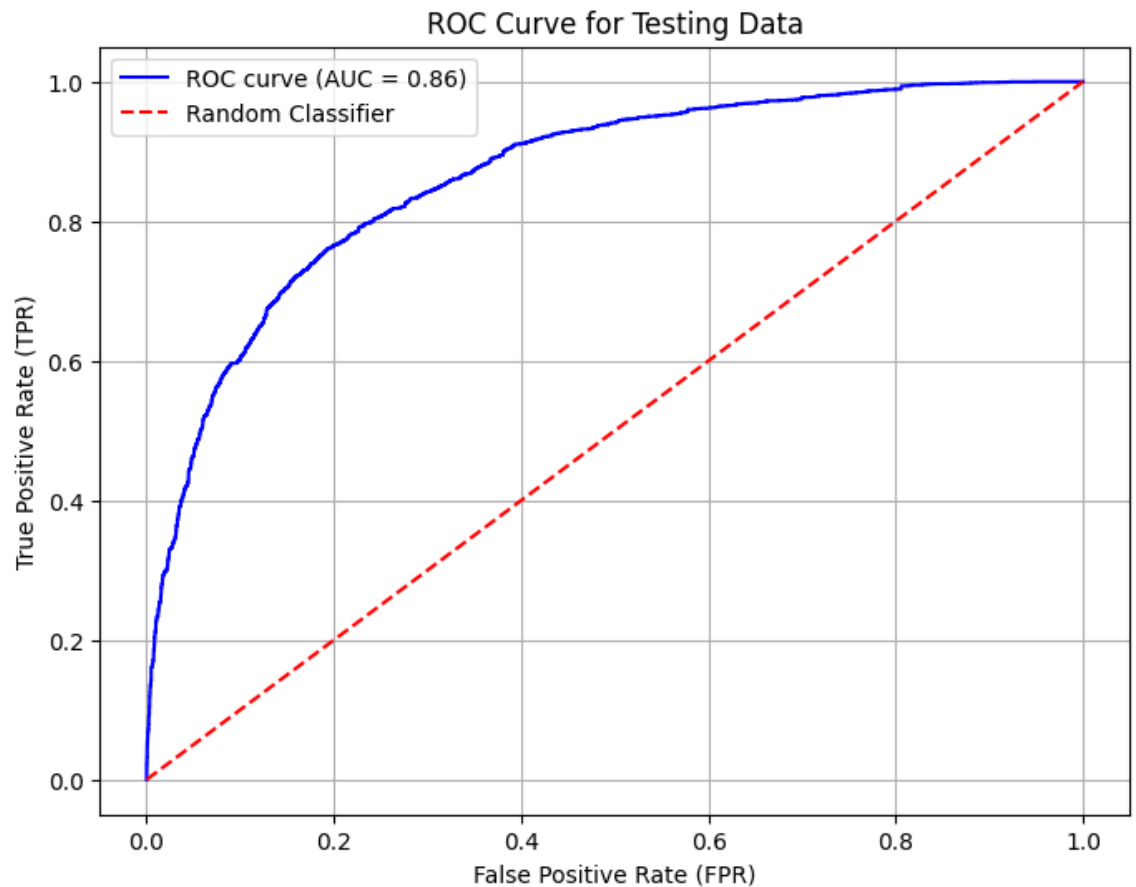
Precision: 0.6687
Recall: 0.7357
F1 Score: 0.7006

```
In [62]: #Training set matrix using Optimal
y_train_pred_optimal = (lg2.predict(X_train_filtered) >=
optimal_threshold).astype(int)
plot_confusion_matrix(y_train, y_train_pred_optimal)
```



```
In [63]: #roc auc curve on the testing set
y_test_pred_proba = lg2.predict(X_test_filtered)
roc_auc_test = roc_auc_score(y_test, y_test_pred_proba)
fpr, tpr, thresholds = roc_curve(y_test, y_test_pred_proba)

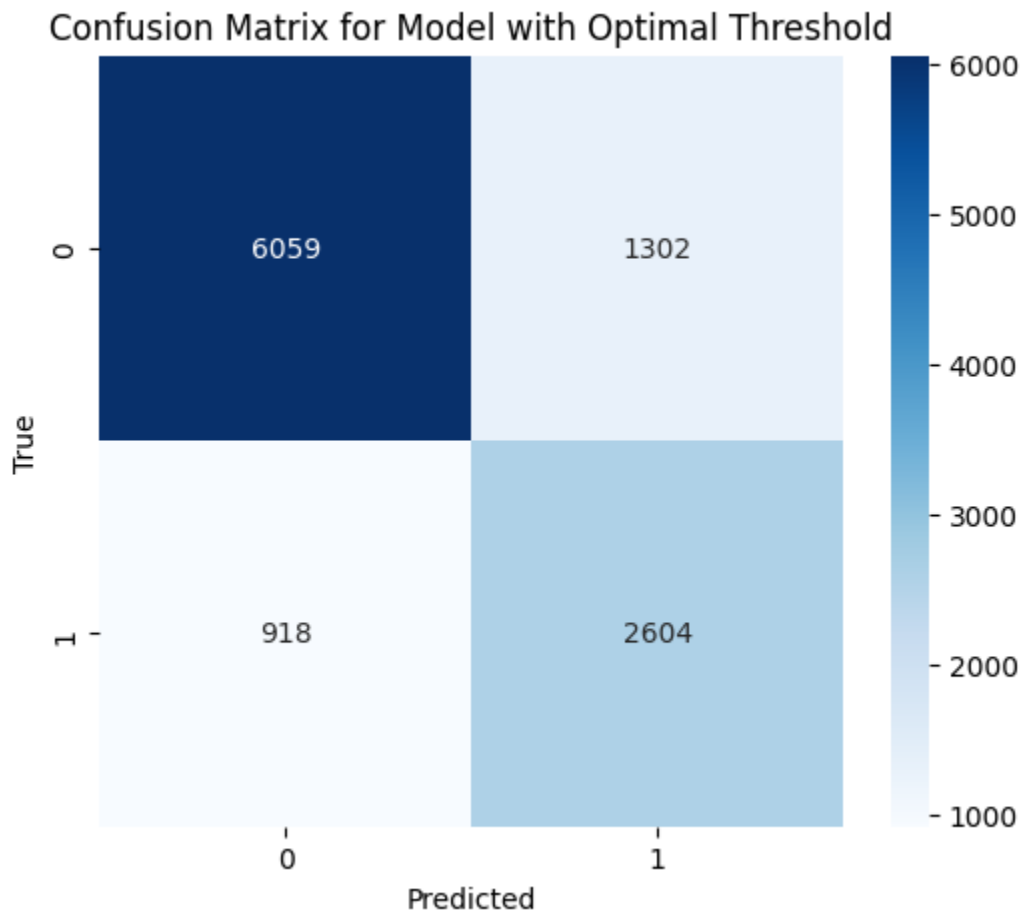
plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, color='blue', label=f'ROC curve (AUC =
{roc_auc_test:.2f})')
plt.plot([0, 1], [0, 1], color='red', linestyle='--', label='Random
Classifier')
plt.xlabel('False Positive Rate (FPR)')
plt.ylabel('True Positive Rate (TPR)')
plt.title('ROC Curve for Testing Data')
plt.legend()
plt.grid(True)
plt.show()
```



```
In
[64]: print("Test metrics with Optimal Threshold:")
      metrics_optimal_test = get_metrics_for_threshold(lg2, X_test_filtered, y_test,
      optimal_threshold)
      print(f"Accuracy: {metrics_optimal_test['Accuracy']:.4f}")
      print(f"Precision: {metrics_optimal_test['Precision']:.4f}")
      print(f"Recall: {metrics_optimal_test['Recall']:.4f}")
      print(f"F1 Score: {metrics_optimal_test['F1 Score']:.4f}")
```

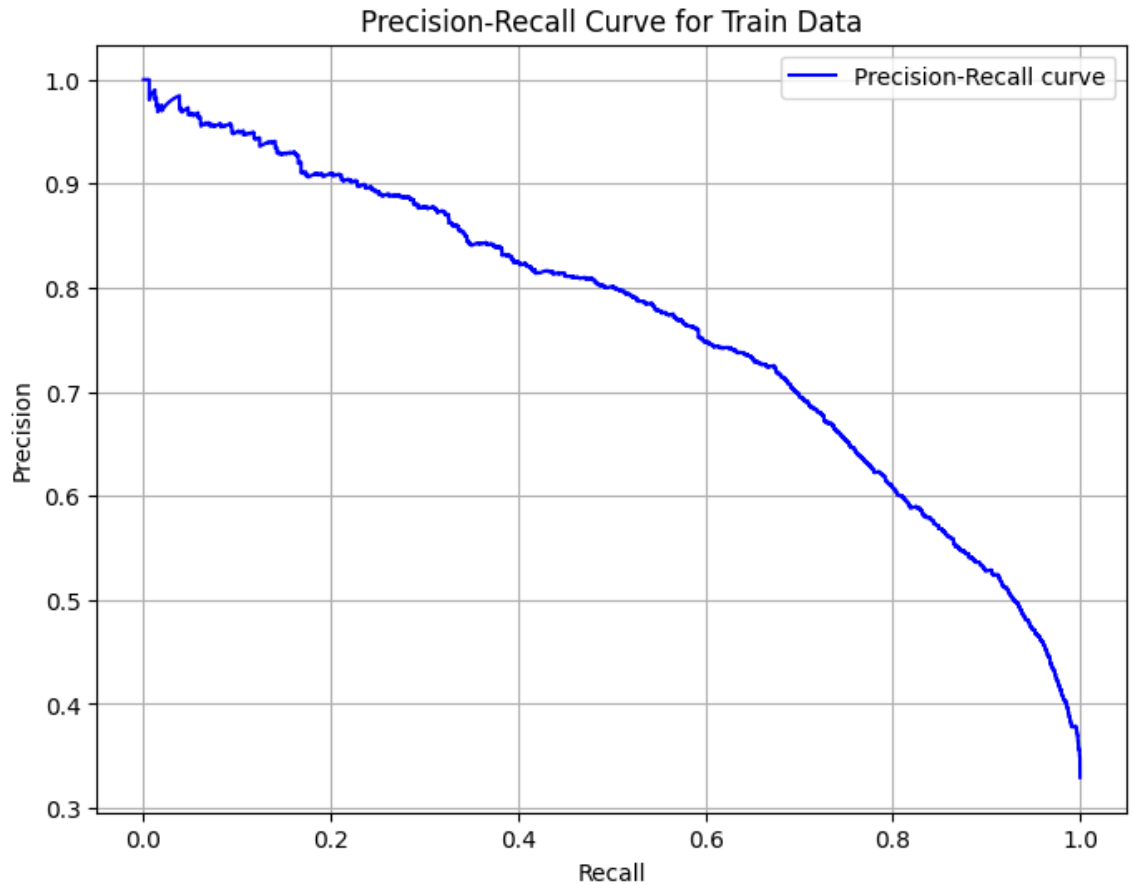
```
Test metrics with Optimal Threshold:
Accuracy: 0.7960
Precision: 0.6667
Recall: 0.7394
F1 Score: 0.7011
```

```
In
[65]: y_test_pred_optimal = (lg2.predict(X_test_filtered) >=
      optimal_threshold).astype(int)
      # Test Confusion Matrix with Optimal Threshold
      plot_confusion_matrix(y_test, y_test_pred_optimal, model_name="Model with
      Optimal Threshold")
```



```
In [66]: y_train_pred_proba = lg2.predict(X_train_filtered)
precision, recall, _ = precision_recall_curve(y_train, y_train_pred_proba)

plt.figure(figsize=(8, 6))
plt.plot(recall, precision, color='blue', label='Precision-Recall curve')
plt.xlabel('Recall')
plt.ylabel('Precision')
plt.title('Precision-Recall Curve for Train Data')
plt.grid(True)
plt.legend()
plt.show()
```



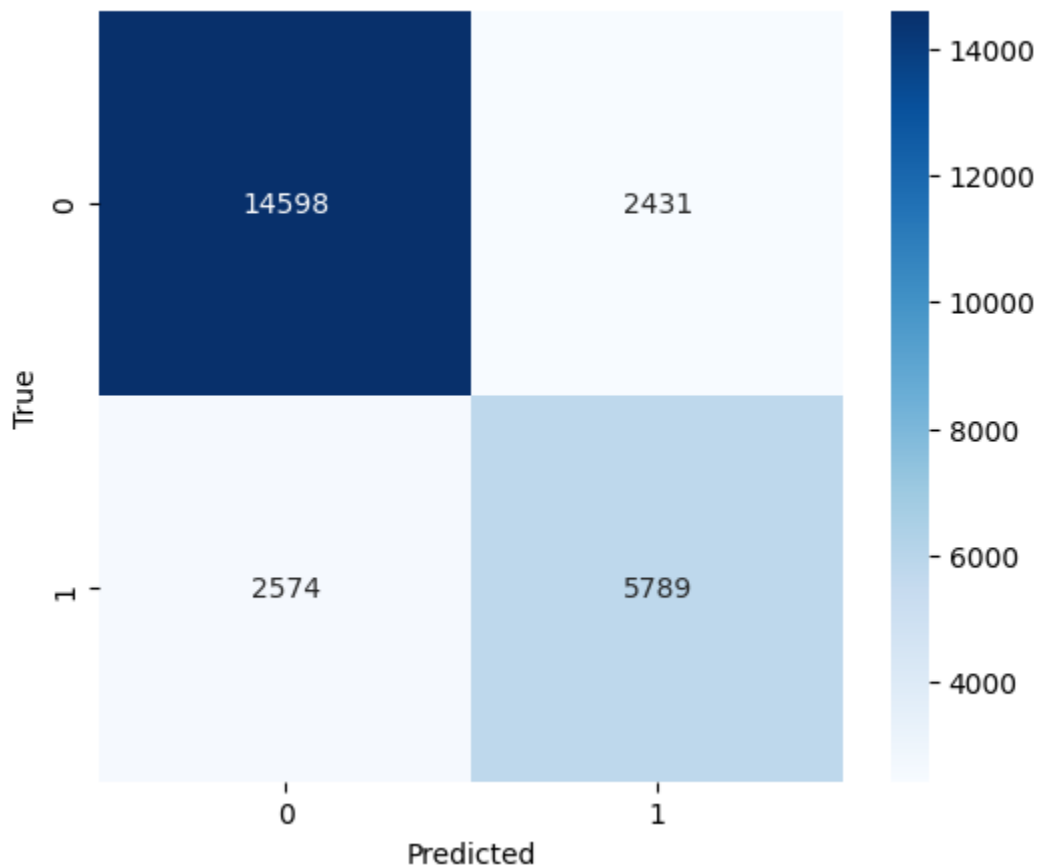
```
In [67]: user_specified_custom_threshold = 0.43

# Training set metrics with custom threshold
print(f"Training metrics with custom Threshold
({user_specified_custom_threshold}):")
metrics_custom_train = get_metrics_for_threshold(lg2, X_train_filtered,
y_train, user_specified_custom_threshold)
print(f"Accuracy: {metrics_custom_train['Accuracy']:.4f}")
print(f"Precision: {metrics_custom_train['Precision']:.4f}")
print(f"Recall: {metrics_custom_train['Recall']:.4f}")
print(f"F1 Score: {metrics_custom_train['F1 Score']:.4f}")
```

```
Training metrics with custom Threshold (0.43):
Accuracy: 0.8029
Precision: 0.7043
Recall: 0.6922
F1 Score: 0.6982
```

```
In [68]: # Training Confusion Matrix with custom Threshold
y_train_pred_custom = (lg2.predict(X_train_filtered) >=
user_specified_custom_threshold).astype(int)
plot_confusion_matrix(y_train, y_train_pred_custom, model_name=f"Model with
{user_specified_custom_threshold} Threshold (Train)")
```

Confusion Matrix for Model with 0.43 Threshold (Train)

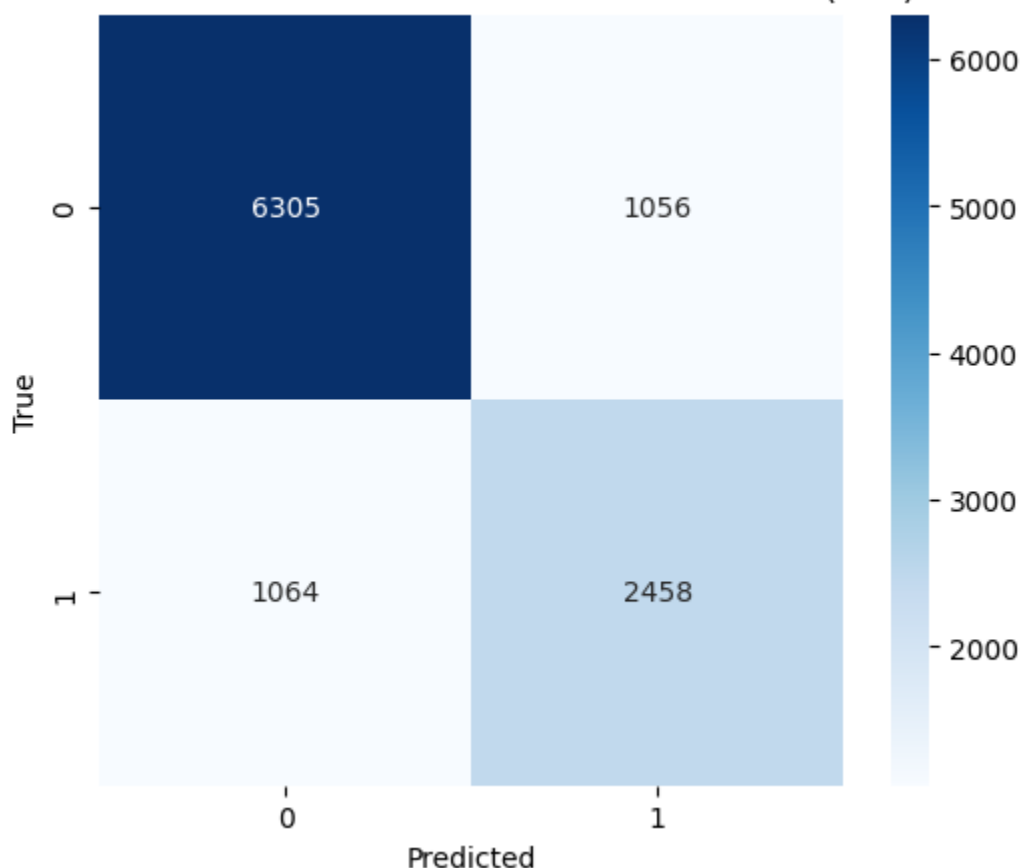


```
In [69]: # Test set metrics with custom threshold
print(f"Test metrics with custom Threshold
({user_specified_custom_threshold}):")
metrics_custom_test = get_metrics_for_threshold(lg2, X_test_filtered, y_test,
user_specified_custom_threshold)
print(f"Accuracy: {metrics_custom_test['Accuracy']:.4f}")
print(f"Precision: {metrics_custom_test['Precision']:.4f}")
print(f"Recall: {metrics_custom_test['Recall']:.4f}")
print(f"F1 Score: {metrics_custom_test['F1 Score']:.4f}")
```

```
Test metrics with custom Threshold (0.43):
Accuracy: 0.8052
Precision: 0.6995
Recall: 0.6979
F1 Score: 0.6987
```

```
In [70]: # Test Confusion Matrix with custom Threshold
y_test_pred_custom = (lg2.predict(X_test_filtered) >=
user_specified_custom_threshold).astype(int)
plot_confusion_matrix(y_test, y_test_pred_custom, model_name=f"Model with
{user_specified_custom_threshold} Threshold (Test)")
```

Confusion Matrix for Model with 0.43 Threshold (Test)



Final Model Summary

```
In [71]: # Comparing thresholds testing performance
default_threshold = 0.5

log_reg_model_test_perf = pd.Series(
    get_metrics_for_threshold(lg2, X_test_filtered, y_test, default_threshold)
)

log_reg_model_test_perf_threshold_auc_roc = pd.Series(
    get_metrics_for_threshold(lg2, X_test_filtered, y_test, optimal_threshold)
)

log_reg_model_test_perf_threshold_curve = pd.Series(
    get_metrics_for_threshold(lg2, X_test_filtered, y_test,
user_specified_custom_threshold)
)

# Concatenate the Series into a DataFrame, where each Series becomes a column
models_test_comp_df = pd.concat(
    [
        log_reg_model_test_perf,
        log_reg_model_test_perf_threshold_auc_roc,
```

```

        log_reg_model_test_perf_threshold_curve,
    ],
    axis=1,
)

# Set the column names to reflect the thresholds
models_test_comp_df.columns = [
    "Default Threshold-0.05 ",
    "Optimal Threshold-0.37",
    "Custom Threshold-0.42"

]
# Optimal still has highest recall
display(models_test_comp_df.round(4))

```

	Default Threshold- 0.05	Optimal Threshold- 0.37	Custom Threshold- 0.42
Accuracy	0.80460	0.79600	0.80520
Precision	0.72900	0.66670	0.69950
Recall	0.63090	0.73940	0.69790
F1 Score	0.67640	0.70110	0.69870

Building a Decision Tree model

```

In [72]: # Preparing data for decision tree model
X = df.drop('booking_status', axis=1)
y = y = df['booking_status'].map({'Not_Canceled': 0, 'Canceled': 1})

X = sm.add_constant(X)
X = pd.get_dummies(X, drop_first=True)

# Convert all boolean columns to int type
for col in X.select_dtypes(include=['bool']).columns:
    X[col] = X[col].astype(int)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=1)

```

```

In [73]: # new function to get decision tree metrics
def get_metrics_decision_tree(model, X, y, threshold=0.5):

    y_pred_proba = model.predict_proba(X)[:, 1]
    y_pred = (y_pred_proba >= threshold).astype(int)

    accuracy = accuracy_score(y, y_pred)
    precision = precision_score(y, y_pred)

```

```
recall = recall_score(y, y_pred)
f1 = f1_score(y, y_pred)

return {
    'Accuracy': accuracy,
    'Precision': precision,
    'Recall': recall,
    'F1 Score': f1
}
```

```
In [74]: #Fitting decision tree model
model = DecisionTreeClassifier(random_state=1)
model.fit(X_train, y_train)
```

Out[74]:

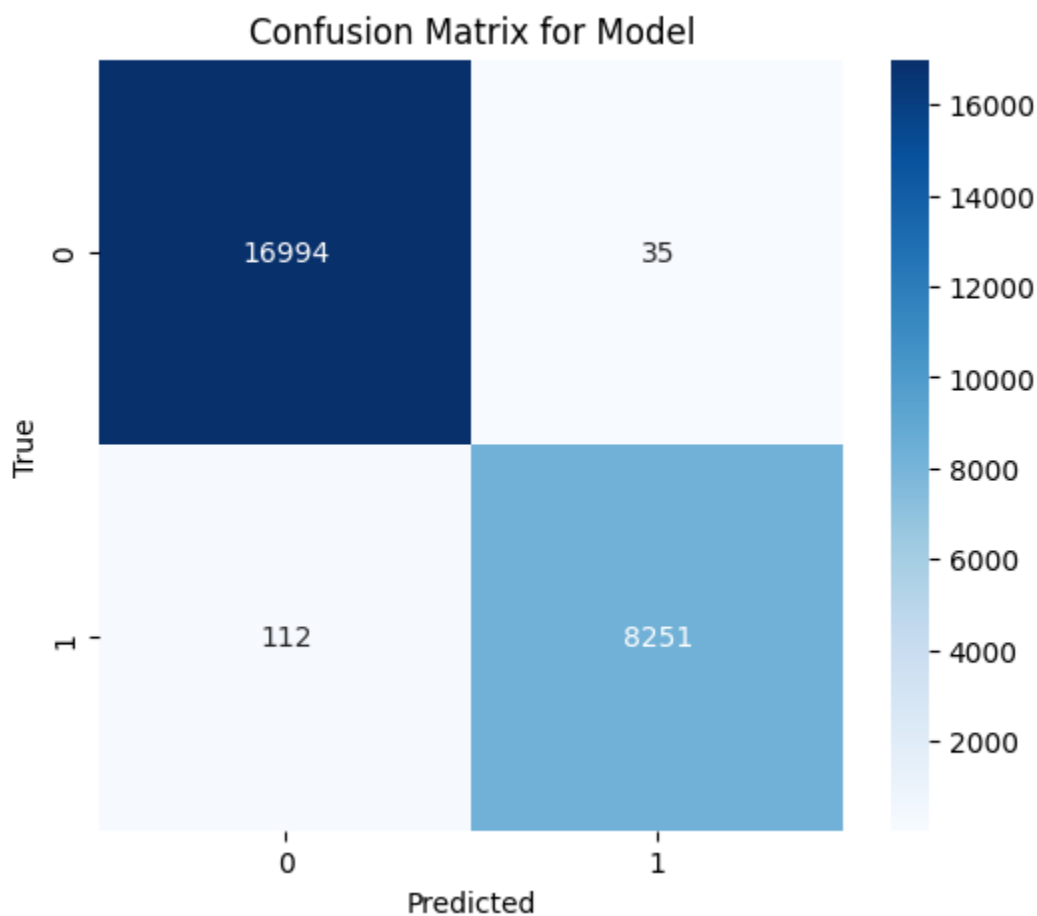
▼ DecisionTreeClassifier ⓘ ?

DecisionTreeClassifier(random_state=1)

```
In [75]: #Training Performance
get_metrics_decision_tree(model, X_train, y_train)
```

Out[75]: {'Accuracy': 0.994210775047259,
 'Precision': 0.9913875598086125,
 'Recall': 0.9910319263422217,
 'F1 Score': 0.9912097111762244}

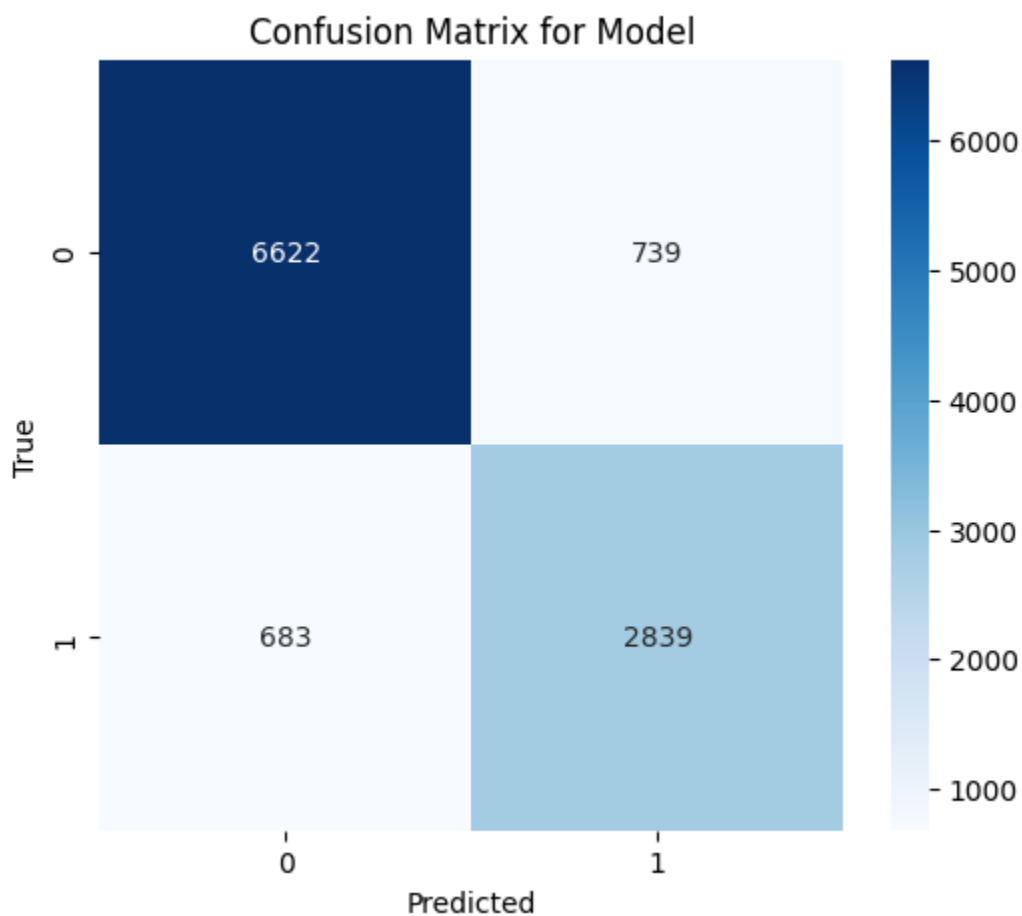
```
In [76]: #Training Matrix
y_train_pred = model.predict(X_train)
plot_confusion_matrix(y_train, y_train_pred)
```



```
In [77]: #Testing performance  
get_metrics_decision_tree(model, X_test, y_test)
```

```
Out[77]: {'Accuracy': 0.8685105209960489,  
          'Precision': 0.7898530634876629,  
          'Recall': 0.8089153889835321,  
          'F1 Score': 0.7992705849347734}
```

```
In [78]: #Testing Matrix  
y_test_pred = model.predict(X_test)  
plot_confusion_matrix(y_test, y_test_pred)
```



```
In [79]: # Checking important features
print("Feature Importance for Decision Tree Model:")
importance_df = pd.DataFrame({'Feature': X_train.columns, 'Importance':
model.feature_importances_})
importance_df = importance_df.sort_values(by='Importance', ascending=False)
print(importance_df)
```

```
Feature Importance for Decision Tree Model:
              Feature  Importance
6              lead_time    0.35319
13          avg_price_per_room    0.16866
28    market_segment_type_Online    0.09310
9              arrival_date    0.08352
14    no_of_special_requests    0.06785
8              arrival_month    0.06225
15            total_nights    0.03227
3      no_of_weekend_nights    0.03207
4      no_of_week_nights    0.03025
1              no_of_adults    0.02655
7              arrival_year    0.01146
21  room_type_reserved_Room_Type 4    0.00764
18  type_of_meal_plan_Not Selected    0.00758
5      required_car_parking_space    0.00687
16  type_of_meal_plan_Meal Plan 2    0.00374
2              no_of_children    0.00373
27    market_segment_type_Offline    0.00285
19  room_type_reserved_Room_Type 2    0.00204
```

22	room_type_reserved_Room_Type 5	0.00171
23	room_type_reserved_Room_Type 6	0.00067
12	no_of_previous_bookings_not_canceled	0.00058
10	repeated_guest	0.00054
26	market_segment_type_Corporate	0.00044
24	room_type_reserved_Room_Type 7	0.00028
25	market_segment_type_Complementary	0.00016
20	room_type_reserved_Room_Type 3	0.00001
0	const	0.00000
11	no_of_previous_cancellations	0.00000
17	type_of_meal_plan_Meal Plan 3	0.00000

#Pruning

```
In [80]: #Pre-pruning using grid search
param_grid = {
    'max_depth': np.arange(2, 10),
    'min_samples_leaf': [2, 5, 10, 15],
    'min_samples_split': [2, 5, 10, 20],
    'max_leaf_nodes': [20, 50, 100, None]
}

dt_model = DecisionTreeClassifier(
    random_state=1,
    class_weight="balanced" # only if classes are imbalanced
)

grid_search = GridSearchCV(
    estimator=dt_model,
    param_grid=param_grid,
    cv=5,
    scoring='f1',
    n_jobs=-1
)

grid_search.fit(X_train, y_train)

best_dt_model = grid_search.best_estimator_

print(grid_search.best_params_)
print(grid_search.best_score_)
```

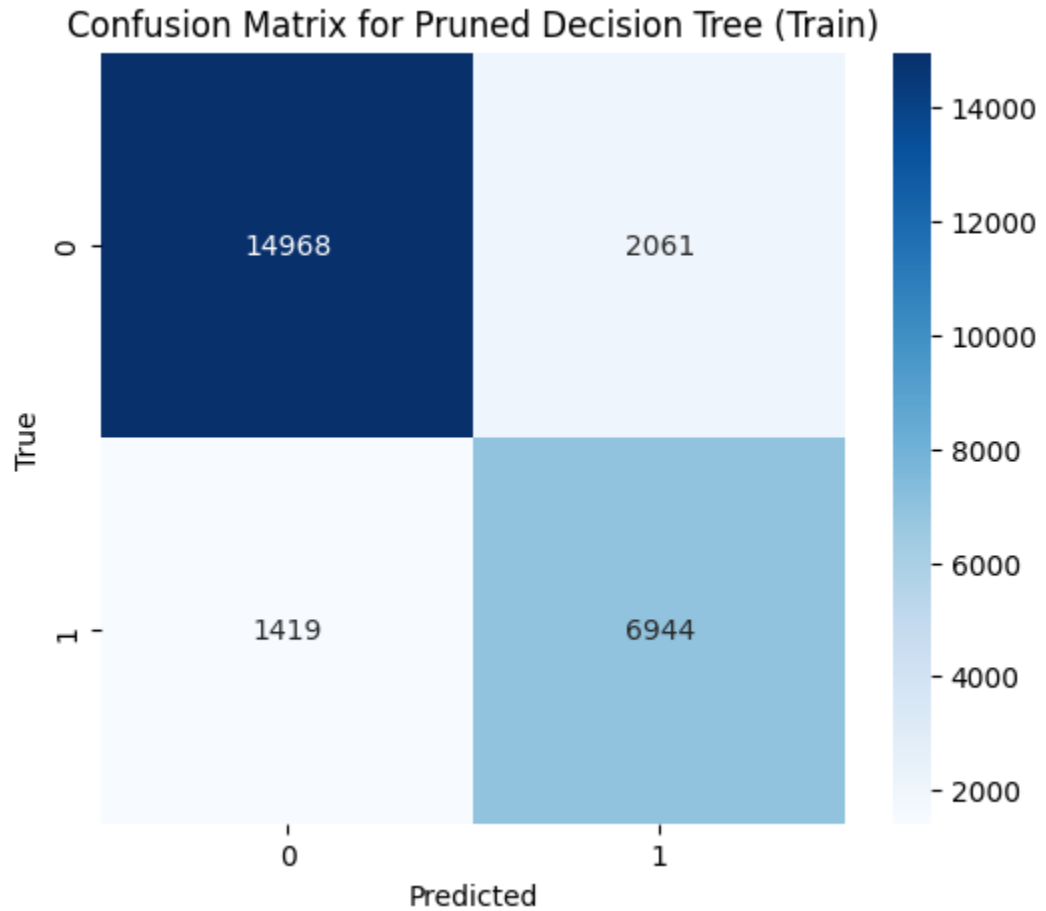
```
{'max_depth': np.int64(9), 'max_leaf_nodes': 100, 'min_samples_leaf': 2,
 'min_samples_split': 2}
0.7807634101024089
```

```
In [81]: # Train Performance Pre-Pruned Model
get_metrics_decision_tree(best_dt_model, X_train, y_train)
```

```
Out[81]: {'Accuracy': 0.8629489603024575,
 'Precision': 0.7711271515824542,
```

```
'Recall': 0.8303240463948344,  
'F1 Score': 0.7996315062183326}
```

```
In [82]: # Train Matrix Pre-Pruned Model  
y_train_pred_pruned = best_dt_model.predict(X_train)  
plot_confusion_matrix(y_train, y_train_pred_pruned, model_name="Pruned  
Decision Tree (Train)")
```

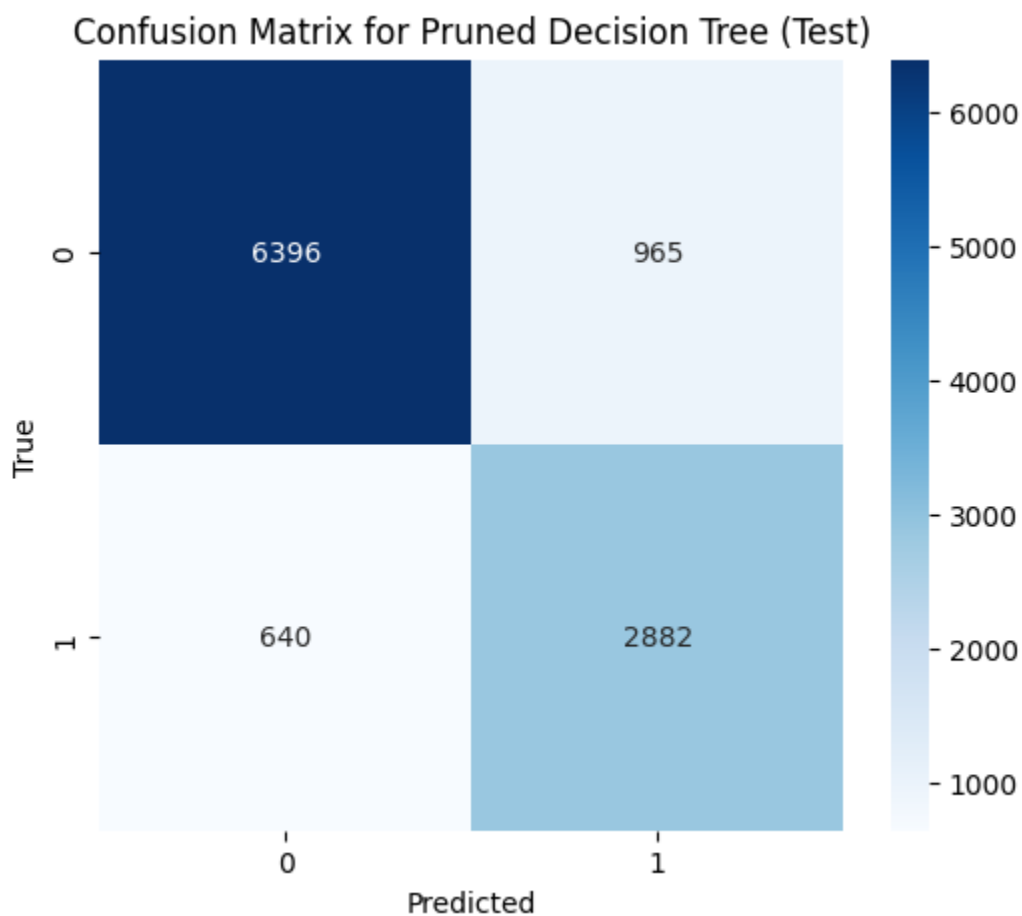


```
In [83]: # Test Performance Pre-Pruned Model  
print("Testing Performance of Pruned Decision Tree:")  
get_metrics_decision_tree(best_dt_model, X_test, y_test)
```

Testing Performance of Pruned Decision Tree:

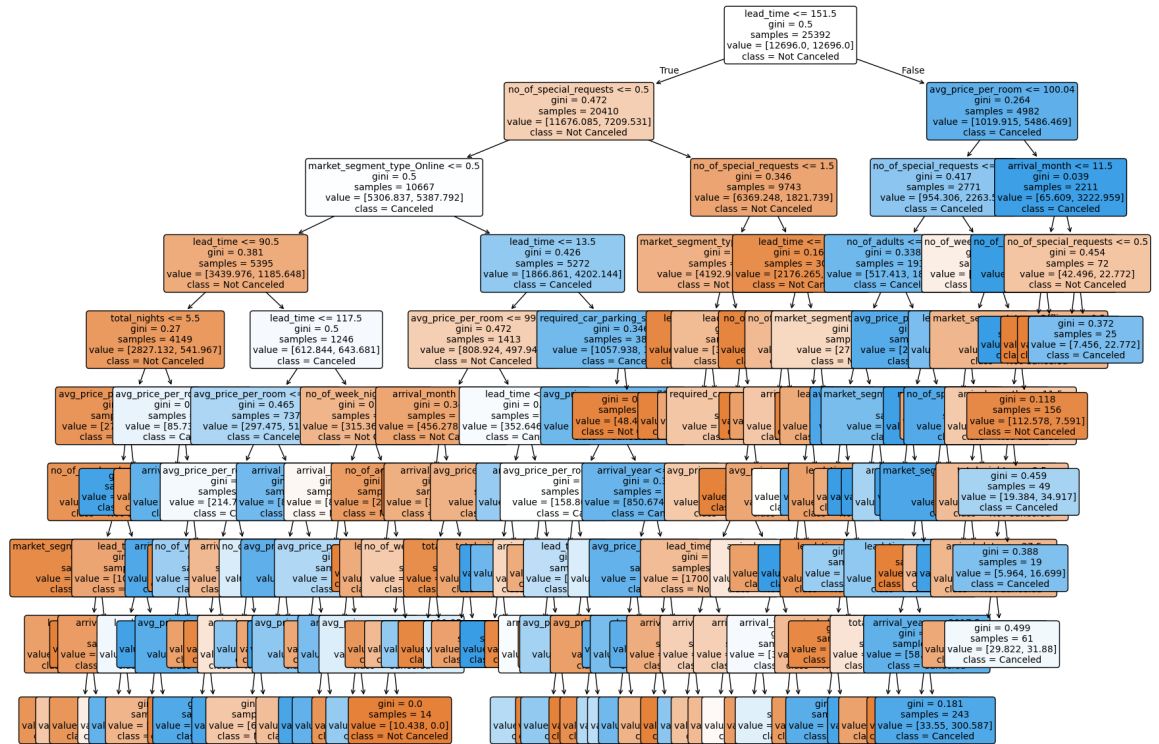
```
Out[83]: {'Accuracy': 0.8525222824588808,  
          'Precision': 0.749155185859111,  
          'Recall': 0.8182850653038046,  
          'F1 Score': 0.7821956846247795}
```

```
In [84]: # Test Matrix Pre-Pruned Model  
y_test_pred_pruned = best_dt_model.predict(X_test)  
plot_confusion_matrix(y_test, y_test_pred_pruned, model_name="Pruned Decision  
Tree (Test)")
```



```
In [85]: #Finished Pre-Pruned Tree
plt.figure(figsize=(20, 15))
tree.plot_tree(best_dt_model,
                feature_names=X_train.columns,
                class_names=['Not Canceled', 'Canceled'],
                filled=True,
                rounded=True,
                fontsize=10)
plt.title('Pruned Decision Tree Visualization')
plt.show()
```

Pruned Decision Tree Visualization



```
In [86]: print("\nText Report of Pruned Decision Tree:")
r = tree.export_text(best_dt_model, feature_names=list(X_train.columns),
class_names=['Not Canceled', 'Canceled'])
print(r)
```

Text Report of Pruned Decision Tree:

```
|--- lead_time <= 151.50
|   |--- no_of_special_requests <= 0.50
|   |   |--- market_segment_type_Online <= 0.50
|   |   |   |--- lead_time <= 90.50
|   |   |   |   |--- total_nights <= 5.50
|   |   |   |   |   |--- avg_price_per_room <= 201.50
|   |   |   |   |   |   |--- no_of_weekend_nights <= 0.50
|   |   |   |   |   |   |   |--- market_segment_type_Offline <= 0.50
|   |   |   |   |   |   |   |   |--- lead_time <= 16.50
|   |   |   |   |   |   |   |   |   |--- class: Not Canceled
|   |   |   |   |   |   |   |   |   |--- lead_time > 16.50
|   |   |   |   |   |   |   |   |   |   |--- class: Not Canceled
|   |   |   |   |   |   |   |--- market_segment_type_Offline > 0.50
|   |   |   |   |   |   |   |   |--- class: Not Canceled
|   |   |   |   |   |--- no_of_weekend_nights > 0.50
|   |   |   |   |   |   |--- lead_time <= 74.50
|   |   |   |   |   |   |   |--- arrival_date <= 27.50
|   |   |   |   |   |   |   |   |--- class: Not Canceled
|   |   |   |   |   |   |   |   |--- arrival_date > 27.50
|   |   |   |   |   |   |   |   |   |--- class: Not Canceled
|   |   |   |   |   |--- lead_time > 74.50
|   |   |   |   |   |   |--- lead_time <= 78.50
|   |   |   |   |   |   |   |--- class: Canceled
|   |   |   |   |   |   |   |--- lead_time > 78.50
```

[illegible]

```
| | | | | | | | |--- arrival_date <= 28.00  
| | | | | | | | | |--- class: Canceled  
| | | | | | | | | |--- arrival_date > 28.00  
| | | | | | | | | |--- class: Not Canceled  
| | | | | | | | |--- no_of_week_nights > 1.50  
| | | | | | | | | |--- no_of_adults <= 1.50  
| | | | | | | | | |--- class: Not Canceled  
| | | | | | | | |--- no_of_adults > 1.50  
| | | | | | | | | |--- lead_time <= 125.50  
| | | | | | | | | | |--- avg_price_per_room <= 90.85  
| | | | | | | | | | | |--- class: Canceled  
| | | | | | | | | | | |--- avg_price_per_room > 90.85  
| | | | | | | | | | | |--- class: Not Canceled  
| | | | | | | | | |--- lead_time > 125.50  
| | | | | | | | | |--- class: Not Canceled  
| | | |--- market_segment_type_Online > 0.50  
| | | |--- lead_time <= 13.50  
| | | | |--- avg_price_per_room <= 99.44  
| | | | |--- arrival_month <= 1.50  
| | | | | |--- class: Not Canceled  
| | | | |--- arrival_month > 1.50  
| | | | |--- arrival_month <= 8.50  
| | | | | |--- no_of_weekend_nights <= 1.50  
| | | | | | |--- class: Not Canceled  
| | | | | |--- no_of_weekend_nights > 1.50  
| | | | | | |--- class: Canceled  
| | | | |--- arrival_month > 8.50  
| | | | |--- total_nights <= 5.50  
| | | | | |--- class: Not Canceled  
| | | | |--- total_nights > 5.50  
| | | | | |--- class: Canceled  
| | | | |--- avg_price_per_room > 99.44  
| | | | |--- lead_time <= 3.50  
| | | | | |--- avg_price_per_room <= 202.67  
| | | | | |--- total_nights <= 6.50  
| | | | | | |--- class: Not Canceled  
| | | | | |--- total_nights > 6.50  
| | | | | | |--- class: Canceled  
| | | | |--- avg_price_per_room > 202.67  
| | | | |--- class: Canceled  
| | | | |--- lead_time > 3.50  
| | | | |--- arrival_month <= 8.50  
| | | | | |--- class: Canceled  
| | | | |--- arrival_month > 8.50  
| | | | |--- arrival_year <= 2017.50  
| | | | | |--- class: Not Canceled  
| | | | |--- arrival_year > 2017.50  
| | | | | |--- arrival_month <= 11.50  
| | | | | | |--- class: Canceled  
| | | | | |--- arrival_month > 11.50  
| | | | | | |--- class: Not Canceled  
| | | |--- lead_time > 13.50  
| | | |--- required_car_parking_space <= 0.50  
| | | |--- avg_price_per_room <= 71.92  
| | | |--- avg price per room <= 59.43
```

```

| | | | | | | |--- class: Not Canceled
| | | | | | | |--- avg_price_per_room > 59.43
| | | | | | | |--- lead_time <= 25.50
| | | | | | | |--- class: Not Canceled
| | | | | | | |--- lead_time > 25.50
| | | | | | | |--- avg_price_per_room <= 71.34
| | | | | | | |--- class: Canceled
| | | | | | | |--- avg_price_per_room > 71.34
| | | | | | | |--- class: Not Canceled
| | | | | | | |--- avg_price_per_room > 71.92
| | | | | | | |--- arrival_year <= 2017.50
| | | | | | | |--- lead_time <= 65.50
| | | | | | | |--- avg_price_per_room <= 120.45
| | | | | | | |--- class: Not Canceled
| | | | | | | |--- avg_price_per_room > 120.45
| | | | | | | |--- class: Canceled
| | | | | | | |--- lead_time > 65.50
| | | | | | | |--- class: Canceled
| | | | | | | |--- arrival_year > 2017.50
| | | | | | | |--- avg_price_per_room <= 104.31
| | | | | | | |--- lead_time <= 25.50
| | | | | | | |--- class: Canceled
| | | | | | | |--- lead_time > 25.50
| | | | | | | |--- class: Canceled
| | | | | | | |--- avg_price_per_room > 104.31
| | | | | | | |--- class: Canceled
| | | | | | | |--- required_car_parking_space > 0.50
| | | | | | | |--- class: Not Canceled
| | | | | | | |--- no_of_special_requests > 0.50
| | | | | | | |--- no_of_special_requests <= 1.50
| | | | | | | |--- market_segment_type_Online <= 0.50
| | | | | | | |--- lead_time <= 102.50
| | | | | | | |--- class: Not Canceled
| | | | | | | |--- lead_time > 102.50
| | | | | | | |--- class: Not Canceled
| | | | | | | |--- market_segment_type_Online > 0.50
| | | | | | | |--- lead_time <= 8.50
| | | | | | | |--- class: Not Canceled
| | | | | | | |--- lead_time > 8.50
| | | | | | | |--- required_car_parking_space <= 0.50
| | | | | | | |--- avg_price_per_room <= 118.55
| | | | | | | |--- lead_time <= 61.50
| | | | | | | |--- arrival_month <= 11.50
| | | | | | | |--- class: Not Canceled
| | | | | | | |--- arrival_month > 11.50
| | | | | | | |--- class: Not Canceled
| | | | | | | |--- lead_time > 61.50
| | | | | | | |--- arrival_year <= 2017.50
| | | | | | | |--- class: Canceled
| | | | | | | |--- arrival_year > 2017.50
| | | | | | | |--- class: Not Canceled
| | | | | | | |--- avg_price_per_room > 118.55
| | | | | | | |--- arrival_month <= 8.50
| | | | | | | |--- arrival_date <= 19.50
| | | | | | | |--- class: Not Canceled

```

```
| | | | | | | | |--- arrival_date> 19.50  
| | | | | | | | | | | | |--- class: Not Canceled  
| | | | | | | | | | | | |--- arrival_month > 8.50  
| | | | | | | | | | | | |--- arrival_year <= 2017.50  
| | | | | | | | | | | | |--- class: Not Canceled  
| | | | | | | | | | | | |--- arrival_year > 2017.50  
| | | | | | | | | | | | |--- class: Canceled  
| | | | | | | | | | | | |--- required_car_parking_space > 0.50  
| | | | | | | | | | | | |--- class: Not Canceled  
| | | | | | | | | | | | |--- no_of_special_requests > 1.50  
| | | | | | | | | | | | |--- lead_time <= 90.50  
| | | | | | | | | | | | |--- no_of_week_nights <= 3.50  
| | | | | | | | | | | | |--- class: Not Canceled  
| | | | | | | | | | | | |--- no_of_week_nights > 3.50  
| | | | | | | | | | | | |--- class: Not Canceled  
| | | | | | | | | | | | |--- lead_time > 90.50  
| | | | | | | | | | | | |--- no_of_special_requests <= 2.50  
| | | | | | | | | | | | |--- arrival_month <= 8.50  
| | | | | | | | | | | | |--- avg_price_per_room <= 202.95  
| | | | | | | | | | | | |--- class: Not Canceled  
| | | | | | | | | | | | |--- avg_price_per_room > 202.95  
| | | | | | | | | | | | |--- class: Canceled  
| | | | | | | | | | | | |--- arrival_month > 8.50  
| | | | | | | | | | | | |--- class: Not Canceled  
| | | | | | | | | | | | |--- no_of_special_requests > 2.50  
| | | | | | | | | | | | |--- class: Not Canceled  
|--- lead_time > 151.50  
| | | | | | | | | | | | |--- avg_price_per_room <= 100.04  
| | | | | | | | | | | | |--- no_of_special_requests <= 0.50  
| | | | | | | | | | | | |--- no_of_adults <= 1.50  
| | | | | | | | | | | | |--- market_segment_type_Online <= 0.50  
| | | | | | | | | | | | |--- lead_time <= 163.50  
| | | | | | | | | | | | |--- class: Canceled  
| | | | | | | | | | | | |--- lead_time > 163.50  
| | | | | | | | | | | | |--- lead_time <= 341.00  
| | | | | | | | | | | | |--- lead_time <= 173.00  
| | | | | | | | | | | | |--- arrival_date <= 3.50  
| | | | | | | | | | | | |--- class: Not Canceled  
| | | | | | | | | | | | |--- arrival_date > 3.50  
| | | | | | | | | | | | |--- class: Canceled  
| | | | | | | | | | | | |--- lead_time > 173.00  
| | | | | | | | | | | | |--- class: Not Canceled  
| | | | | | | | | | | | |--- lead_time > 341.00  
| | | | | | | | | | | | |--- class: Canceled  
| | | | | | | | | | | | |--- market_segment_type_Online > 0.50  
| | | | | | | | | | | | |--- avg_price_per_room <= 2.50  
| | | | | | | | | | | | |--- class: Not Canceled  
| | | | | | | | | | | | |--- avg_price_per_room > 2.50  
| | | | | | | | | | | | |--- class: Canceled  
| | | | | | | | | | | | |--- no_of_adults > 1.50  
| | | | | | | | | | | | |--- avg_price_per_room <= 82.47  
| | | | | | | | | | | | |--- market_segment_type_Offline <= 0.50  
| | | | | | | | | | | | |--- class: Canceled  
| | | | | | | | | | | | |--- market_segment_type_Offline > 0.50  
| | | | | | | | | | | | |--- arrival_month<= 11.50
```

```
| | | | | | | | |--- lead_time <= 244.00  
| | | | | | | | |--- total_nights <= 2.50  
| | | | | | | | |   |--- class: Canceled  
| | | | | | | | |   |--- total_nights > 2.50  
| | | | | | | | |   |--- class: Not Canceled  
| | | | | | | | |--- lead_time > 244.00  
| | | | | | | | |   |--- arrival_year <= 2017.50  
| | | | | | | | |   |--- class: Not Canceled  
| | | | | | | | |   |--- arrival_year > 2017.50  
| | | | | | | | |   |--- class: Canceled  
| | | | | | | | |     |--- arrival_month > 11.50  
| | | | | | | | |     |--- class: Not Canceled  
| | | | | | | | |--- avg_price_per_room > 82.47  
| | | | | | | | |   |--- no_of_adults <= 2.50  
| | | | | | | | |   |--- class: Canceled  
| | | | | | | | |   |--- no_of_adults > 2.50  
| | | | | | | | |   |--- class: Not Canceled  
| | | | | | | | |--- no_of_special_requests > 0.50  
| | | | | | | | |   |--- no_of_weekend_nights <= 0.50  
| | | | | | | | |     |--- lead_time <= 180.50  
| | | | | | | | |     |--- class: Not Canceled  
| | | | | | | | |     |--- lead_time > 180.50  
| | | | | | | | |       |--- no_of_special_requests <= 2.50  
| | | | | | | | |       |   |--- market_segment_type_Online <= 0.50  
| | | | | | | | |       |   |   |--- class: Not Canceled  
| | | | | | | | |       |   |   |--- market_segment_type_Online > 0.50  
| | | | | | | | |       |   |   |   |--- class: Canceled  
| | | | | | | | |       |   |   |   |--- no_of_special_requests > 2.50  
| | | | | | | | |       |   |   |   |--- class: Not Canceled  
| | | | | | | | |     |--- no_of_weekend_nights > 0.50  
| | | | | | | | |       |--- market_segment_type_Offline <= 0.50  
| | | | | | | | |         |--- arrival_month <= 11.50  
| | | | | | | | |         |   |--- total_nights <= 9.50  
| | | | | | | | |         |   |   |--- arrival_date <= 27.50  
| | | | | | | | |         |   |   |   |--- class: Not Canceled  
| | | | | | | | |         |   |   |   |--- arrival_date > 27.50  
| | | | | | | | |         |   |   |   |   |--- class: Canceled  
| | | | | | | | |         |   |   |   |   |--- total_nights > 9.50  
| | | | | | | | |         |   |   |   |   |   |--- class: Canceled  
| | | | | | | | |         |   |   |   |   |   |--- arrival_month > 11.50  
| | | | | | | | |         |   |   |   |   |   |   |--- class: Canceled  
| | | | | | | | |       |   |   |   |--- market_segment_type_Offline > 0.50  
| | | | | | | | |       |   |   |   |   |--- class: Not Canceled  
| | | | | | | | |--- avg_price_per_room > 100.04  
| | | | | | | | |   |--- arrival_month <= 11.50  
| | | | | | | | |     |--- no_of_special_requests <= 2.50  
| | | | | | | | |     |   |--- class: Canceled  
| | | | | | | | |     |   |--- no_of_special_requests > 2.50  
| | | | | | | | |     |   |   |--- class: Not Canceled  
| | | | | | | | |   |--- arrival_month > 11.50  
| | | | | | | | |     |--- no_of_special_requests <= 0.50  
| | | | | | | | |     |   |--- class: Not Canceled  
| | | | | | | | |   |--- no_of_special_requests > 0.50  
| | | | | | | | |     |   |--- class: Canceled
```

```
In
[87]: #Lead_Time still most important feature
print("\nFeature Importance for Pruned Decision Tree Model:")
importance_df_pruned = pd.DataFrame({'Feature': X_train.columns, 'Importance':
best_dt_model.feature_importances_})
importance_df_pruned = importance_df_pruned.sort_values(by='Importance',
ascending=False)
print(importance_df_pruned)
```

```
Feature Importance for Pruned Decision Tree Model:
      Feature  Importance
6      lead_time    0.43287
28  market_segment_type_Online    0.16113
14    no_of_special_requests    0.14522
13    avg_price_per_room    0.09927
8      arrival_month    0.03349
1      no_of_adults    0.02423
7      arrival_year    0.01790
3    no_of_weekend_nights    0.01707
15    total_nights    0.01642
9      arrival_date    0.01434
27  market_segment_type_Offline    0.01318
4      no_of_week_nights    0.01272
5    required_car_parking_space    0.01216
12 no_of_previous_bookings_not_canceled    0.00000
11    no_of_previous_cancellations    0.00000
10    repeated_guest    0.00000
2      no_of_children    0.00000
0      const    0.00000
18  type_of_meal_plan_Not Selected    0.00000
17  type_of_meal_plan_Meal Plan 3    0.00000
16  type_of_meal_plan_Meal Plan 2    0.00000
20  room_type_reserved_Room_Type 3    0.00000
19  room_type_reserved_Room_Type 2    0.00000
21  room_type_reserved_Room_Type 4    0.00000
22  room_type_reserved_Room_Type 5    0.00000
24  room_type_reserved_Room_Type 7    0.00000
23  room_type_reserved_Room_Type 6    0.00000
26  market_segment_type_Corporate    0.00000
25  market_segment_type_Complementary    0.00000
```

```
In
[88]: #Post Pruning using cost-complexity pruning
# Calculate ccp_alphas
path = model.cost_complexity_pruning_path(X_train, y_train)
ccp_alphas, impurities = path.ccp_alphas, path.impurities

print("Available ccp_alphas:", ccp_alphas[:10]) # Display first 10 alphas
print("Number of ccp_alphas:", len(ccp_alphas))
```

```
Available ccp_alphas: [0.00000000e+00 1.21927191e-07 4.68839079e-07
5.32996006e-07
6.13354694e-07 7.77285842e-07 1.25023755e-06 1.73109814e-06
```

1.75870736e-06 1.87535632e-06]
Number of ccp_alphas: 1352

```
In [89]: # Train a decision tree for each ccp_alpha
dt_models = []
for ccp_alpha in ccp_alphas:
    dt_model = DecisionTreeClassifier(random_state=1, ccp_alpha=ccp_alpha)
    dt_model.fit(X_train, y_train)
    dt_models.append(dt_model)

print("Trained", len(dt_models), "decision trees.")
```

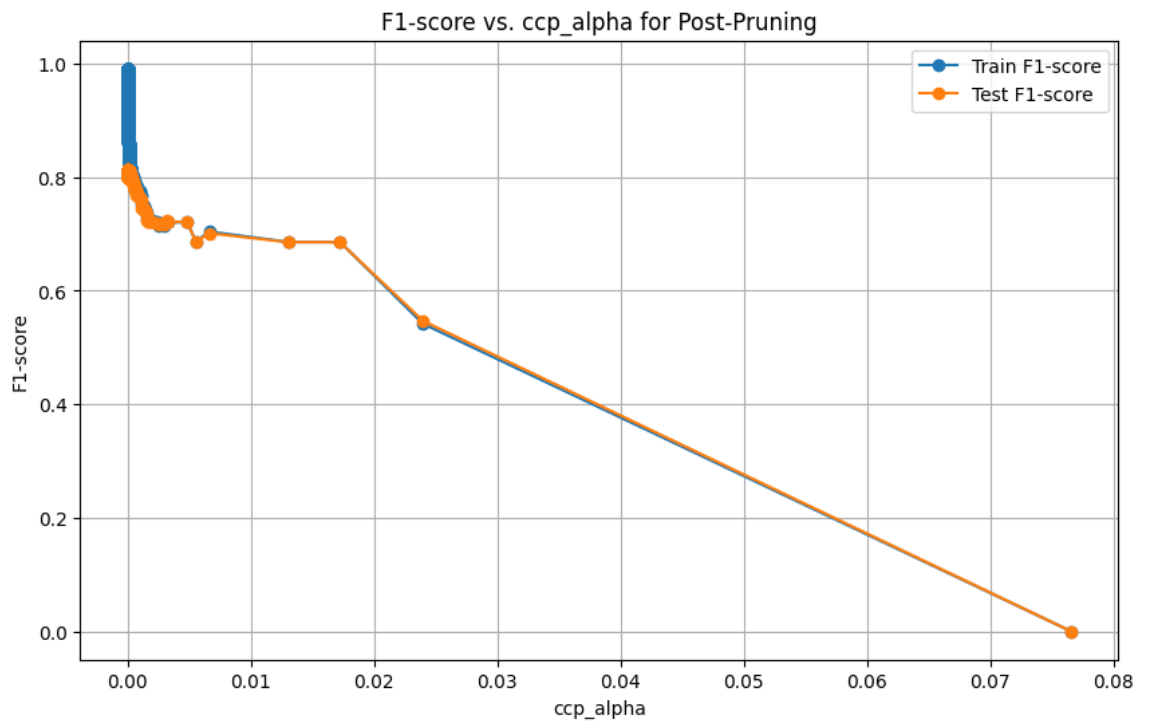
Trained 1352 decision trees.

```
In [90]: # Calculate F1-scores for training and testing sets for each model
train_f1_scores = []
test_f1_scores = []

for dt_model in dt_models:
    y_train_pred = dt_model.predict(X_train)
    y_test_pred = dt_model.predict(X_test)

    train_f1_scores.append(f1_score(y_train, y_train_pred))
    test_f1_scores.append(f1_score(y_test, y_test_pred))

# Plot F1-scores vs ccp_alpha
plt.figure(figsize=(10, 6))
plt.plot(ccp_alphas, train_f1_scores, marker='o', label='Train F1-score')
plt.plot(ccp_alphas, test_f1_scores, marker='o', label='Test F1-score')
plt.xlabel('ccp_alpha')
plt.ylabel('F1-score')
plt.title('F1-score vs. ccp_alpha for Post-Pruning')
plt.legend()
plt.grid(True)
plt.show()
```



```
In [91]: # Find the index of the maximum test F1-score
best_alpha_index = np.argmax(test_f1_scores)
best_ccp_alpha = ccp_alphas[best_alpha_index]
best_post_pruned_model = dt_models[best_alpha_index]

print(f"Optimal ccp_alpha based on test F1-score: {best_ccp_alpha:.6f}")

get_metrics_decision_tree(best_post_pruned_model, X_test, y_test)
```

Optimal ccp_alpha based on test F1-score: 0.000064

```
Out[91]: {'Accuracy': 0.8793531195442433,
          'Precision': 0.8194966734162569,
          'Recall': 0.8043725156161272,
          'F1 Score': 0.8118641639203324}
```

```
In [92]: # Train Performance for best post-pruned model
print("\nMetrics for the best post-pruned model (Training Set):")
get_metrics_decision_tree(best_post_pruned_model, X_train, y_train)
```

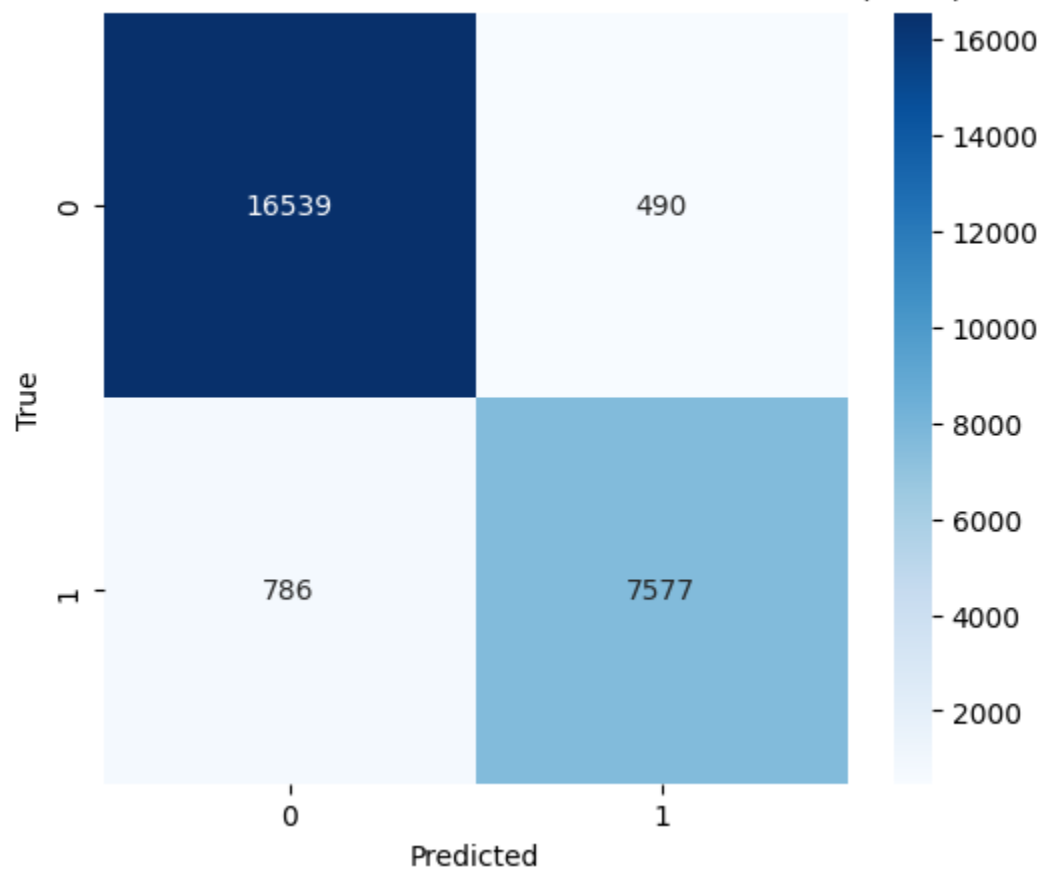
Metrics for the best post-pruned model (Training Set):

```
Out[92]: {'Accuracy': 0.949747952110901,
          'Precision': 0.9384977106793714,
          'Recall': 0.9068516082745426,
          'F1 Score': 0.9224033081975188}
```

```
In [93]: # Train Matrix for best post-pruned model
y_train_pred_post_pruned = best_post_pruned_model.predict(X_train)
```

```
plot_confusion_matrix(y_train, y_train_pred_post_pruned, model_name="Best  
Post-Pruned Decision Tree (Train)")
```

Confusion Matrix for Best Post-Pruned Decision Tree (Train)

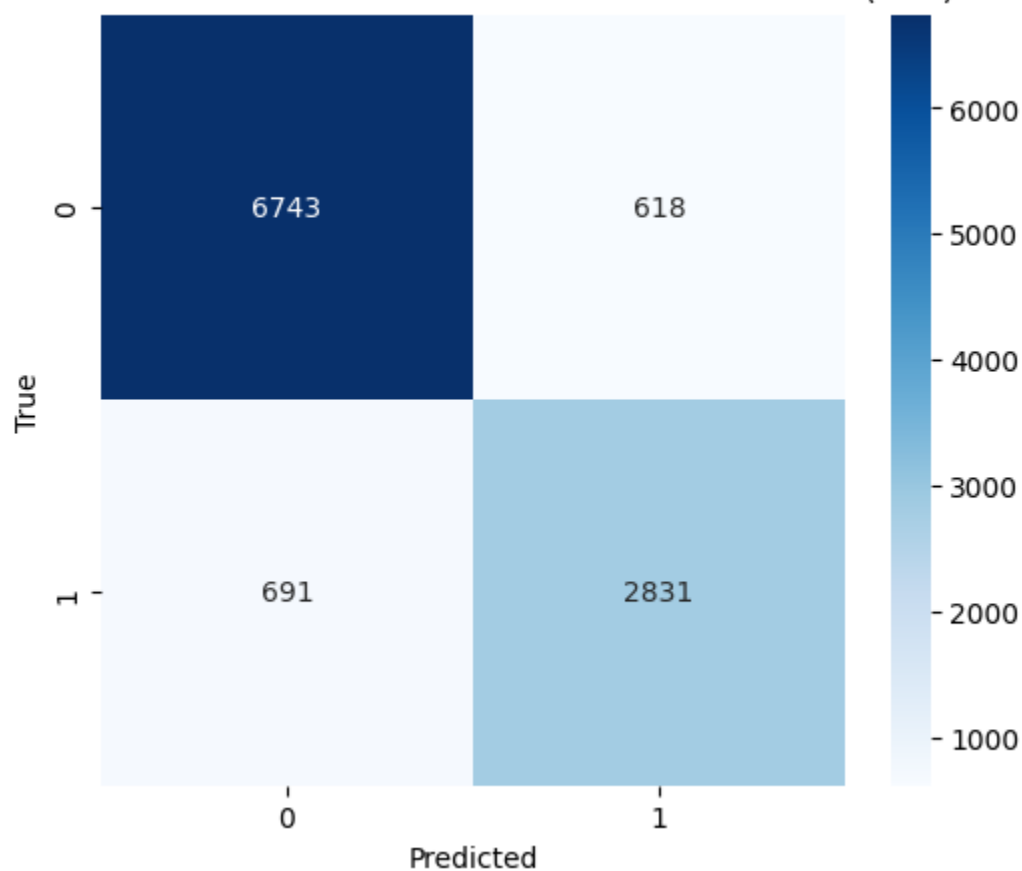


```
In [94]: # Test Performance for best post-pruned model  
get_metrics_decision_tree(best_post_pruned_model, X_test, y_test)
```

```
Out[94]: {'Accuracy': 0.8793531195442433,  
          'Precision': 0.8194966734162569,  
          'Recall': 0.8043725156161272,  
          'F1 Score': 0.8118641639203324}
```

```
In [95]: # Test Matrix for the best post-pruned model  
y_test_pred_post_pruned = best_post_pruned_model.predict(X_test)  
plot_confusion_matrix(y_test, y_test_pred_post_pruned, model_name="Best Post-  
Pruned Decision Tree (Test)")
```

Confusion Matrix for Best Post-Pruned Decision Tree (Test)



```
In [96]: # more complex and prepruning
plt.figure(figsize=(20, 15))
tree.plot_tree(best_post_pruned_model,
                feature_names=X_train.columns,
                class_names=['Not Canceled', 'Canceled'],
                filled=True,
                rounded=True,
                fontsize=10)
plt.title('Post-Pruned Decision Tree Visualization')
plt.show()
```



```
| | | | | | | | | | | | | --- arrival_month <= 11.50  
| | | | | | | | | | | | | |--- truncated branch of  
depth 10  
| | | | | | | | | | | | | --- arrival_month > 11.50  
| | | | | | | | | | | | | |--- class: Not Canceled  
| | | | | | | | | | | | | |--- avg_price_per_room > 135.00  
| | | | | | | | | | | | | |--- class: Canceled  
| | | | | | | | | | | | | |--- market_segment_type_Offline > 0.50  
| | | | | | | | | | | | | |--- class: Not Canceled  
| | | | | | | | | | | | | |--- no_of_weekend_nights > 0.50  
| | | | | | | | | | | | | |--- lead_time <= 74.50  
| | | | | | | | | | | | | |--- arrival_date <= 27.50  
| | | | | | | | | | | | | |--- arrival_month <= 9.50  
| | | | | | | | | | | | | |--- lead_time <= 59.50  
| | | | | | | | | | | | | |--- truncated branch of  
depth 8  
| | | | | | | | | | | | | |--- lead_time > 59.50  
| | | | | | | | | | | | | |--- truncated branch of  
depth 5  
| | | | | | | | | | | | | |--- arrival_month > 9.50  
| | | | | | | | | | | | | |--- class: Not Canceled  
| | | | | | | | | | | | | |--- arrival_date > 27.50  
| | | | | | | | | | | | | |--- lead_time <= 1.50  
| | | | | | | | | | | | | |--- lead_time <= 0.50  
| | | | | | | | | | | | | |--- class: Not Canceled  
| | | | | | | | | | | | | |--- lead_time > 0.50  
| | | | | | | | | | | | | |--- truncated branch of  
depth 2  
| | | | | | | | | | | | | |--- lead_time > 1.50  
| | | | | | | | | | | | | |--- avg_price_per_room <= 41.08  
| | | | | | | | | | | | | |--- class: Canceled  
| | | | | | | | | | | | | |--- avg_price_per_room > 41.08  
| | | | | | | | | | | | | |--- class: Not Canceled  
| | | | | | | | | | | | | |--- lead_time > 74.50  
| | | | | | | | | | | | | |--- lead_time <= 78.50  
| | | | | | | | | | | | | |--- avg_price_per_room <= 77.50  
| | | | | | | | | | | | | |--- no_of_adults <= 1.50  
| | | | | | | | | | | | | |--- class: Canceled  
| | | | | | | | | | | | | |--- no_of_adults > 1.50  
| | | | | | | | | | | | | |--- class: Not Canceled  
| | | | | | | | | | | | | |--- avg_price_per_room > 77.50  
| | | | | | | | | | | | | |--- no_of_week_nights <= 0.50  
| | | | | | | | | | | | | |--- class: Canceled  
| | | | | | | | | | | | | |--- no_of_week_nights > 0.50  
| | | | | | | | | | | | | |--- truncated branch of  
depth 2  
| | | | | | | | | | | | | |--- lead_time > 78.50  
| | | | | | | | | | | | | |--- total_nights <= 3.50  
| | | | | | | | | | | | | |--- lead_time <= 80.50  
| | | | | | | | | | | | | |--- truncated branch of  
depth 3  
| | | | | | | | | | | | | |--- lead_time > 80.50  
| | | | | | | | | | | | | |--- truncated branch of  
depth 2  
| | | | | | | | | | | | | |--- total nights > 3.50
```


[illegible]

[illegible]

```
|
depth 20
| | | | | | | | | | | | |--- truncated branch of
| | | | | | | | | | | | |--- required_car_parking_space >
0.50
| | | | | | | | | | | | |--- class: Not Canceled
| | | | | | | | | | | | |--- type_of_meal_plan_Meal Plan 2 >
0.50
| | | | | | | | | | | | |--- arrival_month <= 5.00
| | | | | | | | | | | | |--- class: Not Canceled
| | | | | | | | | | | | |--- arrival_month > 5.00
| | | | | | | | | | | | |--- class: Canceled
| | | | | | | | | | | | |--- type_of_meal_plan_Not Selected > 0.50
| | | | | | | | | | | | |--- required_car_parking_space <= 0.50
| | | | | | | | | | | | |--- arrival_year <= 2017.50
| | | | | | | | | | | | |--- total_nights <= 7.50
| | | | | | | | | | | | |--- class: Not Canceled
| | | | | | | | | | | | |--- total_nights > 7.50
| | | | | | | | | | | | |--- class: Canceled
| | | | | | | | | | | | |--- arrival_year > 2017.50
| | | | | | | | | | | | |--- no_of_adults <= 1.50
| | | | | | | | | | | | |--- truncated branch of
depth 3
| | | | | | | | | | | | |--- no_of_adults > 1.50
| | | | | | | | | | | | |--- truncated branch of
depth 12
| | | | | | | | | | | | |--- required_car_parking_space > 0.50
| | | | | | | | | | | | |--- class: Not Canceled
| | | | | | | | | | | | |--- avg_price_per_room > 105.27
| | | | | | | | | | | | |--- required_car_parking_space <= 0.50
| | | | | | | | | | | | |--- arrival_month <= 10.50
| | | | | | | | | | | | |--- avg_price_per_room <= 195.30
| | | | | | | | | | | | |--- lead_time <= 54.50
| | | | | | | | | | | | |--- arrival_year <= 2017.50
| | | | | | | | | | | | |--- arrival_month <= 9.50
| | | | | | | | | | | | |--- truncated branch of
depth 3
| | | | | | | | | | | | |--- arrival_month > 9.50
| | | | | | | | | | | | |--- class: Not Canceled
| | | | | | | | | | | | |--- arrival_year > 2017.50
| | | | | | | | | | | | |--- lead_time <= 33.50
| | | | | | | | | | | | |--- truncated branch of
depth 13
| | | | | | | | | | | | |--- lead_time > 33.50
| | | | | | | | | | | | |--- truncated branch of
depth 10
| | | | | | | | | | | | |--- lead_time > 54.50
| | | | | | | | | | | | |--- arrival_month <= 8.50
| | | | | | | | | | | | |--- lead_time <= 135.50
| | | | | | | | | | | | |--- truncated branch of
depth 15
| | | | | | | | | | | | |--- lead_time > 135.50
| | | | | | | | | | | | |--- truncated branch of
depth 7
| | | | | | | | | | | | |--- arrival_month > 8.50
| | | | | | | | | | | | |--- lead time <= 59.50
```

```

| | | | | | | | | | | | |--- truncated branch of
depth 2
| | | | | | | | | | | | |--- lead_time > 59.50
| | | | | | | | | | | | |--- truncated branch of
depth 6
| | | | | | | | | | | | |--- avg_price_per_room > 195.30
| | | | | | | | | | | | |--- type_of_meal_plan_Not Selected <=
0.50
| | | | | | | | | | | | |--- class: Canceled
| | | | | | | | | | | | |--- type_of_meal_plan_Not Selected >
0.50
| | | | | | | | | | | | |--- class: Not Canceled
| | | | | | | | | | | | |--- arrival_month > 10.50
| | | | | | | | | | | | |--- lead_time <= 22.50
| | | | | | | | | | | | |--- no_of_adults <= 1.50
| | | | | | | | | | | | |--- class: Canceled
| | | | | | | | | | | | |--- no_of_adults > 1.50
| | | | | | | | | | | | |--- class: Not Canceled
| | | | | | | | | | | | |--- lead_time > 22.50
| | | | | | | | | | | | |--- avg_price_per_room <= 168.06
| | | | | | | | | | | | |--- avg_price_per_room <= 147.75
| | | | | | | | | | | | |--- no_of_week_nights <= 3.50
| | | | | | | | | | | | |--- truncated branch of
depth 6
| | | | | | | | | | | | |--- no_of_week_nights > 3.50
| | | | | | | | | | | | |--- class: Not Canceled
| | | | | | | | | | | | |--- avg_price_per_room > 147.75
| | | | | | | | | | | | |--- class: Canceled
| | | | | | | | | | | | |--- avg_price_per_room > 168.06
| | | | | | | | | | | | |--- class: Not Canceled
| | | | | | | | | | | | |--- required_car_parking_space > 0.50
| | | | | | | | | | | | |--- no_of_weekend_nights <= 3.00
| | | | | | | | | | | | |--- class: Not Canceled
| | | | | | | | | | | | |--- no_of_weekend_nights > 3.00
| | | | | | | | | | | | |--- class: Canceled
| | | | | | | | | | | | |--- no_of_special_requests > 0.50
| | | | | | | | | | | | |--- no_of_special_requests <= 1.50
| | | | | | | | | | | | |--- market_segment_type_Online <= 0.50
| | | | | | | | | | | | |--- type_of_meal_plan_Not Selected <= 0.50
| | | | | | | | | | | | |--- lead_time <= 102.50
| | | | | | | | | | | | |--- total_nights <= 15.00
| | | | | | | | | | | | |--- class: Not Canceled
| | | | | | | | | | | | |--- total_nights > 15.00
| | | | | | | | | | | | |--- class: Canceled
| | | | | | | | | | | | |--- lead_time > 102.50
| | | | | | | | | | | | |--- lead_time <= 104.50
| | | | | | | | | | | | |--- no_of_week_nights <= 2.50
| | | | | | | | | | | | |--- class: Canceled
| | | | | | | | | | | | |--- no_of_week_nights > 2.50
| | | | | | | | | | | | |--- class: Not Canceled
| | | | | | | | | | | | |--- lead_time > 104.50
| | | | | | | | | | | | |--- avg_price_per_room <= 141.75
| | | | | | | | | | | | |--- class: Not Canceled
| | | | | | | | | | | | |--- avg_price_per_room > 141.75
| | | | | | | | | | | | |--- class: Canceled

```

```
| | | | |--- type_of_meal_plan_Not Selected > 0.50  
| | | | |--- lead_time <= 63.00  
| | | | |--- class: Not Canceled  
| | | | |--- lead_time > 63.00  
| | | | |--- class: Canceled  
| | | |--- market_segment_type_Online > 0.50  
| | | |--- lead_time <= 8.50  
| | | |--- lead_time <= 4.50  
| | | | |--- total_nights <= 14.00  
| | | | |--- class: Not Canceled  
| | | | |--- total_nights > 14.00  
| | | | |--- class: Canceled  
| | | |--- lead_time > 4.50  
| | | |--- room_type_reserved_Room_Type 2 <= 0.50  
| | | |--- avg_price_per_room <= 123.60  
| | | |--- class: Not Canceled  
| | | |--- avg_price_per_room > 123.60  
| | | |--- type_of_meal_plan_Not Selected <=  
0.50  
| | | | |--- arrival_date <= 15.50  
| | | | |--- avg_price_per_room <= 128.91  
| | | | |--- truncated branch of  
depth 2  
| | | | |--- avg_price_per_room > 128.91  
| | | | |--- class: Not Canceled  
| | | | |--- arrival_date > 15.50  
| | | | |--- class: Not Canceled  
| | | |--- type_of_meal_plan_Not Selected >  
0.50  
| | | | |--- total_nights <= 1.50  
| | | | |--- arrival_date <= 16.50  
| | | | |--- class: Not Canceled  
| | | | |--- arrival_date > 16.50  
| | | | |--- truncated branch of  
depth 2  
| | | | |--- total_nights > 1.50  
| | | | |--- arrival_month <= 9.50  
| | | | |--- class: Not Canceled  
| | | | |--- arrival_month > 9.50  
| | | | |--- class: Canceled  
| | | |--- room_type_reserved_Room_Type 2 > 0.50  
| | | |--- total_nights <= 3.50  
| | | |--- class: Not Canceled  
| | | |--- total_nights > 3.50  
| | | |--- class: Canceled  
| | | |--- lead_time > 8.50  
| | | |--- required_car_parking_space <= 0.50  
| | | |--- avg_price_per_room <= 127.62  
| | | |--- no_of_weekend_nights <= 2.50  
| | | |--- lead_time <= 43.50  
| | | |--- arrival_month <= 11.50  
| | | |--- arrival_month <= 1.50  
| | | |--- class: Not Canceled  
| | | |--- arrival_month > 1.50  
| | | |--- truncated branch of
```

```
depth 12
| | | | | | | | | | | | |--- arrival_month > 11.50
| | | | | | | | | | | | |--- class: Not Canceled
| | | | | | | | | | | | |--- lead_time > 43.50
| | | | | | | | | | | | |--- arrival_year <= 2017.50
| | | | | | | | | | | | |--- arrival_month <= 7.50
| | | | | | | | | | | | |--- truncated branch of
depth 3
| | | | | | | | | | | | |--- arrival_month > 7.50
| | | | | | | | | | | | |--- truncated branch of
depth 7
| | | | | | | | | | | | |--- arrival_year > 2017.50
| | | | | | | | | | | | |--- arrival_month <= 8.50
| | | | | | | | | | | | |--- truncated branch of
depth 17
| | | | | | | | | | | | |--- arrival_month > 8.50
| | | | | | | | | | | | |--- truncated branch of
depth 19
| | | | | | | | | | | | |--- no_of_weekend_nights > 2.50
| | | | | | | | | | | | |--- arrival_month <= 1.50
| | | | | | | | | | | | |--- class: Not Canceled
| | | | | | | | | | | | |--- arrival_month > 1.50
| | | | | | | | | | | | |--- avg_price_per_room <= 119.12
| | | | | | | | | | | | |--- no_of_week_nights <= 8.50
| | | | | | | | | | | | |--- truncated branch of
depth 4
| | | | | | | | | | | | |--- no_of_week_nights > 8.50
| | | | | | | | | | | | |--- class: Canceled
| | | | | | | | | | | | |--- avg_price_per_room > 119.12
| | | | | | | | | | | | |--- class: Not Canceled
| | | | | | | | | | | | |--- avg_price_per_room > 127.62
| | | | | | | | | | | | |--- lead_time <= 142.50
| | | | | | | | | | | | |--- arrival_month <= 8.50
| | | | | | | | | | | | |--- arrival_date <= 19.50
| | | | | | | | | | | | |--- avg_price_per_room <= 177.15
| | | | | | | | | | | | |--- truncated branch of
depth 10
| | | | | | | | | | | | |--- avg_price_per_room > 177.15
| | | | | | | | | | | | |--- truncated branch of
depth 4
| | | | | | | | | | | | |--- arrival_date > 19.50
| | | | | | | | | | | | |--- arrival_date <= 27.50
| | | | | | | | | | | | |--- truncated branch of
depth 9
| | | | | | | | | | | | |--- arrival_date > 27.50
| | | | | | | | | | | | |--- truncated branch of
depth 6
| | | | | | | | | | | | |--- arrival_month > 8.50
| | | | | | | | | | | | |--- arrival_month <= 11.50
| | | | | | | | | | | | |--- arrival_year <= 2017.50
| | | | | | | | | | | | |--- truncated branch of
depth 4
| | | | | | | | | | | | |--- arrival_year > 2017.50
| | | | | | | | | | | | |--- truncated branch of
depth 14
```



```
| | | | |--- avg_price_per_room > 202.95  
| | | | |--- class: Canceled  
|--- lead_time > 151.50  
| |--- avg_price_per_room <= 100.04  
| | |--- no_of_special_requests <= 0.50  
| | | |--- market_segment_type_Online <= 0.50  
| | | |--- no_of_adults <= 1.50  
| | | | |--- lead_time <= 163.50  
| | | | | |--- lead_time <= 162.50  
| | | | | |--- class: Not Canceled  
| | | | |--- lead_time > 162.50  
| | | | | |--- class: Canceled  
| | | | |--- lead_time > 163.50  
| | | | | |--- lead_time <= 341.00  
| | | | | | |--- lead_time <= 173.00  
| | | | | | | |--- arrival_date <= 3.50  
| | | | | | | |--- class: Not Canceled  
| | | | | | | |--- arrival_date > 3.50  
| | | | | | | |--- total_nights <= 3.00  
| | | | | | | |--- class: Canceled  
| | | | | | | |--- total_nights > 3.00  
| | | | | | | |--- class: Not Canceled  
| | | | | | |--- lead_time > 173.00  
| | | | | | | |--- arrival_month <= 5.50  
| | | | | | | |--- arrival_date <= 7.50  
| | | | | | | |--- class: Canceled  
| | | | | | | |--- arrival_date > 7.50  
| | | | | | | |--- class: Not Canceled  
| | | | | | |--- arrival_month > 5.50  
| | | | | | | |--- class: Not Canceled  
| | | | |--- lead_time > 341.00  
| | | | | |--- no_of_week_nights <= 4.00  
| | | | | | |--- lead_time <= 402.00  
| | | | | | | |--- class: Not Canceled  
| | | | | | | |--- lead_time > 402.00  
| | | | | | | |--- class: Canceled  
| | | | | | |--- no_of_week_nights > 4.00  
| | | | | | | |--- class: Canceled  
| | | | |--- no_of_adults > 1.50  
| | | | | |--- avg_price_per_room <= 84.58  
| | | | | | |--- lead_time <= 244.00  
| | | | | | | |--- total_nights <= 2.50  
| | | | | | | |--- lead_time <= 166.50  
| | | | | | | |--- class: Not Canceled  
| | | | | | | |--- lead_time > 166.50  
| | | | | | | |--- arrival_date <= 19.00  
| | | | | | | | |--- avg_price_per_room <= 80.38  
| | | | | | | | |--- class: Canceled  
| | | | | | | | |--- avg_price_per_room > 80.38  
| | | | | | | | |--- class: Not Canceled  
| | | | | | | | |--- arrival_date > 19.00  
| | | | | | | | |--- class: Not Canceled  
| | | | | | |--- total_nights > 2.50  
| | | | | | | |--- no_of_week_nights <= 2.50  
| | | | | | | |--- class: Not Canceled
```

[illegible]

```

| | | |--- market_segment_type_Online > 0.50
| | | | |--- avg_price_per_room <= 2.50
| | | | | |--- no_of_weekend_nights <= 0.50
| | | | | | |--- lead_time <= 205.00
| | | | | | |--- class: Not Canceled
| | | | | | |--- lead_time > 205.00
| | | | | | |--- class: Canceled
| | | | | |--- no_of_weekend_nights > 0.50
| | | | | | |--- class: Not Canceled
| | | | | |--- avg_price_per_room > 2.50
| | | | | |--- class: Canceled
| | | |--- no_of_special_requests > 0.50
| | | | |--- no_of_weekend_nights <= 0.50
| | | | | |--- lead_time <= 180.50
| | | | | | |--- lead_time <= 159.50
| | | | | | |--- arrival_month <= 8.50
| | | | | | |--- class: Not Canceled
| | | | | | |--- arrival_month > 8.50
| | | | | | |--- class: Canceled
| | | | | |--- lead_time > 159.50
| | | | | | |--- no_of_adults <= 0.50
| | | | | | |--- class: Canceled
| | | | | | |--- no_of_adults > 0.50
| | | | | | |--- arrival_date <= 1.50
| | | | | | |--- no_of_week_nights <= 1.50
| | | | | | |--- class: Canceled
| | | | | | |--- no_of_week_nights > 1.50
| | | | | | |--- class: Not Canceled
| | | | | | |--- arrival_date > 1.50
| | | | | | |--- class: Not Canceled
| | | | |--- lead_time > 180.50
| | | | |--- market_segment_type_Offline <= 0.50
| | | | | |--- no_of_special_requests <= 2.50
| | | | | | |--- arrival_month <= 11.50
| | | | | | |--- total_nights <= 0.50
| | | | | | |--- class: Not Canceled
| | | | | | |--- total_nights > 0.50
| | | | | | |--- class: Canceled
| | | | | | |--- arrival_month > 11.50
| | | | | | |--- lead_time <= 300.50
| | | | | | |--- lead_time <= 226.50
| | | | | | |--- class: Canceled
| | | | | | |--- lead_time > 226.50
| | | | | | |--- lead_time <= 272.00
| | | | | | |--- class: Not Canceled
| | | | | | |--- lead_time > 272.00
| | | | | | |--- class: Canceled
| | | | | |--- lead_time > 300.50
| | | | | |--- class: Canceled
| | | | |--- no_of_special_requests > 2.50
| | | | |--- class: Not Canceled
| | | | |--- market_segment_type_Offline > 0.50
| | | | |--- no_of_adults <= 2.50
| | | | |--- no_of_adults <= 1.50
| | | | |--- class: Canceled

```



```
| | | | | | | | | | | | |--- class: Not Canceled  
| | | | | | | | | | | | |--- no_of_week_nights > 2.50  
| | | | | | | | | | | | |--- no_of_week_nights <= 4.50  
| | | | | | | | | | | | |--- lead_time <= 281.50  
| | | | | | | | | | | | |--- class: Not Canceled  
| | | | | | | | | | | | |--- lead_time > 281.50  
| | | | | | | | | | | | |--- class: Canceled  
| | | | | | | | | | | | |--- no_of_week_nights > 4.50  
| | | | | | | | | | | | |--- no_of_special_requests <=  
2.50  
| | | | | | | | | | | | |--- truncated branch of  
depth 2  
| | | | | | | | | | | | |--- no_of_special_requests >  
2.50  
| | | | | | | | | | | | |--- class: Not Canceled  
| | | | | | | | | | | | |--- no_of_week_nights > 9.50  
| | | | | | | | | | | | |--- lead_time <= 198.00  
| | | | | | | | | | | | |--- class: Not Canceled  
| | | | | | | | | | | | |--- lead_time > 198.00  
| | | | | | | | | | | | |--- class: Canceled  
| |--- avg_price_per_room > 100.04  
| | |--- arrival_month <= 11.50  
| | | |--- no_of_special_requests <= 2.50  
| | | | |--- class: Canceled  
| | | |--- no_of_special_requests > 2.50  
| | | | |--- class: Not Canceled  
| | |--- arrival_month > 11.50  
| | | |--- no_of_special_requests <= 0.50  
| | | | |--- class: Not Canceled  
| | | |--- no_of_special_requests > 0.50  
| | | | |--- arrival_date <= 24.50  
| | | | |--- class: Not Canceled  
| | | | |--- arrival_date > 24.50  
| | | | |--- lead_time <= 172.50  
| | | | |--- class: Not Canceled  
| | | | |--- lead_time > 172.50  
| | | | |--- class: Canceled
```

```
In #lead time still most important but has lowered compared to pre pruning
[98]: print("\nFeature Importance for Post-Pruned Decision Tree Model:")
      importance_df_post_pruned = pd.DataFrame({'Feature': X_train.columns,
      'Importance': best_post_pruned_model.feature_importances_})
      importance_df_post_pruned =
      importance_df_post_pruned.sort_values(by='Importance', ascending=False)
      display(importance_df_post_pruned)
```

Feature Importance for Post-Pruned Decision Tree Model:

	Feature	Importance
6	lead_time	0.37769

	Feature	Importance
13	avg_price_per_room	0.16321
28	market_segment_type_Online	0.11197
14	no_of_special_requests	0.08109
9	arrival_date	0.06093
8	arrival_month	0.05985
3	no_of_weekend_nights	0.02859
1	no_of_adults	0.02582
15	total_nights	0.02469
4	no_of_week_nights	0.02266
7	arrival_year	0.01257
5	required_car_parking_space	0.00803
18	type_of_meal_plan_Not Selected	0.00506
21	room_type_reserved_Room_Type 4	0.00493
27	market_segment_type_Offline	0.00339
16	type_of_meal_plan_Meal Plan 2	0.00318
19	room_type_reserved_Room_Type 2	0.00185
2	no_of_children	0.00169
22	room_type_reserved_Room_Type 5	0.00143
12	no_of_previous_bookings_not_canceled	0.00044
10	repeated_guest	0.00035
25	market_segment_type_Complementary	0.00020
23	room_type_reserved_Room_Type 6	0.00019
24	room_type_reserved_Room_Type 7	0.00018
0	const	0.00000
11	no_of_previous_cancellations	0.00000
17	type_of_meal_plan_Meal Plan 3	0.00000
20	room_type_reserved_Room_Type 3	0.00000
26	market_segment_type_Corporate	0.00000

Model Performance Comparison and Conclusions

```
In [99]: #Train set comparison
pre_pruned_train_metrics = get_metrics_decision_tree(best_dt_model, X_train,
y_train)
post_pruned_train_metrics = get_metrics_decision_tree(best_post_pruned_model,
X_train, y_train)

comparison_df_train = pd.DataFrame({
    'Train Pre-Pruned Model': pre_pruned_train_metrics,
    'Train Post-Pruned Model': post_pruned_train_metrics
})
# Post pruning has higher acc
display(comparison_df_train.round(4))
```

	Train Pre-Pruned Model	Train Post-Pruned Model
Accuracy	0.86290	0.94970
Precision	0.77110	0.93850
Recall	0.83030	0.90690
F1 Score	0.79960	0.92240

```
In [100]: #Test Set Comparison
pre_pruned_test_metrics = get_metrics_decision_tree(best_dt_model, X_test,
y_test)
post_pruned_test_metrics = get_metrics_decision_tree(best_post_pruned_model,
X_test, y_test)

comparison_df_train = pd.DataFrame({
    'Test Pre-Pruned Model': pre_pruned_test_metrics,
    'Test Post-Pruned Model': post_pruned_test_metrics
})
# Post pruning has higher acc
display(comparison_df_train.round(4))
```

	Test Pre-Pruned Model	Test Post-Pruned Model
Accuracy	0.85250	0.87940
Precision	0.74920	0.81950
Recall	0.81830	0.80440
F1 Score	0.78220	0.81190

Actionable Insights and Recommendations

- What profitable policies for cancellations and refunds can the hotel adopt?
- What other recommendations would you suggest to the hotel?

Actionable Insights and Recommendations

Key Insights from EDA and Models:

- * High Cancellation Rate: Approximately 32.7% of bookings are canceled, which is a significant portion.
- * Lead Time is Crucial: The lead_time is consistently identified as the most important feature influencing cancellations. Bookings with longer lead times have a higher propensity to be canceled.
- * Special Requests Reduce Cancellations: Customers making 1 or 2 special requests are less likely to cancel (around 23% and 15% cancellation rates, respectively), compared to those with no special requests (around 43% cancellation rate). Bookings with 3 or more special requests have almost no cancellations.
- * Repeat Guests are Loyal: Nearly 98% of repeated guests do not cancel their bookings, highlighting their loyalty.
- * Market Segment Influence: The 'Online' market segment has the highest average room price and contributes significantly to bookings, while 'Offline' bookings have a lower cancellation rate when comparing against 'Online' bookings in the pruned decision tree.
- * Family Bookings (Adults ≥ 2 , Children ≥ 1): There is a notable number of cancellations within family bookings (around 37% cancellation rate in this segment).
- * Arrival Month Impact: Certain months (e.g., October, September, August) are busier, and cancellation patterns might vary by month.

Profitable Policies for Cancellations and Refunds:

1. Dynamic Deposit/Prepayment Policies based on Lead Time:

- * For bookings with long lead times, consider implementing a non-refundable deposit system. A small non-refundable deposit could be required at the time of booking, with an option for a larger non-refundable deposit closer to the arrival date for a slight discount. This encourages commitment for the booking
- * For bookings with short lead times, offer more flexible cancellation policies to attract last-minute bookings, as these are less likely to be canceled.

2. Incentivize Special Requests:

* Encourage guests to make special requests during the booking process. This could be framed as a personalized service benefit. The data suggests that guests with special requests are more committed.

3. Loyalty Programs for Repeat Guests:

* Strengthen and promote loyalty programs to encourage repeat guests. Since they have a very low cancellation rate, nurturing this segment is highly profitable.

Other Recommendations:

1. Enhance Modeling: Continuously monitor and update the predictive model with new data to maintain its accuracy in identifying potential cancellations.

2. Feedback for Canceled Bookings: Implement a short, optional survey for guests who cancel to understand their reasons. This qualitative data can provide deeper insights beyond what the quantitative data offers and help refine policies.